

KỸ THUẬT, TỐI ƯU VÀ TRIỂN KHAI

TÓM TẮT

Phần D là tầng công cụ: tự viết forward và backward bằng NumPy để hiểu, rồi PyTorch để làm việc, rồi nén mô hình, triển khai, và mức MLOps tối thiểu đủ để sáu tháng sau dựng lại được kết quả của mình. Đọc xong bạn đưa được một mô hình từ notebook ra thiết bị thật với ngân sách latency cho trước. Nếu chỉ nhớ một điều: đây là phần hết hạn nhanh nhất của cây, trừ một ngoại lệ — việc tự viết backward pass thuộc tầng bền vững và nên làm ngay sau chương 12, không đợi tới đây.

Phần này là tầng công cụ, và nó được đặt sau phần C có chủ ý: đây là phần hết hạn nhanh nhất của cây, nên đầu tư vào nó trước khi có nền của phần B và C là đầu tư vào thứ sẽ cũ. Nguyên tắc chi phối là học đủ để làm việc, không đầu tư sâu, và tra cứu khi cần thay vì ghi nhớ.

Có một ngoại lệ quan trọng, và nó nằm ngay ở chương đầu. Việc tự viết forward và backward pass bằng NumPy, tự chạy kiểm tra gradient bằng sai phân hữu hạn, trông giống một bài tập công cụ nhưng thực ra thuộc tầng [T2]–[T3]. Nó xây trực giác về đồ thị tính toán, về nơi gradient chết, và về cái giá bộ nhớ của activation — ba thứ không hết hạn và không thể học được bằng cách gọi một hàm có sẵn. Vì vậy chương 19 chia làm hai giai đoạn rõ rệt: giai đoạn tự viết để hiểu, rồi mới tới giai đoạn dùng framework để làm việc.

Ba chương còn lại đi theo trình tự của một hệ thống thật rồi khỏi máy nghiên cứu. Chương 20 là tối ưu và nén mô hình, với một thứ tự thử nghiệm thực dụng và lý do của thứ tự đó, vì phần lớn người mới bắt đầu từ kỹ thuật hào nhoáng nhất thay vì từ kỹ thuật rẻ nhất. Chương 21 là inference và triển khai, nơi những con số đo được trong notebook thường không sống sót, và nơi cách đo latency sai là một trong những lỗi phổ biến nhất. Chương 22 là tái lập và mức MLOps tối thiểu: đủ để sáu tháng sau còn dựng lại được đúng kết quả của mình, không hơn.

Bảng 1. Bản đồ Phần D: các chương, nội dung, và mức đầu tư đề xuất.

Ch.	Thư mục	Nội dung	Mức
19	19-numpy-toi-pytorch	Hai giai đoạn: tự viết forward và backward bằng NumPy; rồi PyTorch thành thạo	giai đoạn 1 cốt lõi , giai đoạn 2 công cụ
20	20-toi-uu-va-nen-mo-hin	Quantization, pruning, distillation, chọn độ phân giải; thứ tự thử nghiệm thực dụng	công cụ
21	21-inference-va-trien-k	Export, runtime, batching, đo latency đúng cách, ràng buộc thiết bị biên	công cụ
22	22-tai-lap-va-mlops	Seed và nguồn bất định, version hoá, theo dõi thí nghiệm, giám sát sau triển khai	cốt lõi ở mức tối thiểu

Giai đoạn 1 của chương 19 nên làm ngay sau chương 12, không đợi tới phần D. Ba chương còn lại đọc khi thật sự phải triển khai, vì chúng hết hạn nhanh nhất trong cây.

Từ NumPy tự viết tới PyTorch thành thạo (From Hand-Written NumPy to Fluent PyTorch)

PHỤ THUỘC A/01-toan-toi-thieu/02-vi-phan-va-toi-uu (quy tắc chuỗi); B/09-thiet-ke-ham-mat-mat (*hinge loss*, cross-entropy); B/11-gioi-han-tinh-toan (*roofline* — bắt buộc trước khi đọc phần tăng tốc của mục 19.2); C/12-co-che-huan-luyen (BatchNorm, dropout, optimizer).

TÓM TẮT

Chương này có hai giai đoạn đọc theo đúng thứ tự: tự cài **lan truyền ngược** (*backpropagation*) bằng NumPy để có trực giác về cơ chế, rồi học PyTorch theo mạch vấn đề → công cụ để có năng suất. Đọc xong bạn debug được những lỗi không có thông báo lỗi: gradient sai, trạng thái train/eval lẫn lộn, rò bộ nhớ vì logging. Bạn cũng tăng tốc được một vòng lặp huấn luyện dựa trên số đo, thay vì dựa trên mẹo nghe được. Nếu chỉ nhớ một điều: giai đoạn 1 là bài tập *trực giác* tăng [T2][T3], làm đúng một lần rồi bỏ. Giai đoạn 2 là bài tập *công cụ* tăng [T4], phải cập nhật liên tục. Lẫn lộn hai loại này là cách phổ biến nhất để phí thời gian.

KIẾN THỨC CẦN CÓ TRƯỚC

- **lượt xuôi** (*forward pass*) và **lượt ngược** (*backward pass*) — forward là lượt tính từ đầu vào ra loss; lượt ngược là lượt tính ngược để lấy đạo hàm của loss theo từng tham số. Mọi khung huấn luyện đều là vòng lặp hai lượt này.
- **gradient** — vector đạo hàm của loss theo tham số; nó chỉ hướng làm loss *tăng* nhanh nhất, nên ta đi ngược lại nó.
- **lan truyền ngược** (*backpropagation*) — thuật toán tính toàn bộ gradient bằng một lượt đi ngược qua đồ thị tính toán, tốn cùng bậc chi phí với một lượt xuôi — lý do duy nhất khiến huấn luyện mạng sâu khả thi.
- **learning rate** — độ dài mỗi bước cập nhật trọng số theo hướng ngược gradient. Quá lớn thì loss phát tán, quá nhỏ thì huấn luyện không bao giờ xong; đây là siêu tham số quan trọng nhất.
- **framework** — thư viện lo hộ bạn đồ thị tính toán, lan truyền ngược tự động, quản lý thiết bị và dữ liệu — PyTorch là ví dụ. Giá trị của chương này nằm ở chỗ biết framework đang giấu cái gì.
- **tăng bán rã [T2] so với [T4]** — [T2] là kiến thức nguyên lý, không hết hạn khi công cụ đổi; [T4] là kiến thức công cụ, sẽ cũ trong vài năm. Giai đoạn 1 của chương là [T2]–[T3], giai đoạn 2 là [T4].

CÂU HỎI MỞ ĐẦU

- Q1. Một mạng 10^9 tham số huấn luyện được trong vài ngày, trong khi vi phân thuận cần 10^9 lượt chạy — lượt ngược lấy khoản tiết kiệm chín bậc đó ở đâu ra?
- Q2. Số trọng số y hệt nhau, vậy vì sao huấn luyện tốn bộ nhớ gấp nhiều lần suy luận?
- Q3. Kiểm tra gradient của bạn qua sạch mọi tầng nhưng mạng vẫn không học — phép kiểm đó theo thiết kế đã bỏ sót cái gì?
- Q4. Bạn bật torch.compile và AMP, mọi thứ chạy đúng, nhưng thời gian một *epoch* không đổi một giây nào. Vì sao đó không phải là bug?

Framework che giấu rất nhiều thứ, và phần lớn thời gian đó là điều tốt. Nhưng có một tập nhỏ những thứ mà không tự tay làm một lần thì bạn sẽ không bao giờ debug nổi. Chúng không hiện ra dưới dạng thông báo lỗi, chỉ dưới dạng một con số hơi lệch hoặc một đường cong loss hơi chậm. Câu hỏi dẫn dắt của chương là: *tập nhỏ ấy gồm những gì, và làm tới đâu thì đủ?*

Chương chia làm hai mục vì hai giai đoạn có *bản chất khác nhau*, và nhằm lẫn giữa chúng là cái bẫy chính:

- **19.1 — Giai đoạn 1, tự viết bằng NumPy** [T2][T3]. Mục tiêu là trực giác về cơ chế, không phải kỹ năng công cụ. Bạn sẽ không dùng lại một dòng NumPy nào trong việc thật. Có tiêu chí dừng rõ ràng, và khi đạt thì *dừng hẳn*.
- **19.2 — Giai đoạn 2, PyTorch** [T4]. Mục tiêu là năng suất và khả năng chẩn đoán. Nội dung sẽ cũ khi API đổi, nhưng danh sách các nút thắt mà công cụ gỡ thì không cũ.

Thứ tự phụ thuộc chỉ đi một chiều. Mục 19.2 giới thiệu autograd, buffer và cặp `train()/eval()` như *tên gọi mới* của những thứ bạn vừa tự cài bằng tay. Đọc ngược thì chúng thành từ vựng phải học thuộc, thay vì thành khái niệm đã hiểu. Ngược lại, nếu bạn đã từng viết một solver phi tuyến với Jacobian giải tích và bước so sai phân, giai đoạn 1 sẽ nhanh hơn bạn nghĩ. Nó là cùng một kỹ năng, đặt vào miền mới.

Một sợi chỉ chạy suốt cả chương: hầu hết bug ở đây là bug về *trạng thái*, không phải về công thức. Ba câu hỏi lặp lại ở cả hai giai đoạn. Cái gì được nhớ giữa lượt xuôi và lượt ngược? Cái gì được lưu vào checkpoint? Cái gì đổi hành vi giữa `train` và `eval`? Ở giai đoạn 1 chúng hiện ra dưới dạng “tôi phải cache gì”; ở giai đoạn 2 chúng mang tên `parameter` so với `buffer`.

Bản đồ chương 19 — Từ NumPy tự viết tới PyTorch thành thạo

Chương này chia làm hai mục vì hai giai đoạn có *bản chất khác nhau* và có tuổi thọ khác nhau. Nhầm lẫn giữa chúng là cái bẫy chính của cả chương.

19.1 — Giai đoạn 1: tự viết bằng NumPy [T2][T3] (*cốt lõi; làm đúng một lần rồi bỏ*). Cài linear classifier với cả hinge lẫn softmax, viết lượt ngược giải tích cho sáu tầng, kiểm tra gradient số cho từng tầng trước khi ghép, rồi ghép thành mạng n tầng với SGD + momentum. Đây *không* phải bài tập công cụ — mục tiêu là trực giác về cơ chế, thứ không hết hạn khi framework đổi. Tiêu chí dừng rất rõ: giải thích được vì sao BatchNorm khiến lượt ngược tốn thêm bộ nhớ, và vì sao dropout phải scale lúc train. Đạt rồi thì chuyển sang 19.2 và không viết NumPy nữa.

19.2 — Giai đoạn 2: PyTorch thành thạo [T4] (*cốt lõi phần ánh xạ triệu chứng → công cụ; tham khảo phần cú pháp*). Chín nút thắt và công cụ gỡ từng cái: stride và view/copy, autograd, `parameter` so với `buffer`, `DataLoader`, tự viết vòng lặp một lần, AMP và DDP và `torch.compile`, profiler, tính tắt định, và cuối cùng là kernel tùy biến (biết là đủ). Phần bèn của mục này là cây quyết định tăng tốc; phần sẽ cũ là tên API.

Vì sao chia như vậy và thứ tự bắt buộc. 19.2 giới thiệu autograd, buffer, `train()/eval()` như tên gọi mới của thứ bạn vừa tự cài trong 19.1. Đọc ngược sẽ biến chúng thành từ vựng phải học thuộc thay vì khái niệm đã hiểu.

Sợi chỉ xuyên chương. Hầu hết bug ở đây là bug về *trạng thái*, không phải công thức: cái gì được nhớ giữa lượt xuôi và lượt ngược, cái gì được lưu vào checkpoint, cái gì đổi hành vi giữa hai chế độ.

Phụ thuộc lên trên. A/01-toan-toi-thieu/02-vi-phan-va-toi-uu (quy tắc chuỗi), B/09-thiet-ke-ham-mat-mat, B/11-gioi-han-tinh-toan (bắt buộc trước phần tăng tốc), C/12-co-che-huan-luyen.

Toả xuống dưới. [D/20-toi-uu-va-nen-mo-hinh](#) dùng lại đúng những đòn bẩy này nhưng cho **suy luận** (*inference*); [D/21-inference-va-trien-khai](#) dùng lại kỹ năng phân biệt “lệch do bug” với “lệch do số học”; [D/22-tai-lap-va-mlops](#) mở rộng mục về tính tất định thành một chương.

I 1.1 Giai đoạn 1: tự viết bằng NumPy (Building It Yourself in NumPy)

PHỤ THUỘC [A/01-toan-toi-thieu/02-vi-phan-va-toi-uu](#) (quy tắc chuỗi, Jacobian); [B/09-thiet-ke-ham-mat-mat](#) (*hinge loss* và *cross-entropy*); [C/12-co-che-huan-luyen/01-co-hoc-cua-gradient](#) và [02-lam-cho-training-chay-duoc](#) (BatchNorm, dropout — ở đây ta cài lại chúng bằng tay).

TÓM TẮT

Mục này là bài tập một lần: tự cài **lan truyền ngược** (*backpropagation*) bằng NumPy cho sáu tầng, kiểm tra từng tầng bằng **kiểm tra gradient số** (*numerical gradient check*), rồi ghép thành một mạng chạy được. Nó không phải bài tập công cụ mà là bài tập trực giác tầng [T2][T3]: đọc xong bạn giải thích được vì sao bộ nhớ huấn luyện lớn hơn bộ nhớ **suy luận** (*inference*) nhiều lần, vì sao quên `eval()` lại đổi kết quả, và bạn tự chẩn đoán được một gradient sai thay vì đoán mò. Nếu chỉ nhớ một điều: không ghép một tầng chưa qua kiểm tra gradient — vì kiểm từng tầng giữ không gian nghi vấn ở đúng một phần tử, còn ghép trước kiểm sau làm nó phình lên thành số tầng nhân số tương tác.

KIẾN THỨC CẦN CÓ TRƯỚC

- **hàm mất mát (loss function)** — hàm biến đầu ra của mô hình và đáp án đúng thành một số thực đo mức sai. Huấn luyện là cực tiểu hoá số đó.
- **gradient và quy tắc chuỗi** — gradient là vector đạo hàm riêng của loss theo từng tham số. Vì mạng là hàm hợp của nhiều tầng, đạo hàm được tính bằng quy tắc chuỗi — nhân dồn các đạo hàm cục bộ từ đầu ra ngược về đầu vào.
- **kích hoạt (activation)** — đầu ra trung gian của một tầng. Lượng bộ nhớ huấn luyện phần lớn là chỗ chứa các kích hoạt này, vì lượt ngược cần đọc lại chúng.
- **ReLU, dropout, BatchNorm** — ReLU là hàm phi tuyến $\max(0, x)$; dropout tắt ngẫu nhiên một phần kích hoạt lúc huấn luyện để chống *overfitting*; BatchNorm chuẩn hoá kích hoạt theo thống kê của *batch* để việc huấn luyện ổn định hơn. Cả ba đều có hành vi *khác nhau* giữa lúc train và lúc test.
- **batch** — nhóm mẫu được xử lý cùng lúc trong một bước cập nhật. Kích thước batch ảnh hưởng tới bộ nhớ, tốc độ, và — với BatchNorm — tới cả kết quả.
- **SGD và momentum** — *stochastic gradient descent* cập nhật trọng số theo gradient của một batch; *momentum* cộng dồn hướng đi của các bước trước để giảm dao động và vượt qua vùng phẳng.
- **float32 so với float64** — hai độ chính xác dấu phẩy động. float32 có epsilon máy $\approx 1,2 \times 10^{-7}$, float64 khoảng $2,2 \times 10^{-16}$; khác biệt này quyết định việc kiểm tra gradient số có nói thật hay không.

A. Đặt vấn đề

Framework che giấu rất nhiều thứ, và phần lớn thời gian đó là điều tốt — không ai muốn viết lại `conv2d` mỗi sáng. Nhưng có một tập nhỏ những thứ mà nếu không tự tay làm một lần thì bạn sẽ không bao giờ debug nổi, vì triệu chứng của chúng không bao giờ xuất hiện dưới dạng một thông báo lỗi. Mạng vẫn chạy, loss vẫn giảm, chỉ là giảm chậm hơn đáng ra, hoặc kết quả lúc `eval()` lệch khỏi lúc train một cách không giải thích được. Không có exception nào để bắt, không có *stack trace* nào để đọc.

Cần nói rõ ngay từ đầu về *loại* bài tập này, vì hiểu sai loại là cách phổ biến nhất để lãng phí một tháng:

- **Đây không phải bài tập tăng bốn** — không phải bài tập công cụ. Bạn sẽ không dùng lại một dòng NumPy nào trong công việc thật, và đó là chuyện bình thường.
- **Đây là bài tập tăng hai và ba** [T2][T3]: mục tiêu là *trực giác về cơ chế*, thứ không hết hạn khi framework đổi phiên bản, khi PyTorch bị thay bằng cái khác, hay khi kiến trúc thời thượng đổi tên.
- **Làm một lần, đạt tiêu chí dừng, rồi không quay lại.** Cái bấy quen thuộc của người học theo CS231n là yêu quá trình này và tiếp tục viết NumPy sau khi nó đã hết tác dụng — lúc đó nó thành một sở thích thú công chứ không còn là học.

Học xong thì làm được gì mà trước đó không làm được? Bạn nhìn một đường cong loss bất thường và khoanh vùng được ngay nó là lỗi gradient, lỗi trạng thái hay lỗi dữ liệu; bạn đọc thông báo hết bộ nhớ GPU và biết tầng nào đang giữ activation; và bạn viết được một `autograd.Function` tùy biến trong PyTorch mà không phải sao chép mẫu từ diễn đàn.

Người đọc đã viết C++ và CUDA có một lợi thế ở đây, và cũng có một cái bấy riêng. Lợi thế là bạn đã quen với việc một hệ thống số học có thể sai ở mức không nhìn thấy được. Cái bấy là bạn dễ bỏ qua bước kiểm tra gradient vì thấy nó buồn tẻ. Đúng như một người viết solver phi tuyến để bỏ bước so Jacobian giải tích với Jacobian sai phân, rồi mất ba ngày cho một dấu trừ.

TRỰC GIÁC

Bạn đã làm chính xác bài tập này rồi, chỉ là ở lĩnh vực khác. Khi thêm một loại *residual* mới vào một bài toán bình phương tối thiểu phi tuyến, bạn viết Jacobian giải tích để solver chạy nhanh, rồi *trước khi tin nó*, bạn so với Jacobian tính bằng sai phân hữu hạn trên vài điểm ngẫu nhiên. Nếu sai số tương đối nhỏ thì dùng; nếu không thì bạn biết chắc lỗi nằm trong đúng khối vừa viết, chứ không phải ở solver.

Lan truyền ngược là cùng một việc, chỉ khác quy mô: một mạng n tầng là một hàm hợp, **lượt ngược** (*backward pass*) là chế độ nghịch của quy tắc chuỗi — đúng cái mà cộng đồng tối ưu gọi là *adjoint method* — và kiểm tra gradient là bước so Jacobian. Giai đoạn 1 vì thế không dạy bạn khái niệm mới; nó chuyển một kỹ năng bạn đã có sang miền mới, ở quy mô nhỏ tới mức mọi con số còn nhìn được bằng mắt.

B. Bước 1 — linear classifier với hai hàm mất mát

Bắt đầu ở chỗ đơn giản nhất mà vẫn còn thú vị: một phân loại tuyến tính $s = Wx + b$, cài cả *hinge loss* kiểu SVM lẫn *softmax loss* (*softmax cross-entropy loss*), viết cả **lượt xuôi** (*forward pass*) lẫn lượt ngược cho từng cái. Điều bạn sẽ thấy tận mắt — chứ không phải đọc thấy — là hai hàm này có *hành vi gradient khác nhau về bản chất*, không chỉ khác về công thức.

Ví dụ nhỏ nhất. Một mẫu, ba lớp, điểm số $s = (2,0; 0,5; -1,0)$, nhãn đúng là lớp 0.

- **Hinge loss**, lẽ $\Delta = 1$:

$$L = \sum_{j \neq y} \max(0, s_j - s_y + \Delta) = \max(0, -0,5) + \max(0, -2,0) = 0.$$

Loss bằng không, và *gradient cũng bằng đúng không* — mọi phần tử của $\partial L / \partial s$ là 0. Mẫu này đã thoả lẽ; nó không còn nói gì với mô hình nữa.

- **Softmax loss**, cùng bộ điểm số: $e^{2,0} = 7,389$, $e^{0,5} = 1,649$, $e^{-1,0} = 0,368$, tổng 9,406, nên $p = (0,786; 0,175; 0,039)$ và $L = -\ln 0,786 = 0,241$. Gradient có dạng đẹp đến mức đáng nhớ,

$$\partial L / \partial s = p - e_y:$$

$$\partial L / \partial s = (-0,214; +0,175; +0,039).$$

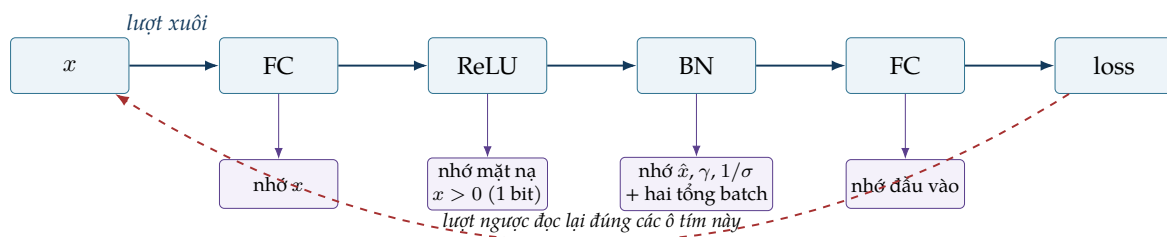
Ý nghĩa của công thức $p - e_y$: gradient chính là “sai lệch giữa niềm tin hiện tại và sự thật”. Không phần tử nào bằng không, nên softmax *luôn* còn đây — nó không có khái niệm “đủ tốt rồi”.

Khác biệt này giải thích rất nhiều hành vi thực nghiệm. Hinge tạo gradient thừa nên ổn định và ít bị chi phối bởi vài mẫu dễ, nhưng nó dừng học ngay khi lẽ được thoả, kể cả khi bạn còn muốn mô hình tự tin hơn. Softmax luôn có gradient nên tiếp tục tinh chỉnh, nhưng chính vì thế nó nhạy với nhãn sai — một mẫu bị gán nhầm sẽ bơm gradient vào suốt quá trình huấn luyện, không bao giờ tắt. Ai đã đọc B/09-thiet-ke-ham-mat-mat sẽ nhận ra đây là cùng trục lập luận đã dẫn tới **focal loss**.

↔ cùng nguyên lý với B/09 — cùng trục lập luận đã dẫn tới focal loss — dời trọng số gradient khỏi những mẫu không còn dạy được gì.

C. Bước 2 — lượt ngược giải tích cho sáu tầng

Bước tiếp theo là viết lượt ngược giải tích cho **fully-connected**, ReLU, dropout, BatchNorm, **convolution** và **max-pooling**. Giá trị của bước này không nằm ở các công thức — chúng có sẵn ở mọi nơi — mà ở chỗ nó buộc bạn trả lời cho từng tầng một câu hỏi mà framework không bao giờ hỏi bạn: *backward của tầng này cần nhớ gì từ lượt xuôi?* Câu trả lời là toàn bộ lý do bộ nhớ huấn luyện lớn hơn bộ nhớ suy luận nhiều lần. Hình 1.1 vẽ đúng cơ chế đó.



Hình 1.1. Vì sao huấn luyện tốn bộ nhớ hơn suy luận: mỗi tầng phải *ghi lại* một phần trạng thái lúc lượt xuôi để lượt ngược đọc lại. Ô nhớ là thứ không giải phóng được cho tới khi lượt ngược chạy qua. ReLU rẻ (một bit mỗi phần tử), BatchNorm đắt nhất vì ngoài $\hat{x}$ nó còn ràng buộc cả **batch**.

Bảng 1.1. Sáu tầng, và thứ mà lượt ngược của mỗi tầng buộc phải giữ lại từ lượt xuôi.

Tầng	Backward cần nhớ	Chi phí	Chỗ hay sai
Fully-connected	Đầu vào x (cho $\partial L / \partial W$)	Bằng kích thước đầu vào	Quên chuyển vị; quên cộng dồn $\partial L / \partial b$ theo batch
ReLU	Chỉ mặt nạ $x > 0$ (1 bit mỗi phần tử)	Rất rẻ	Xử lý $x = 0$ tùy tiện — chính là điểm gây làm kiểm tra gradient báo động giả
Dropout	Mặt nạ đã dùng lúc lượt xuôi	Bằng kích thước đầu ra	Không dùng inverted dropout , hoặc sinh mặt nạ mới ở lượt ngược
BatchNorm	$\hat{x}, \gamma, 1/\sigma$ — và lượt ngược khớp cả batch	Đắt nhất trong sáu tầng	Coi lượt ngược là phép theo từng phần tử
Convolution	Ma trận im2col hoặc đầu vào để dựng lại nó	Rất đắt nếu vật chất hoá im2col	col2im phải cộng dồn vùng chồng lấn, không phải ghi đè
Max-pool	Chỉ số argmax của mỗi cửa sổ	Rẻ	Xử lý trường hợp bằng nhau khác giữa hai cài đặt

Ba dòng trong Bảng 1.1 đáng dừng lại.

BatchNorm – lượt ngược không phải phép theo từng phần tử

$$\frac{\partial L}{\partial x_i} = \frac{\gamma}{N\sigma} \left[N \frac{\partial L}{\partial y_i} - \sum_j \frac{\partial L}{\partial y_j} - \hat{x}_i \sum_j \frac{\partial L}{\partial y_j} \hat{x}_j \right].$$

Công thức này nói điều gì? Rằng gradient của mẫu i bị trừ đi hai đại lượng tính trên *toàn batch* — một cho trung bình, một cho phương sai. Nói cách khác: BatchNorm không cho phép một mẫu tự do đẩy gradient của nó; mọi mẫu trong batch cùng thoả hiệp. Ba hệ quả bạn sẽ gặp lại suốt sự nghiệp:

- **Bộ nhớ lượt ngược tăng** vì phải giữ cả $\hat{x}$ lẫn các thống kê, và không giải phóng được theo từng mẫu.
- **Batch nhỏ làm gradient nhiễu**, vì ước lượng σ nhiễu — lý do GroupNorm và LayerNorm tồn tại [1], [2].
- **Mô hình huấn luyện với BatchNorm là hàm của cả batch, không phải của từng mẫu** — lý do sâu xa khiến `train()` và `eval()` cho kết quả khác nhau.

Dropout – vì sao phải scale lúc train

Cài theo kiểu *inverted dropout*: lúc train, giữ lại phần tử với xác suất $1 - p$ rồi chia cho $1 - p$; lúc test không làm gì cả. Lý do là bảo toàn kỳ vọng — nếu $\tilde{y} = m \odot y / (1 - p)$ với $m \sim \text{Bernoulli}(1 - p)$ thì $\mathbb{E}[\tilde{y}] = y$, nên đường suy luận của các tầng sau không bị dịch khi ta bật tắt dropout. Cách còn lại (nhân $1 - p$ lúc test) tương đương về toán học nhưng dở về kỹ thuật, vì nó đặt một bước bắt buộc vào đường suy luận — và mọi đường inference đều có ngày bị viết lại, bị *export*, bị port sang C++, và bước ấy sẽ rơi mất. Một điểm trung thực cần ghi: *inverted dropout* chỉ khớp *mô-men bậc một*; phương sai lúc train vẫn lớn hơn lúc test, và đó là một phần lý do dropout tương tác xấu với BatchNorm [3].

Convolution – chỗ nào sinh ra bất định trên GPU

Cách cài dễ hiểu nhất là *im2col*: trải các cửa sổ thành cột rồi gọi một phép nhân ma trận, nên $\partial L / \partial W = \partial L / \partial Y \cdot X_{\text{col}}^\top$ và $\partial L / \partial X_{\text{col}} = W^\top \cdot \partial L / \partial Y$. Bước cuối — *col2im* — phải cộng dồn gradient vào những vị trí mà các cửa sổ chồng lên nhau. Ghi nhớ chi tiết này. Phép cộng dồn tán xạ đó chính là chỗ các cài đặt GPU dùng `atomicAdd`. Cộng số thực dấu phẩy động không có tính kết hợp, nên thứ tự luồng hoàn thành quyết định bit cuối. Đây là một trong những nguồn bất định chính của huấn luyện trên GPU, chủ đề của [D/22-tai-lap-va-mlops](#). Nó cùng một họ nguyên nhân với các lỗi bất định do thứ tự con trỏ mà bạn đã gặp trong SLAM.

↪ **tiền đề D/22** — cộng số thực không có tính kết hợp, nên mọi phép cộng dồn tán xạ trên GPU là một nguồn bất định — cùng họ nguyên nhân với thứ tự con trỏ trong một hệ thống C++ đa luồng.

D. Bước 3 – kiểm tra gradient, và kỷ luật đi kèm

Mọi công thức ở trên đều có thể sai, và sai một cách im lặng. Cách duy nhất phát hiện là so với sai phân trung tâm:

$$\frac{\partial f}{\partial x_k} \approx \frac{f(x + he_k) - f(x - he_k)}{2h},$$

Ta nhiễu *một* toạ độ về hai phía và xem loss thay đổi bao nhiêu. Rồi tính sai số tương đối $|g_{\text{số}} - g_{\text{giải tích}}| / \max(|g_{\text{số}}|, |g_{\text{giải tích}}|)$.

```
def grad_check(f, x, analytic, h=1e-5, n=20):
```

```
# f: hàm tra ve scalar loss; analytic: gradient đã tính giai tích
idx = np.random.choice(x.size, size=n, replace=False)
for k in idx:
    old = x.flat[k]
    x.flat[k] = old + h; fp = f(x)
    x.flat[k] = old - h; fm = f(x)
    x.flat[k] = old
    num = (fp - fm) / (2 * h)
    ana = analytic.flat[k]
    rel = abs(num - ana) / max(1e-12, abs(num), abs(ana))
print(k, num, ana, rel)
```

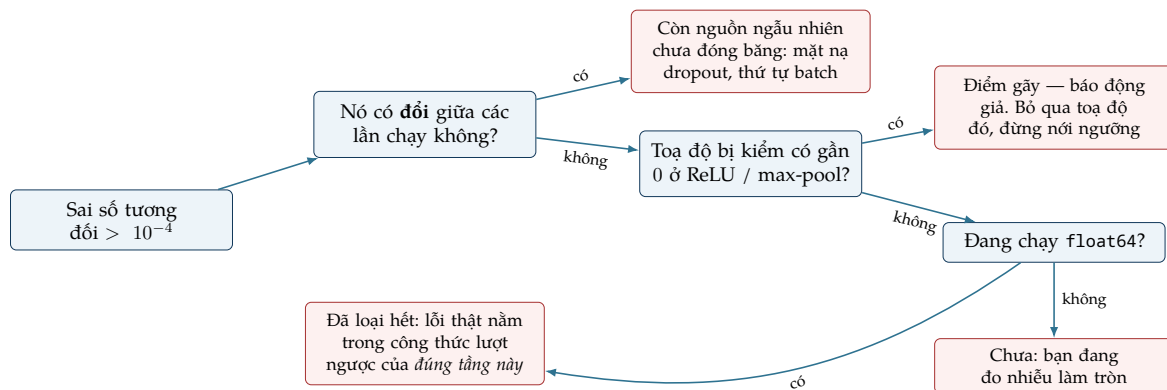
Ngưỡng diễn giải, theo *kinh nghiệm thực hành* chứ không phải kết quả lý thuyết:

- $< 10^{-7}$ — yên tâm.
- 10^{-7} đến 10^{-4} — chấp nhận được với mạng sâu có nhiều điểm gãy, đáng nghi với một tầng đơn giản.
- $> 10^{-4}$ — gần như chắc chắn có lỗi. $> 10^{-2}$ — chắc chắn có lỗi.

Ba chi tiết quyết định việc kiểm tra gradient nói thật hay nói dối:

1. **Phải dùng float64.** Sai phân trung tâm lấy hiệu hai số gần bằng nhau; với float32 (epsilon máy $\approx 1,2 \times 10^{-7}$) và $h = 10^{-5}$, bạn mất phần lớn chữ số có nghĩa vì mất chữ số có nghĩa khi trừ hai số gần bằng nhau — kết quả là bạn đang đo nhiễu làm tròn chứ không đo đạo hàm.
2. **Kiểm ở vài chục tọa độ ngẫu nhiên**, không phải toàn bộ tensor — vừa đủ tin cậy, vừa đủ nhanh để bạn thực sự chạy nó.
3. **Mọi nguồn ngẫu nhiên phải bị đóng băng giữa hai lần gọi f** ; nếu không, bạn đang so đạo hàm của hai hàm khác nhau.

Khi con số báo động, Hình 1.2 là thứ tự chẩn đoán tiết kiệm nhất.



Hình 1.2. Thứ tự chẩn đoán khi kiểm tra gradient báo động. Ba nhánh đầu là các nguyên nhân *giả*; chỉ khi loại hết chúng thì kết luận “công thức lượt ngược sai” mới có giá trị.

Kỷ luật đi kèm chỉ có một câu, và nó là phần đáng giá nhất của cả mục: *không ghép một tầng chưa qua kiểm tra*. Lý do thuần túy là số học tổ hợp. Ghép sáu tầng rồi mới phát hiện gradient sai thì không gian nghi vấn là sáu tầng cộng mọi tương tác giữa chúng; kiểm từng tầng ngay khi viết xong thì không gian nghi vấn luôn có đúng một phần tử. Đây chính xác là lập luận bạn đã dùng khi thêm từng loại ràng buộc vào một **đồ thị nhân tố** (*factor graph*) và kiểm tra sau mỗi lần thêm.

↪ **trường hợp riêng của A/01** — lan truyền ngược là chế độ nghịch của quy tắc chuỗi, nên kiểm tra gradient ở đây đúng là bước so Jacobian giải tích với Jacobian sai phân trong tối ưu phi tuyến.

E. Bước 4 — ghép thành mạng và tự viết vòng lặp

Bước cuối là ghép các tầng đã kiểm thành một mạng n tầng và tự viết vòng lặp huấn luyện với SGD kèm *momentum*. Ở đây bạn gặp một loại lỗi mới mà kiểm từng tầng không bắt được: lỗi về *trạng thái*.

- **Mặt nạ dropout** phải giống nhau giữa lượt xuôi và lượt ngược của cùng một bước, nhưng khác nhau giữa các bước.
- **BatchNorm** phải cập nhật thống kê chạy ở chế độ train và dùng thống kê ấy ở chế độ test.
- **Bộ đệm momentum** phải được cập phát một lần, không phải mỗi bước.

Ba lỗi này là ba phiên bản của cùng một câu hỏi: *cái gì là tham số, cái gì là trạng thái, cái gì là tạm*. Chính câu hỏi đó quay lại nguyên vẹn ở mục sau, dưới cái tên parameter so với buffer.

F. Tiêu chí dừng, giới hạn và trường hợp hỏng

Tiêu chí dừng nên phát biểu bằng năng lực giải thích, không bằng số bài đã làm. Bạn dừng được khi giải thích trôi chảy hai điều. Thứ nhất, vì sao BatchNorm khiến lượt ngược tốn thêm bộ nhớ: hai tổng trên toàn batch, nên không giải phóng được theo từng mẫu. Thứ hai, vì sao dropout phải scale lúc train chứ không lúc test: để bảo toàn kỳ vọng mà không đặt bước bắt buộc nào lên đường suy luận. Đạt rồi thì chuyển sang PyTorch và không viết NumPy nữa.

Giới hạn của cả giai đoạn cần nói thẳng:

- **Mọi trực giác về hiệu năng rút ra từ đây đều sai.** NumPy thuần chậm hơn nhiều bậc so với kernel thật, nên bạn chỉ chạy được ở quy mô đồ chơi; phần hiệu năng phải học lại ở B/11-gioi-han-tinh-toan.
- **Không học được gì về ổn định số học ở độ chính xác thấp.** *Log-sum-exp*, *loss scaling*, tràn số fp16 chỉ xuất hiện khi bạn rời float64.
- **Kiểm tra gradient kiểm tính nhất quán, không kiểm tính đúng.** Một tầng cài sai công thức lượt xuôi nhưng có lượt ngược khớp với chính nó sẽ qua kiểm một cách hoàn hảo.

CẠM BÃY

Bốn cách làm cho kiểm tra gradient qua trong khi mã vẫn sai, xếp theo mức độ hay gặp. **Một:** quên cố định mặt nạ dropout, khiến $f(x + h)$ và $f(x - h)$ là hai hàm khác nhau — triệu chứng là sai số lớn và *nhảy loạn giữa các lần chạy*. **Hai:** để BatchNorm ở chế độ train trong lúc kiểm, khiến hàm bị kiểm phụ thuộc thống kê batch và thay đổi khi bạn nhiễu một toạ độ. **Ba:** kiểm ở đúng điểm gãy của ReLU hoặc max-pool, tức x rất gần 0. Ở đó sai phân hai phía đo qua một chỗ hàm không khả vi, nên báo động là giả. Cách xử lý đúng là bỏ qua toạ độ đó, không phải nới ngưỡng. **Bốn:** chạy ở float32 rồi kết luận rằng “sai số 10^{-3} là bình thường”; không, đó là dấu hiệu bạn đang đo nhiễu làm tròn.

CẦN NHỚ

Không ghép một tầng chưa qua kiểm tra gradient. Lý do không phải sự cẩn thận mà là tổ hợp: kiểm từng tầng giữ không gian nghi vấn ở đúng một phần tử; ghép trước rồi kiểm sau làm nó phình lên thành số tầng nhân với số tương tác. Và nhớ rằng kiểm tra gradient chỉ chứng minh lượt xuôi và lượt ngược *khớp nhau*, không chứng minh lượt xuôi đúng. [T2]

G. Kết nối

Mục này là tiền đề bắt buộc của [D/19/02-pytorch-thanh-thao](#). Ba khái niệm bạn vừa cài bằng tay — đồ thị tính toán, thứ phải nhớ giữa hai lượt, và tham số so với trạng thái — sẽ xuất hiện lại dưới dạng autograd, nn.Module và cặp `train()/eval()`. Bạn sẽ hiểu chúng như tên gọi mới của thứ đã biết. Nó là trường hợp riêng của [A/01-toan-toi-thieu/02-vi-phan-va-toi-uu](#): backpropagation chỉ là chế độ nghịch của quy tắc chuỗi áp cho hàm hợp [4]. Nó bổ sung cho [C/12-co-che-huan-luyen](#): chương kia nói BatchNorm và dropout *làm gì* cho việc huấn luyện, mục này nói chúng *tồn gì* và hỏng ở đâu. Và nó gieo trước một hạt cho [D/22-tai-lap-va-mlops](#): phép cộng dồn tán xạ trong *col2im* là nguồn bất định GPU mà bạn sẽ gặp lại ở đó.

■ Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao hinge loss cho gradient bằng đúng không ở một mẫu đã thoả lề, còn softmax thì không bao giờ? Hệ quả nào của điều đó xuất hiện khi tập dữ liệu có nhãn sai?
2. Hai tổng trên toàn batch trong lượt ngược của BatchNorm là gì, và chúng buộc ta giữ thêm gì trong bộ nhớ?
3. Vì sao *inverted dropout* tốt hơn về mặt kỹ thuật dù hai cách scale tương đương về toán học? Và nó *không* bảo toàn thứ gì?
4. Kiểm tra gradient cho sai số tương đối 3×10^{-3} và con số đó đổi mỗi lần chạy. Nguyên nhân khả dĩ nhất là gì, và nếu con số ổn định thì danh sách nghi vấn đổi thế nào?
5. Kiểm tra gradient qua hết mọi tầng nhưng mạng vẫn không học — hai loại lỗi nào mà kiểm tra gradient theo thiết kế không thể bắt được?
6. Chỗ nào trong lượt ngược của convolution là phép cộng dồn tán xạ, và vì sao chi tiết đó sẽ quay lại ám bạn khi chạy trên GPU?

Nguồn

- [1] Y. Wu and K. He, “Group Normalization,” in *ECCV*, 2018.
- [2] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer Normalization,” *arXiv:1607.06450*, 2016.
- [3] N. Srivastava, G. Hinton, A. Krizhevsky, et al., “Dropout: A Simple Way to Prevent Neural Networks from Overfitting,” *JMLR*, vol. 15, pp. 1929–1958, 2014.
- [4] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986.
- [5] S. Rajbhandari, J. Rasley, O. Ruwase, et al., “ZeRO: Memory Optimizations Toward Training Trillion Parameter Models,” *SC*, 2020.
- [6] P. Goyal, P. Dollár, R. Girshick, et al., “Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour,” *arXiv:1706.02677*, 2017.
- [7] P. Micikevicius, S. Narang, et al., “Mixed Precision Training,” in *ICLR*, 2018.
- [8] G. Hinton, O. Vinyals, and J. Dean, “Distilling the Knowledge in a Neural Network,” in *NeurIPS Deep Learning Workshop*, 2015.
- [9] M. Nagel, M. Fournarakis, R. A. Amjad, et al., “A White Paper on Neural Network Quantization,” *arXiv:2106.08295*, 2021.
- [10] B. Jacob, S. Kligys, B. Chen, et al., “Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference,” in *CVPR*, 2018.
- [11] S. Han, H. Mao, and W. J. Dally, “Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding,” in *ICLR*, 2016.
- [12] J. Frankle and M. Carbin, “The Lottery Ticket Hypothesis: Finding Sparse, Trainable Neural Networks,” in *ICLR*, 2019.
- [13] H. Touvron, M. Cord, M. Douze, et al., “Training Data-Efficient Image Transformers and Distillation Through Attention,” in *ICML*, 2021.
- [14] A. G. Howard, M. Zhu, et al., “MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications,” *arXiv:1704.04861*, 2017.
- [15] M. Tan and Q. V. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” in *ICML*, 2019.
- [16] D. Sculley, G. Holt, D. Golovin, et al., “Hidden Technical Debt in Machine Learning Systems,” *NeurIPS*, 2015.
- [17] N. Bodla, B. Singh, R. Chellappa, et al., “Soft-NMS: Improving Object Detection with One Line of Code,” in *ICCV*, 2017.
- [18] P. Henderson, R. Islam, P. Bachman, et al., “Deep Reinforcement Learning That Matters,” *AAAI*, 2018.

1.2 Giai đoạn 2: PyTorch thành thạo (Becoming Fluent in PyTorch)

PHỤ THUỘC `D/19/01-tu-viet-bang-numpy` (bắt buộc — mọi khái niệm ở đây là tên gọi mới của thứ bạn đã cài bằng tay); `B/11-gioi-han-tinh-toan` (*roofline*, memory-bound so với compute-bound — không có nó thì mục về tăng tốc chỉ là danh sách mọ).

TÓM TẮT

Mục này đi theo mạch vấn đề → công cụ: mỗi tính năng của PyTorch được giới thiệu qua đúng cái bug hoặc cái nút thắt đã sinh ra nó, chứ không theo thứ tự tài liệu API. Đọc xong bạn đọc được một *profile* và biết nên bật *AMP*, sửa *DataLoader*, hay gọi `torch.compile` — thay vì bật cả ba rồi không biết cái nào có tác dụng. Nếu chỉ nhớ một điều: mọi công cụ tăng tốc ở đây chỉ giúp khi bạn đang nghẽn ở đúng chỗ nó gỡ, nên thứ tự bắt buộc luôn là *đo trước, sửa sau*.

KIẾN THỨC CẦN CÓ TRƯỚC

- **tensor** — mảng nhiều chiều kèm kiểu dữ liệu (`dtype`) và thiết bị (`device`, CPU hay GPU). Trong bộ nhớ nó chỉ là một dải liên tục cộng với bộ *stride* nói mỗi chiều nhảy bao nhiêu phần tử.
- **đồ thị tính toán (computation graph)** — bản ghi các phép toán đã thực hiện ở lượt xuôi, để lượt ngược biết đi ngược theo đường nào. PyTorch dựng nó *động*, tức mỗi lần lượt xuôi là một đồ thị mới.
- **epoch, batch, DataLoader** — một *epoch* là một lượt đi hết dữ liệu; *batch* là nhóm mẫu xử lý cùng lúc; `DataLoader` là bộ máy đọc đĩa, giải mã, augment và gom mẫu thành batch — thường chạy trên CPU và thường là chỗ nghẽn.
- **độ chính xác hỗn hợp (mixed precision)** — tính phần lớn phép toán ở 16 bit trong khi giữ trọng số và một số phép cộng dồn ở 32 bit. Mục đích là giảm số byte phải nạp qua bộ nhớ, không phải giảm số phép tính.
- **memory-bound so với compute-bound** — một kernel memory-bound bị chặn bởi băng thông bộ nhớ (làm ít phép tính trên mỗi byte nạp về), compute-bound thì bị chặn bởi năng lực tính toán. Hầu hết mạng thị giác là memory-bound — lý do *AMP* có tác dụng.
- **profiler** — công cụ đo thời gian thực sự của từng kernel và từng giai đoạn, thay vì đoán. Trong PyTorch là `torch.profiler`; ở tầng dưới là `Nsight`.
- **checkpoint và state_dict** — `state_dict` là từ điển chứa toàn bộ trọng số và *buffer* của mô hình; lưu nó ra đĩa được gọi là checkpoint. Nó *không* chứa mã nguồn, nên vẫn cần đúng phiên bản code để nạp lại.
- **tính tất định (determinism)** — tính chất chạy lại cùng đầu vào cho ra cùng đầu ra tới từng bit. Trên GPU nó không mặc định, và ép nó thì phải trả giá bằng tốc độ.

A. Đặt vấn đề

Sau giai đoạn 1 bạn biết cơ chế nhưng chưa có công cụ. PyTorch cung cấp công cụ, nhưng nó cung cấp *rất nhiều* công cụ, và tài liệu chính thức tổ chức theo API chứ không theo vấn đề. Hệ quả là kiểu hiểu biết phổ biến nhất: biết gọi `amp.autocast` mà không biết nó giúp gì; bật `torch.compile` rồi thấy không nhanh hơn và kết luận rằng “compile không ăn thua”.

Mục này đảo lại thứ tự trình bày. Mỗi công cụ xuất hiện sau đúng cái vấn đề đã sinh ra nó, vì trong nghề, thứ bạn thực sự cần nhớ không phải tên hàm — tra được trong ba mươi giây — mà là *ánh xạ từ triệu chứng sang công cụ*. Cần nói rõ đây là nội dung tăng bốn [T4]: tên API sẽ đổi, và điều đó bình thường; cái không đổi là danh sách các nút thắt.

↪ **tiền đề D/19/01** — `autograd`, `buffer` và cặp `train/eval` chỉ là tên gọi mới của đồ thị tính toán, trạng thái và công tắc chế độ mà bạn đã tự cài bằng tay.

Học xong thì làm được gì? Bạn cầm một vòng lặp huấn luyện chậm gấp ba lần đáng ra, chạy *profiler*, và chỉ đúng một dòng cần sửa — đây chính là kỹ năng bạn đã có với `nsys` và `perf` ở phía C++/CUDA, lần này áp cho một *runtime* có thêm một tầng trừu tượng ở giữa.

TRỰC GIÁC

Autograd là một cuốn băng ghi âm chỉ dùng được một lần. Mỗi phép toán trên *tensor* có `requires_grad=True` ghi thêm một mục vào băng; `backward()` tua ngược băng, tính gradient, rồi *xoá* băng đi. Từ hình ảnh đó suy ra ba điều mà người ta thường phải học bằng cách vấp: gọi `backward()` hai lần thì lỗi vì băng đã bị xoá (trừ khi bạn xin `retain_graph`); giữ một *tensor* còn dính băng trong một danh sách log nghĩa là giữ luôn toàn bộ đoạn băng phía sau nó, nên bộ nhớ phình dần suốt *epoch*; và `detach()` chỉ đơn giản là cắt băng ở đúng chỗ đó.

B. Mạch vấn đề → công cụ

Chín nút thắt, theo thứ tự bạn sẽ gặp

- 1 Vấn đề** bug im lặng về bộ nhớ và tính đúng đắn.

Giải pháp hiểu *tensor* ở mức bố cục bộ nhớ: `dtype`, `device`, *stride* (mỗi bước theo một chiều nhảy bao nhiêu phần tử trong bộ nhớ), và khác biệt *view* với *copy*.

Hệ quả `.view()` thất bại trên *tensor* không liên mạch, còn `.reshape()` âm thầm sao chép — một `.contiguous()` đặt sai chỗ có thể nhân đôi bộ nhớ mà không báo gì.
- 2 Vấn đề** không biết đồ thị được dựng và giải phóng lúc nào.

Giải pháp *autograd*: `backward`, `no_grad`, `detach`, và `autograd.Function` tùy biến khi bạn có công thức gradient riêng.

Vấn đề mới bug kinh điển: giữ *tensor* còn đồ thị trong một list để logging, làm rò bộ nhớ suốt *epoch*.
- 3 Vấn đề** trạng thái mô hình không chỉ là trọng số.

Giải pháp `nn.Module`: phân biệt *parameter* với *buffer*, hiểu `state_dict`, dùng *hook*, và cặp `train()/eval()`.

Giả định bạn nhớ rằng thống kê chạy của `BatchNorm` là *buffer* chứ không phải tham số — nó được lưu, được chuyển `device`, nhưng không được optimizer đụng tới.
- 4 Vấn đề** GPU đói dữ liệu.

Giải pháp `Dataset/DataLoader/sampler`, `num_workers`, `pin_memory`, *prefetch*, `collate_fn`; và một tầng *augmentation* (`albumentations`, `transforms v2`, hoặc `DALI` khi CPU không kịp).

Hệ quả đây là nút thắt phổ biến nhất và cũng bị bỏ qua nhiều nhất — xem B/11-gioi-han-tinh-toan.
- 5 Vấn đề** framework giấu quá nhiều.

Giải pháp tự viết vòng lặp huấn luyện ít nhất một lần trước khi dùng `Lightning` hay `HF Trainer`.

Giá phải trả vài giờ.

Hệ quả sau đó dùng framework là hợp lý, vì bạn đã biết nó đang làm gì và biết chỗ để chọc vào khi nó làm sai.
- 6 Vấn đề** bộ nhớ và tốc độ.

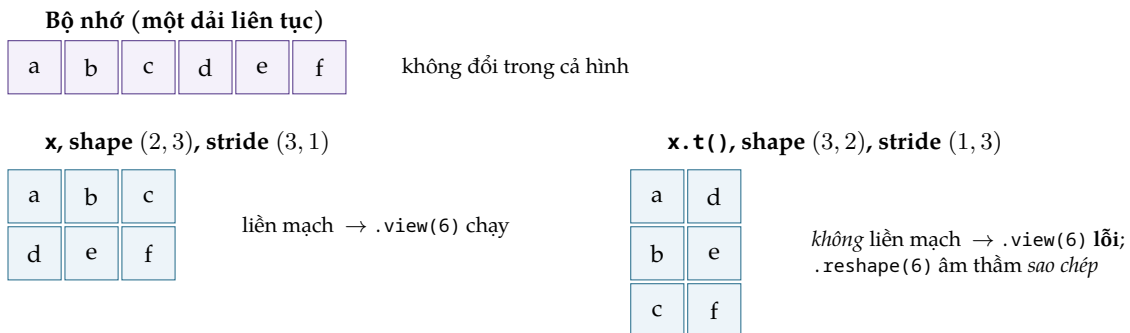
Giải pháp *AMP* (tính phần lớn phép toán ở 16 bit, giữ trọng số ở 32 bit) với `GradScaler`; *DDP* (chia batch cho nhiều GPU, mỗi GPU giữ một bản sao mô hình)

qua `torchrun`, hoặc FSDP khi một GPU không chứa nổi mô hình [5]; và `torch.compile` để hợp nhất kernel và giảm chi phí điều phối.

- 7 Vấn đề** đo trước khi tối ưu.
Giải pháp `torch.profiler`, Nsight, trace viewer.
Giá định bạn tin số đo hơn tin trực giác — nguyên tắc đã có ở B/11-gioi-han-tinh-toan.
- 8 Vấn đề** kết quả không lặp lại.
Giải pháp `seed`, `torch.use_deterministic_algorithms`, tắt `cudnn.benchmark` — kèm cái giá về tốc độ.
Giá phải trả đáng kể, và *vẫn không* đảm bảo tái lập giữa hai máy khác GPU. Toàn bộ chủ đề này thuộc [D/22-tai-lap-va-mlops](#).
- Hệ quả** biết là đủ: viết kernel riêng bằng CUDA hoặc Triton. Cần khi bạn có một mẫu truy cập bộ nhớ đặc thù mà không kernel nào khớp — hiếm, và thường có cách rẻ hơn.

C. Stride, view và copy — bố cục bộ nhớ là thứ thật

Đây là chỗ người từ C++ có lợi thế lớn nhất, vì *stride* chính là thứ bạn vẫn dùng khi viết kernel: một tensor là một con trỏ, một *shape*, và một bộ *stride* nói mỗi bước theo mỗi chiều nhảy bao nhiêu phần tử. Hình 1.3 cho thấy vì sao chuyển vị không tốn gì và vì sao `.view()` sau đó lại thất bại.



Hình 1.3. Chuyển vị chỉ đổi *stride*, không chạm vào bộ nhớ — nên nó gần như miễn phí. Nhưng `.view()` đòi hỏi các phần tử nằm đúng thứ tự trong bộ nhớ, nên nó thất bại; `.reshape()` thì không thất bại mà lặng lẽ cấp phát một bản sao. Chính sự “lặng lẽ” đó là nguồn của các lần bùng nổ bộ nhớ không giải thích được.

Quy tắc thực hành rút ra: dùng `.view()` khi bạn *muốn* chương trình báo lỗi nếu bố cục không như bạn nghĩ, và dùng `.reshape()` chỉ khi bạn đã chấp nhận cái giá sao chép. Đây là cùng một triết lý với `at()` so với `operator[]` trong C++: chọn phiên bản ồn ào ở chỗ bạn chưa chắc.

D. Parameter, buffer và cặp train/eval

Ở giai đoạn 1 bạn đã phải tự trả lời “cái gì là tham số, cái gì là trạng thái”. PyTorch đặt tên cho câu trả lời đó:

- **Parameter** — được optimizer cập nhật, có gradient, nằm trong `state_dict`. Ví dụ: γ, β của BatchNorm.

- **Buffer** — *không* có gradient và optimizer không đọng tới, nhưng vẫn nằm trong `state_dict` và vẫn theo mô hình khi bạn gọi `.to(device)`. Ví dụ: `running_mean`, `running_var` của Batch-Norm.
- **Biến tạm** — không lưu, không chuyển; nếu bạn cần nó tồn tại qua một lần lưu checkpoint thì bạn đã chọn sai loại.

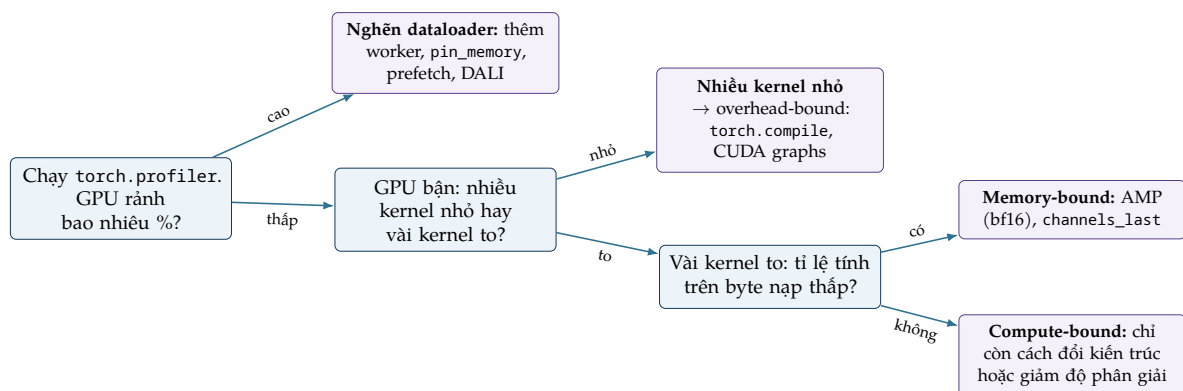
Từ đó suy ra vì sao quên `model.eval()` là lỗi kinh điển và im lặng: ở chế độ train, BatchNorm dùng thống kê của *batch hiện tại* thay vì buffer, và dropout bật lại. Kết quả là đầu ra phụ thuộc vào việc bạn tình cờ đưa mẫu nào vào cùng batch — một mẫu đánh giá có thể ra hai kết quả khác nhau chỉ vì bạn đổi thứ tự dữ liệu. Không có exception nào; chỉ có một con số hơi lệch.

↪ **hệ quả của C/12** — BatchNorm được đưa vào để ổn định huấn luyện; chính cơ chế ấy làm mô hình phụ thuộc cả batch, nên nó kéo theo hai chế độ hành vi khác nhau.

E. Tăng tốc: đo trước, và biết mỗi đòn bẫy gỡ nút thắt nào

Đây là phần bị lạm dụng nhiều nhất, nên bắt đầu bằng nguyên tắc: *mỗi đòn bẫy chỉ giúp khi bạn đang nghẽn ở đúng chỗ nó gỡ*. Hình 1.4 là cây quyết định rút gọn, và Bảng 1.2 là phần chi tiết.

↪ **ràng buộc bởi B/11** — cây quyết định tăng tốc chỉ là mô hình roofline diễn đạt bằng tên API: nhánh nào đúng phụ thuộc bạn đang bị chặn bởi băng thông, năng lực tính, hay chi phí điều phối.



Hình 1.4. Cây quyết định tăng tốc một vòng lặp huấn luyện. Nhánh đầu tiên — GPU đang rảnh — là nhánh đúng trong phần lớn trường hợp thực tế, và cũng là nhánh mà mọi mẹo về `torch.compile` đều vô dụng.

Bảng 1.2. Năm đòn bẫy tăng tốc, và nút thắt mà mỗi cái gỡ.

Đòn bẫy	Cơ chế	Gỡ nút thắt nào	Hỏng / vô dụng khi nào
Sửa DataLoader	Nhiều <i>worker</i> , <i>pin_memory</i> , <i>prefetch</i> , chuyển augmentation lên GPU	GPU đói dữ liệu	Quá nhiều worker làm cạn RAM và tăng thời gian khởi động; augmentation trên GPU tranh chấp với training
AMP (fp16/bf16)	Nửa số byte phải nạp và ghi	Memory-bound	Compute-bound thuận; fp16 tràn số khi không có GradScaler
<i>channels_last</i>	Đổi bố cục sang NHWC cho <i>Tensor Core</i>	Conv memory-bound	Mô hình có op không hỗ trợ NHWC → thêm phép hoán vị, chậm hơn
<code>torch.compile</code>	Hợp nhất kernel, giảm chi phí điều phối Python	Overhead-bound	Shape động gây biên dịch lại liên tục; <i>graph break</i> do code Python động
DDP / FSDP	Chia batch (DDP) hoặc chia trọng số (FSDP) qua nhiều GPU	Không đủ thời gian hoặc bộ nhớ	Băng thông liên kết kém → <i>all-reduce</i> nuốt hết lợi ích; batch hiệu dụng đối nên phải chỉnh <i>learning rate</i> [6]

Ví dụ nhỏ nhất cho AMP. Một tensor kích hoạt cỡ $64 \times 256 \times 56 \times 56$ có 51,4 triệu phần tử. Ở fp32 nó chiếm 205 MB; ở bf16 chiếm 103 MB. Trên một GPU có băng thông 900 GB/s, chỉ riêng việc đọc nó một lần tốn $228 \mu\text{s}$ so với $114 \mu\text{s}$. Ý nghĩa: nếu tăng đó chỉ làm vài phép tính trên mỗi byte nạp về, AMP cắt gần một nửa thời gian; nếu nó làm rất nhiều phép tính trên mỗi byte, AMP gần như không đổi gì. Đó chính là trục *arithmetic intensity* của B/11-gioi-han-tinh-toan, xuất hiện lại dưới dạng một lựa chọn hai dòng code.

fp16 hay bf16? Hai lựa chọn đối lập rõ ràng nên đáng đặt cạnh nhau: fp16 có 10 bit phần định trị và dải mũ hẹp, nên chính xác hơn ở vùng giá trị bình thường nhưng tràn hoặc triệt tiêu về không khi gradient nhỏ — vì thế nó *bắt buộc* cần GradScaler để nhân loss lên trước khi lượt ngược [7]. bf16 có cùng dải mũ với fp32 nhưng chỉ 7 bit định trị: kém chính xác hơn, đổi lại gần như không bao giờ tràn và không cần GradScaler. Trên phần cứng hỗ trợ bf16, mặc định hợp lý là bf16; fp16 chỉ còn lý do tồn tại trên phần cứng cũ.

```
scaler = torch.amp.GradScaler() # chỉ cần cho fp16
model = model.to(memory_format=torch.channels_last)
for x, y in loader:
    x = x.cuda(non_blocking=True).to(memory_format=torch.channels_last)
    opt.zero_grad(set_to_none=True) # set_to_none re hon zero_
    with torch.amp.autocast("cuda", dtype=torch.bfloat16):
        loss = criterion(model(x), y.cuda(non_blocking=True))
    scaler.scale(loss).backward()
    scaler.step(opt); scaler.update()
```

F. Giới hạn và trường hợp hỏng

- **torch.compile không phải nút bấm thần kỳ.** Nó giúp khi bạn đang overhead-bound; nó không giúp gì cho mô hình nghẽn ở dataloader, và nó có thể *chậm hơn* nếu shape đầu vào thay đổi liên tục khiến nó biên dịch lại. Với dữ liệu ảnh có kích thước thay đổi hoặc số hộp phát hiện thay đổi, đây là tình huống thường gặp chứ không phải ngoại lệ.
- **Profiler cũng nói dối nếu bạn đo sai.** Lệnh CUDA là bất đồng bộ, nên đo bằng đồng hồ Python quanh một lời gọi chỉ đo thời gian *xếp hàng*, không đo thời gian chạy. Phải đồng bộ hoá hoặc dùng CUDA event — một cái bẫy bạn đã gặp ở phía C++, và nó không biến mất khi lên Python.
- **DDP đối ngữ nghĩa của batch.** Batch hiệu dụng là batch mỗi GPU nhân số GPU, nên mọi siêu tham số phụ thuộc batch (learning rate, lịch warm-up, thống kê BatchNorm) đều đổi theo. Một so sánh “1 GPU với 8 GPU” mà không chỉnh learning rate là một so sánh vô nghĩa.
- **Tắt định có giá.** Bật use_deterministic_algorithms buộc PyTorch chọn kernel chậm hơn hoặc báo lỗi ở những op không có bản tắt định — và nó vẫn không đảm bảo hai máy khác GPU cho cùng kết quả.

CẠM BÃY

Rò bộ nhớ vì logging là bug tốn thời gian nhất trong danh sách này, vì triệu chứng xuất hiện sau vài trăm bước chứ không phải ngay lập tức. Viết `losses.append(loss)` thì bạn đang giữ luôn toàn bộ đồ thị tính toán dẫn tới loss, kể cả mọi activation trung gian; sau một epoch, GPU hết bộ nhớ và *stack trace* chỉ vào một dòng hoàn toàn vô can. Cách đúng là `losses.append(loss.detach())` hoặc `loss.item()`. Cùng một họ lỗi: giữ output của mô hình trong một *dict* để debug, hoặc trả tensor còn đồ thị ra khỏi một hàm được gọi trong vòng lặp.

CẦN NHỚ

Ảnh xạ triệu chứng → công cụ mới là thứ đáng nhớ, không phải tên API. GPU rảnh thì sửa dataloader; nhiều kernel nhỏ thì `torch.compile`; ít tính trên mỗi byte thì AMP và `channels_last`; không đủ bộ nhớ thì FSDP hoặc *gradient checkpointing*. Và trước tất cả: chạy profiler, vì tối ưu thứ chưa đo là cách chắc chắn nhất để mất một tuần mà không nhanh hơn. [T2]

G. Kết nối

Mục này là bản dịch của [D/19/01-tu-viet-bang-numpy](#) sang ngôn ngữ công cụ: *autograd* là cuốn băng bạn đã tự cuộn bằng tay, *buffer* là “trạng thái” bạn đã tự quản lý, `eval()` là cái công tắc bạn đã tự viết. Nó là ứng dụng trực tiếp của B/11-gioi-han-tinh-toan: cây quyết định ở Hình 1.4 chỉ là mô hình *roofline* được diễn đạt bằng tên API. Nó là tiền đề của [D/20-toi-uu-va-nen-mo-hinh](#) (nơi cùng những đòn bẩy này được áp cho **suy luận** (*inference*) thay vì huấn luyện) và của [D/22-tai-lap-va-mlops](#) (nơi mục về *seed* và tính tất định được mở rộng thành một chương).

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Khi nào `.view()` thất bại mà `.reshape()` thì không, và điều đó liên quan gì tới *stride*? Vì sao thất bại đôi khi lại là hành vi bạn muốn?
2. Vì sao `torch.compile` không giúp gì cho mô hình bị nghẽn ở dataloader, và làm sao biết mình đang ở tình huống đó trong ba mươi giây?
3. Thống kê chạy của BatchNorm là *parameter* hay *buffer*, và câu trả lời đó giải thích thế nào cho việc quên `eval()` lại làm kết quả phụ thuộc thứ tự dữ liệu?
4. Vì sao fp16 cần GradScaler mà bf16 thì không? Cái giá của bf16 là gì?
5. Bạn đo một lời gọi mô hình bằng `time.time()` và thấy nó tốn 0,2 ms. Vì sao con số đó gần như chắc chắn sai, và phải đo thế nào?
6. Chuyển từ 1 GPU sang 8 GPU bằng DDP, mọi thứ chạy nhưng độ chính xác tụt — hai nguyên nhân khả dĩ nhất là gì, biết rằng cả hai đều không phải bug trong DDP?

Nguồn

- [1] Y. Wu and K. He, “Group Normalization,” in *ECCV*, 2018.
- [2] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer Normalization,” *arXiv:1607.06450*, 2016.
- [3] N. Srivastava, G. Hinton, A. Krizhevsky, et al., “Dropout: A Simple Way to Prevent Neural Networks from Overfitting,” *JMLR*, vol. 15, pp. 1929–1958, 2014.
- [4] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986.
- [5] S. Rajbhandari, J. Rasley, O. Ruwase, et al., “ZeRO: Memory Optimizations Toward Training Trillion Parameter Models,” *SC*, 2020.
- [6] P. Goyal, P. Dollár, R. Girshick, et al., “Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour,” *arXiv:1706.02677*, 2017.
- [7] P. Micikevicius, S. Narang, et al., “Mixed Precision Training,” in *ICLR*, 2018.
- [8] G. Hinton, O. Vinyals, and J. Dean, “Distilling the Knowledge in a Neural Network,” in *NeurIPS Deep Learning Workshop*, 2015.
- [9] M. Nagel, M. Fournarakis, R. A. Amjad, et al., “A White Paper on Neural Network Quantization,” *arXiv:2106.08295*, 2021.
- [10] B. Jacob, S. Kligys, B. Chen, et al., “Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference,” in *CVPR*, 2018.
- [11] S. Han, H. Mao, and W. J. Dally, “Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding,” in *ICLR*, 2016.
- [12] J. Frankle and M. Carbin, “The Lottery Ticket Hypothesis: Finding Sparse, Trainable Neural Networks,” in *ICLR*, 2019.
- [13] H. Touvron, M. Cord, M. Douze, et al., “Training Data-Efficient Image Transformers and Distillation Through Attention,” in *ICML*, 2021.
- [14] A. G. Howard, M. Zhu, et al., “MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications,” *arXiv:1704.04861*, 2017.
- [15] M. Tan and Q. V. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” in *ICML*, 2019.
- [16] D. Sculley, G. Holt, D. Golovin, et al., “Hidden Technical Debt in Machine Learning Systems,” *NeurIPS*, 2015.
- [17] N. Bodla, B. Singh, R. Chellappa, et al., “Soft-NMS: Improving Object Detection with One Line of Code,” in *ICCV*, 2017.
- [18] P. Henderson, R. Islam, P. Bachman, et al., “Deep Reinforcement Learning That Matters,” *AAAI*, 2018.

1.3 Thực hành

Hai mini project dưới đây được thiết kế để *đóng* giai đoạn 1 và *mở* giai đoạn 2. Cái thứ nhất chứng minh bạn đã hiểu đúng cơ chế; cái thứ hai chứng minh bạn tăng tốc được dựa trên số đo.

Repo và tài nguyên. Để học: bài tập CS231n (làm hết assignment 1 và 2), [karpathy/micrograd](#) (autograd trong hai trăm dòng), [karpathy/nn-zero-to-hero](#), [pytorch/examples](#). Để dùng thật: [huggingface/pytorch-image-models](#) (timm), [Lightning-AI/pytorch-lightning](#), [facebookresearch/hydra](#), [NVIDIA/DALI](#), [albumentations](#). Để hiểu framework từ bên trong: [tinygrad/tinygrad](#) — đủ nhỏ để đọc hết. Danh sách này chốt tại tháng 9/2026; các repo hạ tầng ở đây (timm, DALI, Hydra) bền hơn nhiều so với tên mô hình dẫn đầu bảng, nhưng API của chính PyTorch cũng đổi, nên hãy nhớ vấn đề và tra lại cú pháp.

Câu hỏi kiểm tra hiểu. Khi nào `.view()` thất bại mà `.reshape()` thì không, và điều đó liên quan gì tới *stride*? Vì sao `torch.compile` không giúp gì cho mô hình bị nghẽn ở dataloader? *Hook* nào cần dùng để lấy activation ở giữa mạng mà không sửa forward? Và câu hỏi đóng giai đoạn 1: vì sao BatchNorm khiến lượt ngược tốn thêm bộ nhớ, còn dropout phải scale lúc train chứ không lúc test?

Mini project (vài giờ đến hai ngày). Viết lại *chính xác* một mạng NumPy của bạn bằng PyTorch và kiểm tra hai bên khớp tới 10^{-6} ở cả lượt xuôi lẫn gradient. Phần đánh giá không phải lúc nó khớp mà lúc nó không khớp: với mỗi chỗ lệch, phân loại xem đó là *bug* (lệch có hệ thống, không giảm khi bạn tăng độ chính xác) hay *số học* (lệch cỡ epsilon máy, đôi khi bạn đổi thứ tự phép cộng). Kỹ năng phân loại này là thứ bạn sẽ dùng lại mỗi lần export mô hình ở chương 21.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Lượt xuôi, lượt ngược	Lượt xuôi tính từ đầu vào ra loss; lượt ngược đi ngược để lấy đạo hàm theo từng tham số. Mọi vòng lặp huấn luyện là hai lượt này lặp lại.
••• Lan truyền ngược	Thuật toán vi phân cho mọi đồ thị khả vi, tốn cùng bậc chi phí với một lượt xuôi. Đừng nhầm nó (thuật toán) với lượt ngược (một lần chạy).
•• Hinge so softmax	Hinge cho gradient bằng đúng không khi lề đã thoả, nên ổn định. Softmax không bao giờ tắt, nên nó tính chính tiếp nhưng nhạy với nhân sai.
••• Gradient check	So gradient giải tích với sai phân trung tâm ở vài chục tọa độ ngẫu nhiên. Nó chỉ chứng minh xuôi và ngược khớp nhau, không chứng minh lượt xuôi đúng.
••• Bộ nhớ kích hoạt	Chỗ chứa đầu ra trung gian mà lượt ngược phải đọc lại. Đây là lý do huấn luyện tốn bộ nhớ hơn suy luận nhiều lần, và là thứ chặn batch size.
•• Vì sao lượt ngược BatchNorm không theo từng phần tử?	Nó chứa hai tổng trên toàn batch, một cho trung bình và một cho phương sai. Gradient của một mẫu vì thế phụ thuộc mọi mẫu khác trong batch.
•• Inverted dropout	Giữ phần tử với xác suất $1 - p$ rồi chia cho $1 - p$ lúc train; lúc test không làm gì. Bảo toàn kỳ vọng mà không đặt bước bắt buộc nào lên đường suy luận.
•• Vì sao col2im sinh ra bất định trên GPU?	Nó cộng dồn gradient vào các vị trí của số chồng lấn, và cài đặt GPU dùng <code>atomicAdd</code> . Cộng số thực không kết hợp, nên thứ tự luồng quyết định bit cuối.
•• Vì sao gradient check phải chạy float64?	Sai phân trung tâm lấy hiệu hai số gần bằng nhau. Ở float32, epsilon máy khoảng $1,2 \times 10^{-7}$ nên bạn đo nhiều làm tròn chứ không đo đạo hàm.
••• Autograd	Cuốn băng ghi lại các phép toán lúc lượt xuôi, tua ngược khi gọi <code>backward()</code> rồi xoá. Giữ một tensor còn dính băng nghĩa là giữ luôn cả đoạn băng phía sau nó.
•• Stride	Mỗi bước theo một chiều nhảy bao nhiêu phần tử trong bộ nhớ. Chuyển vị chỉ đổi stride nên miễn phí, nhưng làm tensor không còn liền mạch.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●● view so với reshape	view đòi tensor liền mạch và báo lỗi nếu không; reshape lạng lẽ sao chép. Dùng view ở chỗ bạn muốn chương trình ổn ào.
●●● Parameter so với buffer	Parameter được optimizer cập nhật; buffer thì không, nhưng vẫn nằm trong <code>state_dict</code> và vẫn đổi thiết bị theo mô hình. Thống kê chạy của BatchNorm là buffer.
●● Vì sao quên <code>eval()</code> làm kết quả đổi theo batch?	Ở chế độ train, BatchNorm dùng thống kê của batch hiện tại thay vì buffer, và dropout bật lại. Một mẫu vì thế cho hai kết quả khác nhau tùy bạn xếp nó cùng ai.
●●● AMP	Tính phần lớn phép toán ở 16 bit, giữ trọng số ở 32 bit. Nó giảm số byte phải nạp chứ không giảm số phép tính, nên vô ích với kernel compute-bound.
●● <code>torch.compile</code>	Hợp nhất kernel và cắt chi phí điều phối Python. Chỉ giúp khi bạn overhead-bound; shape thay đổi liên tục làm nó biên dịch lại và chậm hơn.
●● Sai số <code>gradient check</code> khoảng 3×10^{-3} và đổi mỗi lần chạy?	Còn một nguồn ngẫu nhiên chưa đóng băng: mặt nạ dropout, hoặc BatchNorm đang ở chế độ train. Con số ổn định thì mới nên nghi công thức.
●● Bật AMP và <code>torch.compile</code> mà epoch không nhanh hơn?	Bạn đang ngهن ở dataloader, không phải ở GPU. Chạy profiler xem GPU rảnh bao nhiêu phần trăm trước khi đùng vào mô hình.
●● Bộ nhớ GPU phình dần suốt epoch rồi hết?	Bạn đang giữ tensor còn đồ thị trong một danh sách log. Dùng <code>loss.detach()</code> hoặc <code>loss.item()</code> .

●●● phải thuộc ●● nên nắm ● biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
PyTorch, JAX	Toàn bộ autograd là hiện thực của vi phân chế độ nghịch trên đồ thị động — đúng thứ bạn tự cuộn bằng tay ở giai đoạn 1	Hạ tầng, ổn định từ 2017 tới nay
FlashAttention	Đánh đổi tính lại lượt xuôi để giảm bộ nhớ kích hoạt, tức chính <i>gradient checkpointing</i> áp cho attention	Đã thành mặc định trong gần như mọi thư viện transformer
<code>timm</code>	Vòng lặp huấn luyện chuẩn với AMP, <code>channels_last</code> và <code>torch.compile</code> — đúng bốn đòn bẩy của Bảng 1.2	Hạ tầng thị giác; công thức trong repo đáng đọc hơn tài liệu
FSDP và ZeRO	Chia trọng số và trạng thái optimizer thay vì nhân bản chúng, khi một GPU không chứa nổi mô hình	Ý tưởng từ 2020, vẫn là nền của mọi huấn luyện quy mô lớn
<code>tinygrad</code> , <code>micrograd</code>	Chính bài tập giai đoạn 1, viết đủ nhỏ để đọc hết trong một buổi	Giá trị sư phạm, không phải công cụ sản xuất

MINI PROJECT SLAM · Tự cài lại tầng mô tả của frontend bằng NumPy · 1–2 ngày

- Mục tiêu** làm đúng một lần cái việc mà framework luôn làm hộ, trên đúng khối bạn đang dùng thật: viết cả lượt xuôi lẫn lượt ngược cho một mạng mô tả đặc trưng, và không tin nó cho tới khi kiểm tra gradient số cho phép.
- Dữ liệu** chính tập mảnh ảnh 32×32 dựng ở mini project chương 18, cùng nhãn cấp lấy từ liên kết dữ liệu của bản đồ.
- Các bước** viết lượt xuôi cho một khối gồm *convolution* 3×3 , BatchNorm, ReLU, max-pool và một tầng *fully-connected* rồi chuẩn hoá L2 ra vector mô tả 32 chiều. Viết lượt ngược giải tích cho từng tầng. Chạy kiểm tra gradient ở float64 trên 20 toạ độ ngẫu nhiên

- của *mỗi* tham số, với mọi nguồn ngẫu nhiên đóng băng và BatchNorm đặt ở chế độ cố định. Chỉ khi mọi tầng qua ngưỡng mới ghép lại và huấn luyện bằng một hàm mất mát tương phản trên các cặp có nhãn.
- Xong khi** sai số tương đối của kiểm tra gradient dưới 10^{-6} cho mọi tham số của mọi tầng. Toạ độ nằm sát điểm gãy của ReLU hoặc max-pool được phép bỏ qua, nhưng phải ghi rõ đã bỏ bao nhiêu và vì sao. Sau đó bản NumPy khớp bản PyTorch tới 10^{-6} ở cả lượt xuôi lẫn *gradient*.
- Nối với** D/19 cho bảng “backward cần nhớ gì” và cây chẩn đoán; A/01 cho quy tắc chuỗi. Mô hình mô tả sinh ra ở đây chính là thứ chương 20 đem đi lượng tử hoá.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì vi phân chế độ nghịch trả về đạo hàm theo *mọi* tham số trong một lượt, còn vi phân thuận chỉ trả về theo một hướng mỗi lượt. Chi phí một lượt ngược cùng bậc với một lượt xuôi, nên tổng chi phí không phụ thuộc số tham số — đó là toàn bộ khoản tiết kiệm.
- A2. Vì backward phải đọc lại trạng thái của lượt xuôi: đầu vào cho tầng *fully-connected*, mặt nạ cho ReLU và dropout, $\hat{x}$ cùng hai tổng batch cho BatchNorm. Suy luận không cần lượt ngược nên giải phóng được ngay sau mỗi tầng; huấn luyện thì không.
- A3. Nó chỉ kiểm tính *nhất quán* giữa lượt xuôi và lượt ngược, không kiểm tính đúng của lượt xuôi. Một tầng cài sai công thức xuôi nhưng có ngược khớp với chính nó sẽ qua hoàn hảo; ngoài ra nó không bắt được lỗi trạng thái, lỗi nhân và lỗi ở các chỗ nối.
- A4. Vì mỗi đòn bẩy chỉ gỡ đúng nút thắt của nó. Nếu GPU đang đói dữ liệu thì thời gian nằm ở dataloader, và cả AMP lẫn `torch.compile` đều không chạm tới chỗ đó. Cây quyết định ở Hình 1.4 bắt đầu bằng “GPU rảnh bao nhiêu phần trăm” chính vì lý do này.

NÓI LẠI THẬT ĐƠN GIẢN

Hãy tưởng tượng một dây chuyền làm bánh gồm mười khâu, mỗi khâu có vài cái núm chỉnh. Bánh ra lò bị mặn. Câu hỏi là: núm nào ở khâu nào đã làm nó mặn, và mặn thêm bao nhiêu? Cách ngây thơ là vặn thử từng núm rồi chạy lại cả mẻ bánh. Có một nghìn cái núm thì phải chạy một nghìn mẻ. Không ai làm nổi.

Mẹo là đi ngược dây chuyền đúng một lần. Đứng ở cuối, bạn biết bánh mặn hơn mong muốn bao nhiêu. Lùi về khâu thứ mười và hỏi: nếu khâu này đưa ra thứ mặn hơn một chút, bánh cuối mặn thêm bao nhiêu? Có câu trả lời rồi thì lùi tiếp về khâu chín, và cứ thế. Một lượt đi ngược chia được trách nhiệm cho cả nghìn cái núm, tốn xấp xỉ bằng một lượt làm bánh bình thường.

Cái giá rất cụ thể: muốn hỏi ngược thì mỗi khâu phải giữ lại mẫu bán thành phẩm của lượt xuôi. Xưởng vì thế cần kho chứa gấp nhiều lần, và đó đúng là lý do máy tính hết bộ nhớ khi huấn luyện chứ không hết khi chạy thật. Cái giá thứ hai: công thức chia trách nhiệm ở mỗi khâu do bạn tự viết, và nếu sai một dấu thì cả dây chuyền vẫn chạy êm, bánh vẫn ra, chỉ là không bao giờ ngon lên. Vì vậy phải thử tay từng khâu trước khi ghép.

Một câu: Chạy xuôi một lần và giữ lại dấu vết, đi ngược một lần để chia trách nhiệm — rẻ hơn thử từng núm hàng nghìn lần, và cái giá đúng bằng chỗ chứa dấu vết đó.

Tổng kết chương

Chương này đi từ chỗ tự tay cuộn một cuốn băng autograd trong NumPy tới chỗ đọc được một trace của `torch.profiler` và biết chính xác dòng nào cần sửa. Sợi chỉ xuyên suốt là *trạng thái*: cái gì được nhớ, cái gì được lưu, cái gì đổi hành vi giữa hai chế độ — và phần lớn bug im lặng trong deep learning là bug ở đúng ba câu hỏi đó.

Điều còn bỏ ngỏ là toàn bộ phía **suy luận** (*inference*). Ở đây ta chỉ quan tâm huấn luyện: bộ nhớ hoạt động, thông lượng, tính đúng đắn của gradient. Nhưng ràng buộc thật của một hệ thống trên thiết bị nằm ở phía kia: độ trễ đuôi, năng lượng, và tập toán tử mà runtime hỗ trợ.

Cùng những đòn bẩy trong Bảng 1.2 sẽ xuất hiện lại ở [D/20-toi-uu-va-nen-mo-hinh](#), nhưng với một hàm mục tiêu khác hẳn. Điều thứ hai còn bỏ ngỏ là tính tái lập: mục 19.2 chỉ nêu tên `use_deterministic_algorithms` và nói nó không đủ; vì sao nó không đủ, và mục tiêu đúng là gì, thuộc về [D/22-tai-lap-va-mlops](#).

■ Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Chương chia hai giai đoạn theo tầng bán rã [T2][T3] và [T4]. Hãy nêu một hiểu biết từ giai đoạn 1 mà giai đoạn 2 *không* dạy được, và một sai lầm ở giai đoạn 2 mà giai đoạn 1 không giúp bạn tránh?
2. Cùng một triệu chứng “mạng không học” có thể do đạo hàm sai, do lẫn lộn trạng thái train và eval, hoặc do dữ liệu. Hai giai đoạn cho bạn những công cụ phát hiện khác nhau nào, và thứ tự dùng chúng nên là gì?
3. BatchNorm xuất hiện ở cả hai mục: một lần là hai tổng trên batch trong lượt ngược, một lần là buffer chứ không phải parameter. Hai góc nhìn đó nói cùng điều gì về BatchNorm, và điều đó giải thích thế nào cho hành vi khi lưu rồi nạp lại checkpoint?
4. Phép cộng dồn tán xạ trong lượt ngược của convolution ở giai đoạn 1 quay lại ám bạn ở giai đoạn 2 dưới dạng nào, và nó liên quan gì tới việc hai lần chạy cho kết quả không trùng nhau?
5. Vì sao “huấn luyện tốn bộ nhớ gấp nhiều lần suy luận” là kết luận rút ra được từ giai đoạn 1 nhưng chỉ đo được ở giai đoạn 2, và bạn đo thế nào để tách phần kích hoạt khỏi phần trạng thái optimizer?
6. Bạn bật `torch.compile` và AMP mà thời gian một epoch không đổi. Trước khi kết luận công cụ vô dụng, bạn đo ba đại lượng nào, và mỗi đại lượng loại bỏ giả thuyết nào?
7. Một mẹo tối ưu nghe được trên diễn đàn có tuổi thọ ngắn hơn hẳn hiểu biết ở giai đoạn 1. Bạn dùng tiêu chí nào để quyết định một mẹo có đáng đưa vào code nền của mình hay không?

Nguồn

- [1] Y. Wu and K. He, “Group Normalization,” in *ECCV*, 2018.
- [2] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer Normalization,” *arXiv:1607.06450*, 2016.
- [3] N. Srivastava, G. Hinton, A. Krizhevsky, et al., “Dropout: A Simple Way to Prevent Neural Networks from Overfitting,” *JMLR*, vol. 15, pp. 1929–1958, 2014.
- [4] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986.
- [5] S. Rajbhandari, J. Rasley, O. Ruwase, et al., “ZeRO: Memory Optimizations Toward Training Trillion Parameter Models,” *SC*, 2020.
- [6] P. Goyal, P. Dollár, R. Girshick, et al., “Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour,” *arXiv:1706.02677*, 2017.
- [7] P. Micikevicius, S. Narang, et al., “Mixed Precision Training,” in *ICLR*, 2018.
- [8] G. Hinton, O. Vinyals, and J. Dean, “Distilling the Knowledge in a Neural Network,” in *NeurIPS Deep Learning Workshop*, 2015.
- [9] M. Nagel, M. Fournarakis, R. A. Amjad, et al., “A White Paper on Neural Network Quantization,” *arXiv:2106.08295*, 2021.
- [10] B. Jacob, S. Kligys, B. Chen, et al., “Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference,” in *CVPR*, 2018.
- [11] S. Han, H. Mao, and W. J. Dally, “Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding,” in *ICLR*, 2016.
- [12] J. Frankle and M. Carbin, “The Lottery Ticket Hypothesis: Finding Sparse, Trainable Neural Networks,” in *ICLR*, 2019.
- [13] H. Touvron, M. Cord, M. Douze, et al., “Training Data-Efficient Image Transformers and Distillation Through Attention,” in *ICML*, 2021.
- [14] A. G. Howard, M. Zhu, et al., “MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications,” *arXiv:1704.04861*, 2017.
- [15] M. Tan and Q. V. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” in *ICML*, 2019.
- [16] D. Sculley, G. Holt, D. Golovin, et al., “Hidden Technical Debt in Machine Learning Systems,” *NeurIPS*, 2015.
- [17] N. Bodla, B. Singh, R. Chellappa, et al., “Soft-NMS: Improving Object Detection with One Line of Code,” in *ICCV*, 2017.
- [18] P. Henderson, R. Islam, P. Bachman, et al., “Deep Reinforcement Learning That Matters,” *AAAI*, 2018.

Tối ưu và nén mô hình (Model Optimization and Compression)

PHỤ THUỘC B/11-gioi-han-tinh-toan (bắt buộc — *roofline*, memory-bound so với compute-bound; không có nó thì chương này chỉ là danh sách mẹo); B/09-thiet-ke-ham-mat-mat (phân phối mềm dùng trong chưng cất); C/18/02-kinh-te-hoc-pretraining (cùng logic ngân sách, đổi cột chi); D/19-numpy-toi-pytorch (AMP, channels_last, profiler).

TÓM TẮT

Mô hình tốt nhất của bạn không vừa ràng buộc triển khai. Chương này trình bày năm đòn bẩy để ép nó vừa — giảm độ phân giải, đổi kiến trúc, lượng tử hoá, chưng cất, tia — và lập luận rằng *thứ tự* dùng chúng quan trọng hơn kỹ thuật của từng cái. Đọc xong bạn lập được đường Pareto độ chính xác–độ trễ cho bài toán của mình và chọn một điểm có lý do, thay vì thử ngẫu nhiên rồi giữ cái nào trông ổn. Nếu chỉ nhớ một điều: FLOPs là đại diện tồi cho độ trễ, nên mỗi kỹ thuật nén phải kèm phép đo trên *đúng phân cứng đích*, và sai lệch giữa FLOPs và thời gian thật đi theo hướng khó đoán.

KIẾN THỨC CẦN CÓ TRƯỚC

- **độ trễ (latency) và thông lượng (throughput)** — độ trễ là thời gian một yêu cầu đi từ vào tới ra; thông lượng là số yêu cầu xử lý được mỗi giây. Gộp batch làm thông lượng tăng và độ trễ tăng — hai đại lượng này thường xung đột.
- **FLOPs** — số phép tính dấu phẩy động của một **lượt xuôi** (*forward pass*). Nó dễ tính trên giấy nhưng chỉ tương quan lỏng lẻo với thời gian thật, vì phần lớn mạng bị chặn bởi băng thông bộ nhớ chứ không bởi năng lực tính toán.
- **lượng tử hoá (quantization)** — biểu diễn trọng số và kích hoạt bằng số nguyên ít bit (thường INT8) thay vì dấu phẩy động 32 bit, để giảm số byte phải nạp và tăng thông lượng; cái giá là sai số làm tròn tích lũy qua các tầng.
- **calibration set** — một tập ảnh nhỏ (thường vài trăm) chạy qua mô hình để đo dải giá trị thực tế của kích hoạt, từ đó chọn hệ số tỉ lệ cho lượng tử hoá. Nó phải *đại diện* cho dữ liệu triển khai, nếu không dải đo được sẽ sai.
- **tia (pruning) và sparsity** — đặt một phần trọng số về không. *Sparsity* là tỉ lệ trọng số bằng không. Tia *không cấu trúc* rải các số không khắp nơi; tia *có cấu trúc* bỏ nguyên kênh hoặc nguyên khối.
- **chưng cất tri thức (knowledge distillation)** — huấn luyện một mô hình nhỏ (student) bắt chước đầu ra của một mô hình lớn (teacher), thay vì chỉ học từ nhãn cứng. Phân phối xác suất mềm của teacher mang nhiều thông tin hơn một nhãn đúng/sai.
- **độ phân giải đầu vào** — kích thước ảnh đưa vào mạng, ví dụ 640×640 . Chi phí tính toán của phần lớn mạng thị giác tỉ lệ với số điểm ảnh, tức tỉ lệ với *bình phương* cạnh.
- **phần cứng đích** — đúng con chip và đúng cấu hình mà mô hình sẽ chạy khi triển khai — Jetson Orin ở chế độ 30 W không phải cùng một cỗ máy với chính nó ở chế độ 15 W, và cả hai đều không phải là chiếc RTX trên bàn bạn.

CÂU HỎI MỞ ĐẦU

- Q1. Bạn tĩa bỏ 90% trọng số, mô hình giữ nguyên độ chính xác, và nó không chạy nhanh hơn một mili-giây nào — phần cứng đã từ chối món quà đó ở chỗ nào?
- Q2. Vì sao kỹ thuật thô nhất trong cả chương — hạ độ phân giải đầu vào — lại được xếp đứng trước mọi kỹ thuật nén tinh vi?
- Q3. Một tầng có đúng *một* kênh trọng số bất thường; vì sao chừng đó đủ để đẩy phần còn lại của tầng xuống dưới ba bit hiệu dụng sau khi lượng tử hoá?
- Q4. Mô hình A ít FLOPs hơn mô hình B nhưng chạy chậm hơn trên cùng một Jetson — FLOPs đã nói dối theo bao nhiêu cách khác nhau?

2.1 Đặt vấn đề

Câu hỏi dẫn dắt của chương rất cụ thể: *mô hình tốt nhất của bạn không vừa ràng buộc triển khai — làm gì tiếp?* Bạn có năm đòn bẫy, chúng không tương đương nhau, và thứ tự dùng chúng quan trọng hơn kỹ thuật của từng cái.

Điều làm chương này khác một danh sách mẹo là chỗ nó xuất phát từ một sự thật vật lý đã có ở B/11-gioi-han-tinh-toan: phần lớn mạng thị giác không bị chặn bởi năng lực tính toán mà bởi *băng thông bộ nhớ*. Từ sự thật đó suy ra gần như mọi thứ còn lại. Lượng tử hoá có tác dụng vì nó giảm số byte phải nạp chứ không phải vì nó giảm số phép nhân. Tĩa không cấu trúc thất bại vì nó giảm số phép nhân mà không giảm số byte phải nạp. Hợp nhất kernel có tác dụng vì nó cắt bớt các lượt đi về bộ nhớ. Ai chưa nắm trực này sẽ học chương này như học một danh sách; ai nắm rồi thì tự suy ra được phần lớn nội dung.

→ **ràng buộc bởi** B/11 — phần lớn mạng thị giác bị chặn bởi băng thông bộ nhớ chứ không bởi năng lực tính, và chính sự thật đó quyết định kỹ thuật nén nào có tác dụng.

Học xong chương này thì làm được gì mà trước đó không làm được? Bạn cầm một ngân sách mili-giây và một mô hình quá chậm. Trong một buổi bạn vạch được đường Pareto độ chính xác–độ trễ, chọn một điểm, và giải thích được vì sao chọn nó. Cách cũ là thử ba kỹ thuật ngẫu nhiên rồi giữ lại cái nào trông ổn.

TRỰC GIÁC

Lượng tử hoá *theo tensor* giống việc chọn một khẩu độ và một tốc độ màn trập duy nhất cho cả khung hình có một bóng đèn sáng chói trong đó. Bóng đèn ép bạn phải khép sáng lại, và mọi vùng còn lại chìm xuống sát sàn nhiều — không phải vì máy ảnh kém, mà vì bạn chỉ có một dải động và đã tiêu nó cho một điểm.

Lượng tử hoá *theo kênh* là chụp mỗi vùng bằng một thiết lập riêng rồi ghép lại. Cùng số bit, nhưng dải động được phân bổ theo nhu cầu thật của từng vùng. Đây không phải ẩn dụ lỏng lẻo. Cả hai trường hợp đều là bài toán ánh xạ một dải giá trị liên tục vào một số mức rời rạc cố định. Và ở cả hai, *một giá trị ngoại lai duy nhất cũng đủ phá hỏng độ phân giải của tất cả những giá trị còn lại*.

2.2 Thứ tự thử nghiệm, và lý do của thứ tự đó

A. Bảng thứ tự

Bảng 2.1 là kết luận thực dụng của cả chương, đặt lên trước để bạn dùng được ngay; phần còn lại giải thích từng dòng.

Bảng 2.1. Năm đòn bẩy nén, theo thứ tự nên thử. Cột “vì sao đứng trước” mới là nội dung; cột “cái giá” là thứ bạn phải chấp nhận.

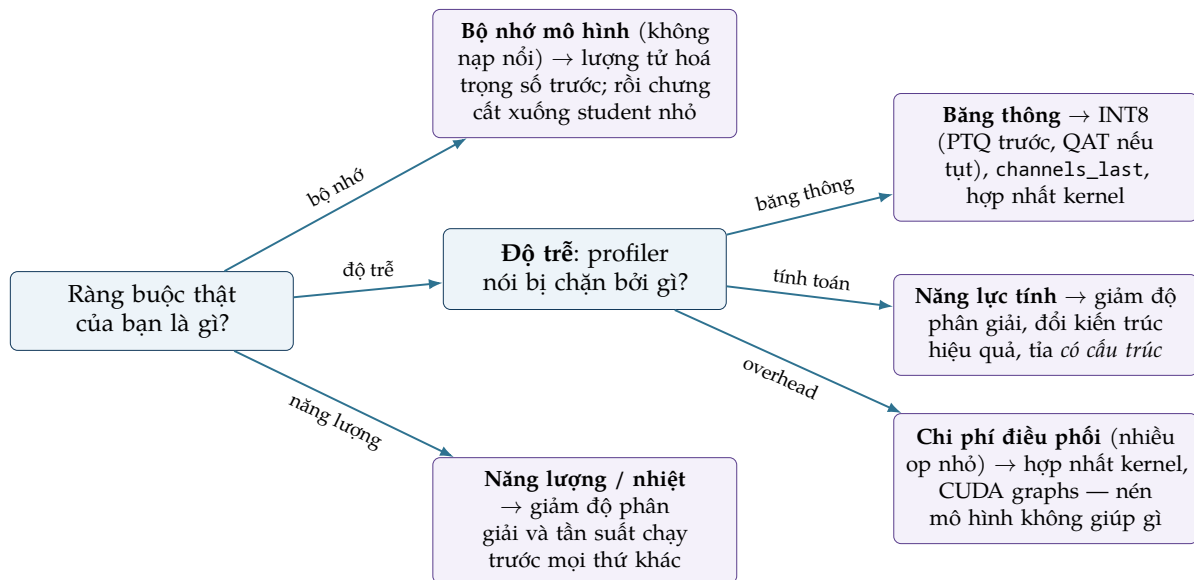
#	Đòn bẩy	Vì sao đứng trước	Cái giá
1	Giảm độ phân giải hoặc độ dài đầu vào	Rẻ nhất, hiệu quả gần như bậc hai theo cạnh ảnh, đảo ngược được ngay trong một dòng config	Mất vật nhỏ và chi tiết mảnh — đúng thứ bạn có thể không đo được trên tập test hiện có
2	Đổi sang kiến trúc hiệu quả	Tối ưu từ thiết kế luôn tốt hơn nén sau; cùng độ chính xác nhưng ít byte hơn ngay từ đầu	Phải huấn luyện lại từ đầu
3	Lượng tử hoá	Tỉ lệ lợi ích trên công sức cao nhất trong nhóm nén; gỡ đúng nút thắt bằng thông	Mất độ chính xác nhỏ, và phụ thuộc mạnh vào calibration đúng
4	Chưng cất	Giữ được nhiều chất lượng nhất trên mỗi đơn vị kích thước	Cần teacher và một chu kỳ huấn luyện đầy đủ
5	Tỉa	Thường <i>không</i> cho tốc độ thật trên phần cứng phổ thông	Công sức lớn, lợi ích không chắc

B. Vì sao thứ tự này chứ không phải thứ tự khác

Ba nguyên tắc sắp xếp, và cả ba đều là nguyên tắc kỹ thuật chứ không phải sở thích:

- **Đảo ngược được thì thử trước.** Giảm độ phân giải là một dòng config; nếu hỏng, bạn biết trong nửa giờ. Chưng cất là một tuần; nếu hỏng, bạn mất một tuần. Xếp theo chi phí của việc *sai* chứ không theo chi phí của việc *đúng*.
- **Thiết kế thắng nén.** Một mạng được thiết kế để có ít byte đi qua luôn tốt hơn một mạng to bị ép nhỏ lại, vì nén sau chỉ bỏ bớt thứ đã có chứ không đổi được cấu trúc luồng dữ liệu.
- **Ưu tiên đòn bẩy gỡ đúng nút thắt.** Nếu bạn memory-bound thì lượng tử hoá gỡ đúng chỗ; tỉa không cấu trúc thì không, dù nó nghe hứa hẹn hơn.

Hình 2.1 chuyển ba nguyên tắc đó thành một cây quyết định theo *ràng buộc* bạn đang bị chặn, vì trong thực tế bạn hiếm khi bị chặn bởi cả ba thứ cùng lúc.



Hình 2.1. Chọn đòn bẩy theo ràng buộc đang chặn bạn, không theo danh sách. Nhánh dưới cùng bên phải là nhánh hay bị bỏ sót nhất: khi thời gian bị nuốt bởi chi phí điều phối kernel, mọi kỹ thuật *nén* đều vô ích và việc phải làm là hợp nhất.

Ví dụ nhỏ nhất: đòn bẩy số 1 rẻ tới mức nào

Chi phí tính của phần lớn mạng tích chập tỉ lệ với số điểm ảnh. Hạ độ phân giải từ 640×640 xuống 512×512 cho tỉ lệ $(512/640)^2 = 0,64$. Tức là cắt 36% chi phí bằng một dòng config, không huấn luyện lại, và hoàn tác được tức thì. Không kỹ thuật nào khác trong chương có tỉ lệ lợi ích trên công sức gần bằng. Cái giá cũng cụ thể như vậy: một vật cao 20 điểm ảnh ở 640 chỉ còn 16 điểm ảnh ở 512, và nếu tập test của bạn không có vật nhỏ thì bạn sẽ *không đo được* thiệt hại — một trường hợp điển hình của việc đánh giá che giấu chế độ hỏng, đúng chủ đề của B/07-chia-du-lieu-va-danh-gia.

↪ **ràng buộc bởi** B/07 — thiệt hại của việc giảm độ phân giải tập trung ở nhóm vật nhỏ, nên nó vô hình nếu tập đánh giá không được phân tách theo nhóm.

2.3 MẠCH VẤN ĐỀ → GIẢI PHÁP

Sáu nút thắt của việc đưa một mô hình vào ngân sách

1 Vấn đề

Giải pháp

Giả định

Giá phải trả

fp32 lãng phí bằng thông.

lượng tử hoá: giảm số bit thì giảm luôn lượng byte phải nạp, và trên phần cứng có đơn vị số nguyên chuyên dụng thì giảm cả thời gian tính.

phân bố giá trị đủ “gọn” để một hệ số tỉ lệ mô tả được.

sai số làm tròn tích lũy, và độ nhảy rất cao với việc chọn hệ số tỉ lệ.

2 Vấn đề

Giải pháp

Vấn đề mới

nhiều trọng số vô dụng.

tĩa.

phần cứng phổ thông không tận dụng được các số không rải rác, nên tiết kiệm trên giấy không thành tốc độ thật. Chỉ tia có cấu trúc, hoặc mẫu 2:4 trên phần cứng hỗ trợ, mới cho lợi ích đo được.

3 Vấn đề

mô hình nhỏ huấn luyện từ đầu thì yếu.

- Giải pháp** chưng cất: dùng phân phối mềm của teacher làm mục tiêu, giàu thông tin hơn nhãn cứng nhiều lần [8].
- Giá phải trả** cần teacher và một chu kỳ huấn luyện đầy đủ.
- 4 Vấn đề** nén sau luôn thua thiết kế tốt.
- Giải pháp** kiến trúc hiệu quả — tích chập tách theo chiều sâu, khối *inverted residual*, và tái tham số hoá kiểu RepVGG.
- Hệ quả** đây lại là mẫu hình “đẩy chi phí sang training-time” đã gặp ở chương 17 và sẽ gặp lại ở chương 21.
- 5 Vấn đề** chọn kiến trúc bằng tay là thủ công và thiên lệch.
- Giải pháp** NAS — tự động tìm kiến trúc.
- Giá phải trả** chi phí tìm kiếm thường lớn hơn lợi ích, và kết quả gắn chặt với phần cứng dùng để tìm, nên hết giá trị khi bạn đổi chip.
- 6 Vấn đề** cùng FLOPs nhưng khác tốc độ.
- Giải pháp** hợp nhất kernel để cắt số lượt đi về bộ nhớ, và bố cục bộ nhớ đúng (channels_last, căn lề, tận dụng cache).
- Hệ quả** đây là B/11-gioi-han-tinh-toan áp dụng trực tiếp, và là lý do hai mô hình cùng FLOPs có thể chênh nhau hai lần về thời gian thật.

A. Lượng tử hoá — chi tiết quyết định là hệ số tỉ lệ

Lượng tử hoá đối xứng INT8 ánh xạ một số thực w thành số nguyên $q = \text{round}(w/s)$ với $s = \max |w|/127$, và khôi phục bằng $\hat{w} = s q$. Công thức này nói điều gì? Rằng bạn có đúng 255 mức để phủ toàn bộ dải giá trị, và $\max |w|$ — giá trị lớn nhất, tức giá trị ngoại lai nếu có — quyết định độ mịn của tất cả các mức còn lại.

Ví dụ nhỏ nhất tính tay được. Một tầng có trọng số nằm trong $[-0,1; 0,1]$ ở mọi kênh, trừ một kênh duy nhất có trọng số lên tới 2,0.

- **Per-tensor:** $s = 2,0/127 = 0,01575$. Một trọng số 0,1 được ánh xạ thành $\text{round}(0,1/0,01575) = 6$. Vậy toàn bộ dải $[0; 0,1]$ chỉ còn *sáu* mức — tức chưa tới 3 bit hiệu dụng cho phần lớn mạng.
- **Per-channel:** mỗi kênh có s riêng, nên kênh bình thường dùng $s = 0,1/127$ và trọng số 0,1 được ánh xạ thành 127 — đủ 8 bit.

Cùng số bit, khác nhau hơn một bậc độ lớn về độ phân giải hiệu dụng. Đó là lý do per-channel gần như luôn là lựa chọn đúng cho trọng số của conv và linear, và là lý do một tầng duy nhất có phân bố lệch cũng đủ làm hỏng cả mô hình sau khi lượng tử hoá [9], [10].

Bảng 2.2. Ba chế độ lượng tử hoá thường gặp.

Chế độ	Cơ chế	Chi phí	Hỏng khi nào
Weight-only	Chỉ nén trọng số, kích hoạt vẫn fp16	Gần bằng không	Không giúp gì nếu nút thắt là kích hoạt chứ không phải trọng số
PTQ (post-training)	Chạy calibration set để đo dải kích hoạt, rồi chốt hệ số tỉ lệ	Vài phút	Calibration set không đại diện; mô hình có kích hoạt ngoại lai mạnh
QAT (quantization-aware)	Mô phỏng làm tròn ngay trong lúc huấn luyện để mạng học cách chịu đựng	Một chu kỳ huấn luyện	Không có dữ liệu huấn luyện; hoặc bạn dùng nó khi PTQ vốn đã đủ tốt

Kinh nghiệm thực hành đáng tin: thử PTQ trước, đo, và chỉ chuyển sang QAT khi PTQ tụt quá ngưỡng chấp nhận. Và nguyên nhân số một của việc tụt bất thường không phải thuật toán mà là *calibration set* chọn sai. Vài trăm ảnh lấy toàn từ một điều kiện chiếu sáng cho dài kích hoạt hẹp hơn thực tế. Mọi giá trị nằm ngoài dải đó bị cắt cụt khi triển khai.

B. Tỉa — vì sao 90% sparsity không nhanh hơn

Đây là chỗ trực giác đánh lừa mạnh nhất, nên đáng nói thẳng là bức tranh phức tạp chứ không đơn giản hoá được. Tỉa không cấu trúc đạt được sparsity rất cao mà gần như không mất độ chính xác [11], [12], nhưng *không* nhanh hơn trên GPU thông thường, vì ba lý do cộng lại: các số không rải rác không cho phép bỏ qua việc nạp dữ liệu (bạn vẫn phải đọc chỉ số để biết chỗ nào là không); kernel dense đã được tối ưu nhiều năm còn kernel thưa thì chưa; và lưu chỉ số của các phần tử khác không tự nó đã tốn băng thông.

Bảng 2.3. Ba mức cấu trúc của tỉa. Chỉ hai dòng dưới cho tốc độ thật.

Kiểu	Cơ chế	Tốc độ thật	Hỏng / hạn chế
Không cấu trúc	Đặt từng trọng số nhỏ về không	Gần như không đổi trên GPU phổ thông	Chỉ có ích khi bạn cần giảm <i>dung lượng lưu trữ</i> , không phải thời gian
2:4 (bán cấu trúc)	Trong mỗi nhóm 4 trọng số, đúng 2 bằng không	Có, nhưng chỉ trên phần cứng có đơn vị sparse tensor	Ràng buộc cứng làm mất nhiều độ chính xác hơn tỉa tự do cùng tỉ lệ
Có cấu trúc	Bỏ nguyên kênh, nguyên đầu attention, nguyên khối	Có, trên mọi phần cứng — vì mạng thật sự nhỏ đi	Mất nhiều độ chính xác hơn; cần fine-tune lại sau khi tỉa

C. Chưng cất — vì sao phân phối mềm đáng giá

Nhãn cứng nói “đây là chó”. Phân phối mềm của teacher nói “0,7 chó, 0,2 sói, 0,08 cáo, 0,02 mèo” — và ba con số cuối chính là thông tin về *cấu trúc lớp* mà nhãn cứng vứt đi. Student học được rằng sói gần chó hơn mèo, một quan hệ mà nó sẽ phải tự phát hiện lại nếu chỉ có nhãn cứng. Đó là lý do một student nhỏ được chưng cất thường thắng chính student đó huấn luyện từ nhãn cứng, dù cả hai có cùng số tham số.

↔ **cùng nguyên lý với** B/09 — phân phối mềm mang thông tin về cấu trúc lớp mà nhãn cứng vứt đi — cùng lý do khiến làm mượt nhãn có tác dụng.

Ba biến thể, theo mức mạnh tăng dần cho thị giác: chưng cất *logit* (đơn giản nhất), chưng cất đặc trưng trung gian (thường mạnh hơn cho thị giác vì nó ép cả biểu diễn chứ không chỉ đầu ra), và chưng cất *attention* cho *transformer* [13]. Xu hướng tại thời điểm viết là chưng cất một mô hình nền tảng lớn xuống student cỡ 20–40M cho thiết bị biên; đây là cách phổ biến nhất để đưa các mô hình lớp SAM hoặc DINO lên thiết bị — ghi nhận này là quan sát thực hành, không phải một kết quả đo mà tôi kiểm chứng được.

D. Kiến trúc hiệu quả, tái tham số hoá, và NAS

Nén sau luôn thua thiết kế tốt, vì thiết kế đổi được cấu trúc luồng dữ liệu còn nén thì không. Dòng MobileNet/ShuffleNet/EfficientNet [14], [15] là các cách khác nhau để giảm số byte đi qua trên mỗi đơn vị sức chứa.

Đáng chú ý riêng là **tái tham số hoá** (*re-parameterization*) kiểu RepVGG: huấn luyện với một khối nhiều nhánh (dễ tối ưu), rồi *gộp toán học* các nhánh thành một tích chập đơn khi triển khai. Vì phép gộp là chính xác chứ không xấp xỉ, bạn được cả hai: động lực học huấn luyện tốt và đồ thị suy luận

gọn. Đây đúng là mẫu hình “đẩy độ khó sang lúc huấn luyện” của B/03-giai-phau-he-thong, ở dạng thuần khiết nhất.

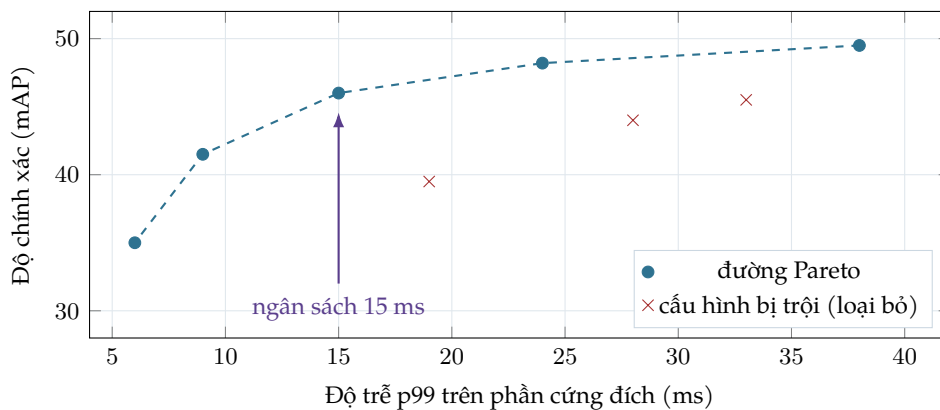
→ cùng nguyên lý với B/03 — độ khó không biến mất mà bị đẩy đi chỗ khác — ở đây là đẩy từ lúc suy luận sang lúc huấn luyện, dạng thuần khiết nhất của nguyên lý đó.

Về NAS thì phát biểu trung thực nhất là: nó hoạt động, nhưng phần lớn dự án không nên dùng. Chi phí tìm kiếm thường lớn hơn lợi ích. Điểm quan trọng hơn: kết quả gắn chặt với phần cứng dùng để tìm. Một kiến trúc NAS tối ưu cho GPU máy chủ có thể không tối ưu chút nào trên NPU của điện thoại.

E. Nguyên tắc bao trùm: đo trên phần cứng đích

Mỗi kỹ thuật trong chương phải kèm phép đo độ trễ thật trên đúng phần cứng sẽ chạy. FLOPs là đại diện tồi, và điều tệ hơn là nó sai theo hướng khó đoán: một mô hình ít FLOPs hơn có thể chậm hơn vì nó có nhiều op nhỏ hơn, vì bố cục bộ nhớ của nó không hợp với kernel sẵn có, hoặc vì nó dùng một toán tử mà runtime phải chạy trên CPU.

Sản phẩm cuối của một chu kỳ nén không phải một mô hình mà là một đường Pareto như Hình 2.2: tập các điểm mà không điểm nào bị điểm khác trội hơn ở cả hai trục. Chỉ khi có đường này bạn mới chọn được một cách có lý do. Câu hỏi lúc đó không còn là “mô hình nào tốt nhất”, mà là “đổi bao nhiêu độ chính xác lấy bao nhiêu mili-giây”. Câu hỏi thứ hai có câu trả lời.



Hình 2.2. Sản phẩm của một chu kỳ nén (số liệu minh họa). Các dấu chéo là những cấu hình bị trội — vừa chậm hơn vừa kém hơn một cấu hình khác — và chúng bị loại bỏ ngay, không cần thảo luận. Quyết định chỉ còn là chọn một điểm trên đường, và ngân sách mili-giây quyết định điểm nào.

2.4 Thực hành

Repo và tài nguyên. Lượng tử hoá và sparsity: [pytorch/ao](#) (torchao), [NVIDIA/TensorRT-Model-Optimizer](#), [openvinotoolkit/nncf](#), [intel/neural-compressor](#), [bitsandbytes](#). Tỉa: [VainF/Torch-Pruning](#). Chứng cất cho thị giác: công thức có sẵn trong [timm](#); ngoài ra nên đọc các bài chứng cất gần đây trong chính lĩnh vực của bạn, vì chi tiết phụ thuộc bài toán nhiều hơn người ta tưởng. Danh sách này chốt tại tháng 9/2026 và tên các bộ công cụ dẫn đầu sẽ đổi trong 6–12 tháng; các repo hạ tầng (torchao, TensorRT, OpenVINO) bền hơn hẳn tên của thuật toán thời thượng.

Câu hỏi kiểm tra hiểu. Vì sao tỉa không cấu trúc đạt sparsity 90% mà không nhanh hơn trên GPU? Một calibration set 100 ảnh chọn sai thì hỏng theo cơ chế nào, và triệu chứng nhìn thấy được là gì? Khi nào chứng cất thắng lượng tử hoá, và ngược lại?

Mini project (vài giờ đến hai ngày). Lấy một classifier và áp lần lượt bốn bước: giảm độ phân giải 20%, đổi sang backbone hiệu quả, PTQ INT8, rồi QAT. Sau *mỗi* bước đo cả độ chính xác lẫn độ trễ thật trên phần cứng đích. Vẽ đường Pareto và chọn một điểm, kèm một đoạn giải thích vì sao chọn nó. Phần đáng giá nhất của bài này là phát hiện ra bước nào không cho lợi ích như bảng hứa hẹn — và tìm ra lý do.

2.5 Giới hạn và trường hợp hỏng

- **Mọi con số trong chương phụ thuộc phần cứng, và phụ thuộc mạnh.** Một kết quả đo trên GPU máy chủ không dự báo được gì về NPU điện thoại hay về DLA của Jetson. Chuyển chip thì phải đo lại từ đầu, không phải điều chỉnh.
- **Độ chính xác trung bình che giấu thiệt hại tập trung.** Lượng tử hoá và giảm độ phân giải thường lấy đi hiệu năng ở đúng những nhóm hiếm — vật nhỏ, lớp ít mẫu, điều kiện chiếu sáng bất thường. Nếu bạn chỉ theo dõi một con số tổng, bạn sẽ không thấy.
- **Nén nhiều bước không cộng tuyến tính.** Lượng tử hoá một mô hình đã tĩa thường tệ hơn dự đoán, vì tĩa làm phân bố trọng số lệch hơn và do đó khó lượng tử hoá hơn. Thứ tự áp dụng cũng là một siêu tham số.
- **Chưng cất thừa hưởng cả lỗi của teacher.** Student học cả những chỗ teacher sai một cách tự tin, và vì nó nhỏ hơn nên nó không có đủ năng lực để tự sửa.
- **Benchmark ngắn nói dối trên thiết bị bị giới hạn nhiệt.** Đây là chủ đề của chương 21, nhưng nó ảnh hưởng ngược lại mọi phép đo trong chương này.

CẠM BÃY

FLOPs không phải độ trễ, và tin vào FLOPs là lỗi tốn thời gian nhất trong chương. Ba tình huống nó sai theo hướng ngược nhau: một mô hình ít FLOPs với hàng trăm op nhỏ có thể chậm hơn một mô hình nhiều FLOPs với vài op lớn, vì chi phí điều phối trội hơn chi phí tính; một tích chập tách theo chiều sâu có FLOPs rất thấp nhưng tỉ lệ tính trên byte cũng rất thấp, nên nó memory-bound và không nhanh như bảng hứa; và một toán tử mà runtime không hỗ trợ sẽ bị đẩy về CPU, nuốt trọn mọi lợi ích mà chẳng xuất hiện trong bất kỳ phép đếm FLOPs nào.

CẦN NHỚ

Thứ tự đúng là: giảm độ phân giải → đổi kiến trúc → lượng tử hoá → chưng cất → tĩa; và mỗi bước phải kèm một phép đo độ trễ thật trên phần cứng đích, ở chế độ nguồn và nhiệt mà bạn sẽ thực sự chạy. Sản phẩm cuối không phải một mô hình mà là một đường Pareto — chỉ khi có nó bạn mới đang *chọn* thay vì *đoán*. [T2]

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Độ phân giải	Kích thước ảnh đưa vào mạng; chi phí tỉ lệ với số điểm ảnh. Hạ từ 640 xuống 512 cắt 36% chi phí bằng một dòng config, đổi lại mất vật nhỏ.
••• Memory-bound so với compute-bound	Memory-bound là bị chặn bởi băng thông, compute-bound là bị chặn bởi năng lực tính. Phần lớn mạng thị giác memory-bound, và đó là lý do lượng tử hoá có tác dụng.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●●● Lượng tử hoá	Biểu diễn trọng số và kích hoạt bằng số nguyên ít bit, thường INT8. Giảm số byte phải nạp; cái giá là sai số làm tròn tích lũy qua các tầng.
●●● Vì sao calibration set sai làm tụt độ chính xác?	Nó quyết định dải giá trị mà hệ số tỉ lệ phủ. Lấy toàn một điều kiện chiếu sáng thì dải hẹp hơn thực tế, nên mọi giá trị ngoài dải bị cắt cụt khi triển khai.
●● PTQ	Lượng tử hoá sau huấn luyện, chỉ cần calibration set và vài phút. Luôn thử trước; chỉ chuyển sang QAT khi nó tụt quá ngưỡng bạn đặt trước.
●● QAT	Mô phỏng làm tròn ngay trong lúc huấn luyện để mạng học cách chịu đựng. Giữ độ chính xác tốt hơn ở bit thấp; cái giá là một chu kỳ huấn luyện đầy đủ.
●●● Per-tensor so với per-channel	Per-tensor dùng một hệ số tỉ lệ cho cả tầng, per-channel dùng một hệ số cho mỗi kênh. Per-channel gần như luôn đúng cho trọng số conv và linear.
●● Vì sao một kênh ngoại lai đủ phá per-tensor?	Hệ số tỉ lệ đặt theo $\max w $. Với ngoại lai 2,0, một trọng số 0,1 chỉ còn sáu mức — chưa tới ba bit hiệu dụng cho phần lớn tầng.
●●● Vì sao tia không cấu trúc không cho tốc độ thật?	Số không rải rác không giảm số byte phải nạp; bạn còn phải nạp thêm chỉ số để biết chỗ nào là không. Ngoài ra kernel dense đã được tối ưu nhiều năm còn kernel thưa thì chưa.
●● 2:4 sparsity	Trong mỗi nhóm bốn trọng số, đúng hai bằng không. Cho tốc độ thật, nhưng chỉ trên phần cứng có đơn vị sparse tensor.
●●● Chưng cất	Huấn luyện student nhỏ bắt chước phân phối mềm của teacher thay vì học nhãn cứng. Giữ được nhiều chất lượng nhất trên mỗi MB; student thừa hưởng cả chỗ teacher sai.
●● Vì sao phân phối mềm giàu thông tin hơn nhãn cứng?	Nó mang quan hệ giữa các lớp: sói gần chó hơn mèo. Nhãn cứng vứt đúng phần thông tin đó đi.
●● Tái tham số hoá	Huấn luyện với khối nhiều nhánh rồi gộp chính xác thành một tích chập đơn khi triển khai. Được cả động lực học tốt lẫn đồ thị suy luận gọn.
● NAS	Tự động tìm kiến trúc. Phần lớn dự án không nên dùng: chi phí tìm thường lớn hơn lợi ích, và kết quả gắn chặt với phần cứng dùng để tìm.
●●● Đường Pareto	Tập các cấu hình mà không cấu hình nào trội hơn ở cả độ chính xác lẫn độ trễ. Đây mới là sản phẩm của một chu kỳ nén, không phải một mô hình đơn lẻ.
●● Tia 90% mà độ trễ không đổi?	Bạn đã tia không cấu trúc. Số không rải rác không giảm số byte phải nạp; chỉ tia cả kênh hoặc mẫu 2:4 mới cho tốc độ.
●● Mô hình ít FLOPs hơn nhưng chậm hơn trên Jetson?	Ba nghi phạm: quá nhiều op nhỏ nên chi phí điều phối trội; tích chập tách theo chiều sâu memory-bound; hoặc một op không được runtime hỗ trợ nên rơi về CPU.
●● Lượng tử hoá xong, độ chính xác trung bình không đổi mà hệ thống vẫn tệ đi?	Thiệt hại tập trung ở nhóm hiếm: vật nhỏ, lớp ít mẫu, điều kiện chiếu sáng bất thường. Phải đánh giá theo nhóm mới nhìn thấy.

●●● phải thuộc ●● nên nắm ● biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
TensorRT, pytorch/ao	Lượng tử hoá per-channel, hợp nhất kernel, chọn bố cục bộ nhớ — gần như toàn bộ mạch của chương	Hạ tầng; API đổi nhưng ý tưởng bên dưới ổn định
Sparsity 2:4 trên Tensor Core	Tia bán cấu trúc: ràng buộc mẫu đủ đều để phần cứng khai thác được	Ví dụ rõ nhất cho luận điểm “tốc độ đến từ cấu trúc, không từ tỉ lệ số không”

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
Dòng MobileNet, EfficientNet, RepVGG	Kiến trúc hiệu quả và tái tham số hoá — đẩy chi phí sang lúc huấn luyện	RepVGG là minh hoạ sạch nhất; ý tưởng gộp nhánh nay xuất hiện lại trong nhiều detector thời gian thực
Chưng cất mô hình nền tảng xuống student nhỏ	Dùng teacher đóng băng làm nguồn phân phối mềm để đưa mô hình lớp SAM hoặc DINO lên thiết bị	Quan sát thực hành phổ biến 2025–2026; tôi không kiểm chứng được con số cụ thể
bitsandbytes, weight-only INT4	Chỉ nén trọng số khi nút thất là băng thông đọc trọng số chứ không phải kích hoạt	Là chuẩn mực khi phục vụ LLM trên GPU đơn

MINI PROJECT SLAM · INT8 hoá tầng mô tả và đo đủ ba trục · 1 ngày

Mục tiêu	biến “lượng tử hoá thì nhanh hơn” thành ba con số cụ thể trên phần cứng của bạn. Rồi tự quyết định cuộc đánh đổi có đáng ở ngân sách mili-giây của bạn hay không.
Dữ liệu	mô hình mô tả từ mini project chương 19 (hoặc một mô hình mô tả có sẵn nếu bạn bỏ qua bước đó). Calibration set lấy từ một chuỗi <i>khác</i> với chuỗi đánh giá, để không tự lừa mình.
Các bước	đo baseline fp32 trên đủ ba trục: độ chính xác ghép cặp, độ trễ p50 và p99 trên Orin ở đúng chế độ nvmodel sẽ triển khai, và dung lượng tệp trọng số. Rồi lần lượt PTQ per-tensor, PTQ per-channel, và QAT nếu PTQ tụt quá ngưỡng bạn đặt trước. Sau <i>mỗi</i> bước đo lại đủ ba trục, không suy đoán từ bước trước.
Xong khi	một bảng bốn dòng — fp32, PTQ per-tensor, PTQ per-channel, QAT — nhân ba cột: độ chính xác ghép cặp, p99, dung lượng. Kèm một câu kết luận <i>có số</i> : “đổi x điểm ghép cặp lấy y ms và z MB, đáng hoặc không đáng ở ngân sách của tôi”. Nếu per-tensor tụt mạnh hơn per-channel, chỉ ra được kênh ngoại lai nào gây ra điều đó.
Nối với	D/20 cho bảng thứ tự đòn bẩy và ví dụ per-tensor so với per-channel; D/21 cho kỷ luật đo (warm-up, phân vị, tải nhiệt) mà mọi con số ở đây phải tuân theo.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì phần cứng không tiết kiệm được gì từ các số không rải rác: bạn vẫn phải nạp dữ liệu và nạp thêm chỉ số để biết chỗ nào là không, kernel dense đã được tối ưu nhiều năm còn kernel thưa thì chưa, và bản thân chỉ số cũng ăn băng thông. Tốc độ chỉ đến khi cấu trúc đủ đều — 2:4 hoặc tía cả kênh.
- A2. Vì nó đảo ngược được trong một dòng config và cho lợi ích gần như bậc hai theo cạnh ảnh: 640 → 512 cắt 36% chi phí ngay. Nguyên tắc xếp thứ tự là chi phí của việc *sai*, không phải độ tinh vi: sai ở đây tốn nửa giờ, sai ở chưng cất tốn một tuần.
- A3. Vì một hệ số tỉ lệ per-tensor được đặt bởi $\max |w|$. Với ngoại lai 2,0 thì $s = 2,0/127$, nên một trọng số 0,1 chỉ được ánh xạ thành mức 6: toàn bộ dải bình thường còn sáu mức, chưa tới ba bit. Per-channel cho mỗi kênh một s riêng và trả lại đủ 8 bit.
- A4. Ít nhất ba cách. Một: nhiều op nhỏ làm chi phí điều phối trội hơn chi phí tính. Hai: tích chập tách theo chiều sâu có FLOPs thấp nhưng tỉ lệ tính trên byte cũng thấp, nên nó memory-bound. Ba: một toán tử runtime không hỗ trợ bị đẩy về CPU, nuốt trọn lợi ích mà không xuất hiện trong bất kỳ phép đếm FLOPs nào.

NÓI LẠI THẬT ĐƠN GIẢN

Bạn phải nhét một tủ sách vào một chiếc vali có hạn cân. Đó là toàn bộ chương này: mô hình tốt nhất của bạn không vừa cái máy mà nó phải chạy trên. Có năm cách giảm cân, và thứ tự quan trọng hơn kỹ thuật của từng cách. Rẻ nhất là bỏ bớt những cuốn bạn không đọc — tương đương với việc đưa ảnh nhỏ hơn vào máy; làm trong một phút, hồi hận thì làm lại ngay. Kế đó là mua bản bìa mềm thay bìa cứng: cùng nội dung, ít giấy hơn. Rồi mới

đến in chữ nhỏ lại một chút, tức ghi mỗi con số bằng ít chữ số hơn. Cách này rất hiệu quả. Nhưng chỉ cần một trang có cỡ chữ khổng lồ là cả cuốn phải in nhỏ theo, và mọi trang khác thành khó đọc. Sau nữa là nhờ người đọc hộ rồi viết bản tóm tắt. Cuối cùng, cách nghe hay nhất mà thường vô ích nhất: xé bớt chữ trên trang. Bạn xoá được chín phần mười chữ, nhưng vẫn phải mang đủ số tờ giấy, nên vali chẳng nhẹ đi.

Cái giá thì luôn có, và nó không rơi đều: thứ mất đi trước tiên là những cuốn hiếm, những chi tiết nhỏ, những trường hợp bạn ít gặp nhưng lại quan tâm nhất. Và con số duy nhất đáng tin là cái cân ở sân bay, không phải con số bạn tự ước ở nhà.

Một câu: Nén mô hình là xếp vali: làm theo thứ tự rẻ tiền trước, và chỉ tin cái cân ở sân bay chứ đừng tin con số bạn ước ở nhà.

2.6 Kết nối

Chương này là ứng dụng trực tiếp của B/11-gioi-han-tinh-toan: mọi kỹ thuật ở đây chỉ là cách di chuyển trên biểu đồ *roofline*, và biết mình đang ở nhánh nào của biểu đồ đó quyết định kỹ thuật nào có tác dụng. Nó song song với C/18/02-kinh-te-hoc-pretraining: cùng một logic ngân sách, chỉ khác là cột chi đổi từ compute huấn luyện sang compute **suy luận** (*inference*) — và tái tham số hoá cùng chứng cất chính là hai cách chuyển tiền giữa hai cột đó. Nó là tiền đề bắt buộc của D/21-inference-va-trien-khai, nơi mô hình đã nén được đem đi *export* và đo thật, và nơi bạn sẽ phát hiện rằng phần lớn thời gian không nằm ở mô hình. Cuối cùng nó phụ thuộc B/07-chia-du-lieu-va-danh-gia: không có tập đánh giá phân tách theo nhóm hiếm thì bạn không nhìn thấy cái giá thật của việc nén.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao tia không cấu trúc cho sparsity 90% mà không nhanh hơn trên GPU phổ thông, và ba lý do đó lý do nào là lý do phần cứng thật sự?
2. Trong ví dụ per-tensor với một kênh ngoại lai, phần lớn trọng số còn lại bao nhiêu bit hiệu dụng, và per-channel sửa điều đó bằng cách nào?
3. Vì sao giảm độ phân giải đứng đầu danh sách dù nó là kỹ thuật “thô” nhất, và cái giá của nó thường bị bỏ sót ở đâu trong quy trình đánh giá?
4. Khi nào chưng cất thăng lượng tử hoá, và khi nào ngược lại — lập luận qua ràng buộc nào đang chặn bạn?
5. Một mô hình có ít FLOPs hơn nhưng chạy chậm hơn trên Jetson: ba nguyên nhân khả dĩ nhất là gì?
6. Vì sao lượng tử hoá một mô hình đã tia thường tệ hơn dự đoán, và điều đó nói gì về việc coi các kỹ thuật nén là độc lập với nhau?

Nguồn

- [1] Y. Wu and K. He, “Group Normalization,” in *ECCV*, 2018.
- [2] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer Normalization,” *arXiv:1607.06450*, 2016.
- [3] N. Srivastava, G. Hinton, A. Krizhevsky, et al., “Dropout: A Simple Way to Prevent Neural Networks from Overfitting,” *JMLR*, vol. 15, pp. 1929–1958, 2014.
- [4] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986.
- [5] S. Rajbhandari, J. Rasley, O. Ruwase, et al., “ZeRO: Memory Optimizations Toward Training Trillion Parameter Models,” *SC*, 2020.
- [6] P. Goyal, P. Dollár, R. Girshick, et al., “Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour,” *arXiv:1706.02677*, 2017.
- [7] P. Micikevicius, S. Narang, et al., “Mixed Precision Training,” in *ICLR*, 2018.
- [8] G. Hinton, O. Vinyals, and J. Dean, “Distilling the Knowledge in a Neural Network,” in *NeurIPS Deep Learning Workshop*, 2015.
- [9] M. Nagel, M. Fournarakis, R. A. Amjad, et al., “A White Paper on Neural Network Quantization,” *arXiv:2106.08295*, 2021.
- [10] B. Jacob, S. Kligys, B. Chen, et al., “Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference,” in *CVPR*, 2018.
- [11] S. Han, H. Mao, and W. J. Dally, “Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding,” in *ICLR*, 2016.
- [12] J. Frankle and M. Carbin, “The Lottery Ticket Hypothesis: Finding Sparse, Trainable Neural Networks,” in *ICLR*, 2019.
- [13] H. Touvron, M. Cord, M. Douze, et al., “Training Data-Efficient Image Transformers and Distillation Through Attention,” in *ICML*, 2021.
- [14] A. G. Howard, M. Zhu, et al., “MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications,” *arXiv:1704.04861*, 2017.
- [15] M. Tan and Q. V. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” in *ICML*, 2019.
- [16] D. Sculley, G. Holt, D. Golovin, et al., “Hidden Technical Debt in Machine Learning Systems,” *NeurIPS*, 2015.
- [17] N. Bodla, B. Singh, R. Chellappa, et al., “Soft-NMS: Improving Object Detection with One Line of Code,” in *ICCV*, 2017.
- [18] P. Henderson, R. Islam, P. Bachman, et al., “Deep Reinforcement Learning That Matters,” *AAAI*, 2018.

Inference và triển khai (Inference and Deployment)

PHỤ THUỘC D/20-toi-uu-va-nen-mo-hinh (mô hình đã nén là đầu vào của chương này); B/11-gioi-han-tinh-toan (bảng thông, chỉ phí điều phối); B/08-dich-chuyen-phan-phoi (vì sao chỉ phân phối thật mới lộ ra lỗi thật); D/19/02-pytorch-thanh-thao (kỹ năng phân biệt “lệch do bug” với “lệch do số học”).

TÓM TẮT

Mô hình chạy tốt trong notebook và hỏng trong production. Chương này liệt kê từng chỗ nó hỏng, theo đúng trình tự bạn sẽ gặp: export, sai lệch số học sau export, thời gian nằm ở chỗ bạn không ngờ, xung đột giữa thông lượng và độ trễ đuôi, giới hạn nhiệt của thiết bị biên, và cuối cùng là rủi ro của việc thay mô hình. Đọc xong bạn thiết kế được một quy trình triển khai có kiểm chứng ở từng bước thay vì một chuỗi thao tác hy vọng. Nếu chỉ nhớ một điều: mô hình không có một con số độ trễ — *hệ thống ở một trạng thái* mới có, nên mọi phép đo phải nêu rõ trạng thái nguồn, nhiệt và tải kèm theo, và phải báo cáo p99 chứ không phải trung bình.

KIẾN THỨC CẦN CÓ TRƯỚC

- **export** — quá trình chuyển mô hình từ mã Python/PyTorch sang một biểu diễn đồ thị tĩnh mà một *runtime* khác nạp được — ONNX là định dạng trung gian phổ biến nhất.
- **opset và plugin** — *opset* là phiên bản tập toán tử mà ONNX định nghĩa; toán tử của bạn không nằm trong đó thì phải viết *plugin* cho runtime, hoặc chấp nhận nó bị thay bằng một tổ hợp xấp xỉ.
- **engine** — kết quả biên dịch của TensorRT cho một tổ hợp cụ thể gồm phiên bản thư viện, kiến trúc GPU và cấu hình shape. Đối bất kỳ thứ nào trong ba thứ đó là phải dựng lại.
- **dynamic shape và optimization profile** — shape đầu vào thay đổi lúc chạy (batch, độ phân giải). TensorRT đòi bạn khai báo trước dải shape (*optimization profile*) để biên dịch kernel phù hợp; không khai báo thì shape bị đóng băng lúc trace.
- **độ trễ p50 / p99** — phân vị của phân phối độ trễ — p99 là ngưỡng mà 99% yêu cầu nằm dưới. Trung bình che giấu đuôi, và đuôi mới là thứ người dùng cảm nhận.
- **batching động (dynamic batching)** — gom các yêu cầu đến gần nhau về thời gian thành một batch để tăng thông lượng, kèm một trần thời gian chờ để không phá ngân sách độ trễ.
- **KV cache** — bộ nhớ đệm lưu khoá và giá trị *attention* của các token đã sinh, để mỗi token mới không phải tính lại toàn bộ chuỗi. Đổi bộ nhớ lấy tốc độ.
- **shadow deployment** — chạy mô hình mới song song với mô hình cũ trên lưu lượng thật nhưng *không* dùng kết quả của nó để phục vụ; chỉ để so sánh và phát hiện lỗi trước khi chuyển đổi.

CÂU HỎI MỞ ĐẦU

- Q1. Vì sao con số độ trễ đo trong notebook gần như luôn sai khi lên thiết bị thật, và nó sai theo hướng nào?
- Q2. Bạn tối ưu lượt xuôi nhanh hơn 30% nhưng hệ thống chỉ nhanh hơn 7% — phần còn lại đã trốn ở đâu?
- Q3. Gộp *batch* làm thông lượng tăng gấp ba; vì sao nó vẫn có thể khiến hệ thống của bạn vi phạm ngân sách?

- Q4.** Cùng một engine, cùng một chip, cùng một đầu vào — vì sao phút thứ tám cho con số khác hẳn phút đầu tiên?
- Q5.** Sau khi export, đầu ra lệch 10^{-2} : làm sao biết đó là một bug hay chỉ là số học?

3.1 Đặt vấn đề

Chương này viết cho người đã triển khai thật, nên nó không giải thích lại độ trễ là gì. Câu hỏi dẫn dắt hẹp hơn và khó hơn: *mô hình chạy tốt trong notebook thì hỏng ở đâu khi ra production, và theo thứ tự nào?*

Điểm khiến việc triển khai deep learning khác với việc triển khai một thư viện C++ thông thường không phải độ phức tạp mà là *số lớp dịch*. Giữa mô hình bạn huấn luyện và thứ chạy trên thiết bị có ít nhất ba lần dịch — đồ thị Python sang ONNX, ONNX sang engine của runtime, engine sang kernel cụ thể — và mỗi lần dịch là một cơ hội để ngữ nghĩa thay đổi im lặng. Cộng thêm việc phần lớn phép đo hiệu năng trên GPU dễ đo sai theo những cách đã quen thuộc với người viết CUDA, và bạn có một hệ thống mà mọi con số đều phải được nghi ngờ cho tới khi có bằng chứng.

Học xong chương này thì làm được gì mà trước đó không làm được? Bạn đặt được một quy trình triển khai trong đó *mỗi bước có một phép kiểm chứng riêng*: so khớp từng tầng sau export, đo từng khối trong pipeline, đo p99 dưới tải nhiệt liên tục. Khi có sự cố, bạn khoanh vùng trong một giờ thay vì bởi cả chuỗi.

TRỰC GIÁC

Một mô hình không có độ trễ. Một hệ thống ở một trạng thái mới có độ trễ. Cùng một engine TensorRT trên cùng một Orin cho các con số khác nhau khi: chưa chạy lần nào (kernel autotune và cấp phát chưa xong), vừa chạy được ba mươi giây (mát, xung nhịp cao nhất), chạy liên tục tám phút (đã tụt xung vì nhiệt), hoặc chạy khi tiến trình SLAM bên cạnh đang ăn băng thông bộ nhớ chung.

Đây chính là bài học bạn đã học khi làm benchmark cho SLAM: một lần chạy không phải một phép đo, và hai lần chạy khác nhau không phải một sự bất thường mà là dữ liệu về phương sai. Sự khác biệt duy nhất là ở đây phương sai có thêm một nguồn vật lý — nhiệt độ — vốn có tính *trôi* chứ không phải *ngẫu nhiên*, nên nó không tự triệt tiêu khi bạn lấy trung bình nhiều lần lặp ngắn.

3.2 Mạch vấn đề → giải pháp, theo trình tự triển khai thật

Tám nút thắt, đúng thứ tự bạn sẽ gặp

1 Vấn đề

Giải pháp

Giá phải trả

PyTorch eager quá chậm và quá nặng để triển khai.

export sang một runtime chuyên dụng.

mất tính linh hoạt của Python và nhận về một tạo tác gắn chặt với phiên bản.

2 Vấn đề

Giải pháp

export xong thì đầu ra lệch.

so khớp từng tầng chứ không chỉ kết quả cuối, và kiểm tra bốn nguyên nhân theo đúng thứ tự.

- Giả định** bạn có một mẫu đầu vào cố định để so hai phía.
- 3 Vấn đề** **lượt xuôi** (*forward pass*) không phải chỗ tốn thời gian.
- Giải pháp** đo từng khối — giải mã, tiền xử lý, lượt xuôi, hậu xử lý — trước khi tối ưu bất cứ thứ gì.
- Hệ quả** trong hệ thống video, việc đưa giải mã và tiền xử lý lên GPU thường cho lợi ích lớn hơn mọi thứ bạn làm với mô hình.
- 4 Vấn đề** thông lượng và độ trễ xung đột.
- Giải pháp** batching động kèm trần thời gian chờ.
- Giá phải trả** p99 xấu đi khi hàng đợi dài, nên trần chờ là tham số thiết kế chứ không phải chi tiết cài đặt.
- 5 Vấn đề** mô hình sinh và VLM có cấu trúc chi phí khác hẳn.
- Giải pháp** KV cache, giảm số bước sampler, chưng cất sampler, giải mã suy đoán.
- Hệ quả** lại là mẫu hình “đẩy chi phí sang training-time” của chương 17 và 20.
- 6 Vấn đề** vận hành.
- Giải pháp** health check, autoscaling, warm-up bắt buộc, và báo cáo cả p50 lẫn p99.
- Giả định** bạn chấp nhận rằng lần chạy đầu luôn chậm và không cố giấu nó bằng cách bỏ qua mẫu đầu.
- 7 Vấn đề** thiết bị biên có giới hạn năng lượng và nhiệt.
- Giải pháp** đo trong mười phút liên tục ở đúng chế độ nguồn sẽ triển khai, không đo một trăm lần lặp.
- Hệ quả** benchmark ngắn nói dối một cách có hệ thống, luôn theo hướng lạc quan.
- 8 Vấn đề** thay mô hình mới là một canh bạc.
- Giải pháp** shadow deployment rồi A/B test.
- Giả định** chỉ phân phối thật mới lộ ra các lỗi mà tập test không có (B/08-dịch-chuyen-phan-phoi).

3.3 Export và chọn runtime

A. Bón họ runtime

Bảng 3.1. Bón họ runtime và cái mà mỗi cái đánh đổi.

Runtime	Tối ưu cho	Đánh đổi	Hỏng khi nào
ONNX Runtime	Trung lập, chạy được nhiều nơi	Không nhanh nhất ở bất kỳ đâu	Khi bạn thực sự cần ép sát ngân sách trên một chip cụ thể
TensorRT	GPU và DLA của NVIDIA	Khoá nhà cung cấp; build lâu; engine gắn với phiên bản và kiến trúc	Toán tử lạ phải viết plugin; nâng cấp JetPack là phải dựng lại toàn bộ
OpenVINO	CPU và iGPU của Intel	Hẹp về phần cứng	Ngoài hệ sinh thái Intel
ExecuTorch / CoreML / TFLite	Di động và nhúng	Tập toán tử hạn chế hơn hẳn	Mô hình dùng op động hoặc điều khiển luồng phức tạp

Với một hệ thống thời gian thực trên Orin, hai chi tiết quyết định nhiều hơn bảng trên. Thứ nhất, engine TensorRT phải được *đựng trên chính thiết bị đích* hoặc trên một môi trường trùng khít về phiên bản, vì nó nhúng cả lựa chọn kernel lẫn giả định về kiến trúc GPU. Thứ hai, Orin có DLA bên cạnh GPU; đẩy một phần đồ thị xuống DLA giải phóng GPU cho phần khác của hệ thống, nhưng DLA hỗ trợ ít toán tử hơn nên phần không chạy được sẽ rơi ngược về GPU và tạo ra các lần chuyển đổi tốn kém giữa hai bộ máy.

B. Sai lệch số học sau export — bốn nguyên nhân theo thứ tự

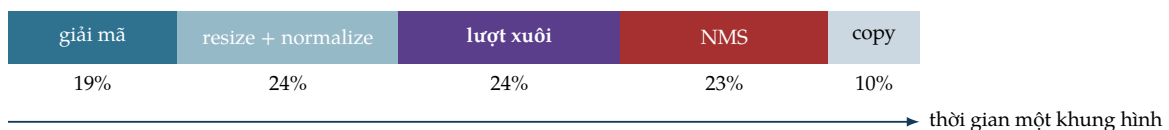
Đây là bước mà kỹ thuật quan trọng hơn kỹ thuật: *so khớp đầu ra từng tầng sau mỗi lần export, không chỉ so kết quả cuối*. Lý do giống hết lý do kiểm tra gradient từng tầng ở chương 19 — giữ không gian nghi vấn ở một phần tử. Khi đã thấy chỗ lệch, kiểm theo thứ tự này:

↪ **cùng nguyên lý với D/19/01** — so khớp từng tầng sau export là đúng một lập luận với kiểm tra gradient từng tầng: giữ không gian nghi vấn ở một phần tử thay vì ở tích các tương tác.

1. **Dynamic shape bị cố định lúc trace.** Triệu chứng đặc trưng: đúng với đúng một kích thước đầu vào, sai hoặc lỗi với mọi kích thước khác. Đây là nguyên nhân phổ biến nhất và cũng rẻ nhất để loại trừ.
2. **Toán tử không được hỗ trợ bị thay bằng bản xấp xỉ.** Triệu chứng: lệch tập trung ở một tầng cụ thể và không giảm khi bạn tăng độ chính xác. Kiểm bằng cách đọc log export — nó thường có cảnh báo mà không ai đọc.
3. **Khác biệt thứ tự phép toán ở fp16.** Triệu chứng: lệch nhỏ, phân tán đều, cỡ epsilon của fp16 và tích lũy dần theo độ sâu. Đây là *số học*, không phải bug — và phân biệt được hai thứ này là kỹ năng bạn đã luyện ở mini project của chương 19.
4. **Tiền xử lý khác nhau giữa hai phía.** Thứ tự kênh BGR so với RGB, giá trị mean/std, cách resize (song tuyến tính có căn góc hay không), và việc chia cho 255 hay không. Triệu chứng: lệch lớn và có hệ thống ngay từ tầng đầu tiên. Nghe tầm thường, nhưng đây là nguyên nhân tốn nhiều giờ người nhất trong danh sách, vì nó nằm ngoài mô hình nên không ai nghĩ tới nó khi đang debug mô hình [16].

3.4 Chỗ thời gian thực sự nằm

Kỳ vọng mặc định là lượt xuôi chiếm phần lớn thời gian. Trong một hệ thống thị giác thật, kỳ vọng đó thường sai. Hình 3.1 minh họa một phân rã điển hình cho một bộ phát hiện chạy trên luồng video — các con số là minh họa cho hình dạng, không phải một phép đo cụ thể.



Hình 3.1. Phân rã thời gian điển hình của một bộ phát hiện trên luồng video (*hình dạng minh họa, không phải số đo*). Phần được viết paper — lượt xuôi — thường chiếm khoảng một phần tư. Tối ưu 30% của một phần tư cho lợi ích tổng 7%; đưa giải mã và tiền xử lý lên GPU cho lợi ích lớn hơn nhiều, và đó là lý do các bộ công cụ như DALI hay DeepStream tồn tại.

Hai chỗ đáng chú ý riêng. **Triệt phi cực đại (non-maximum suppression, NMS)** là hậu xử lý tuần tự và phụ thuộc dữ liệu. Chi phí của nó tăng theo số hộp *trước* ngưỡng. Nên một ngưỡng tin cậy đặt thấp làm p99 xấu đi nhiều hơn p50, đúng ở những khung hình đông đúc mà bạn quan tâm nhất [17]. **Sao chép giữa CPU và GPU** thường bị bỏ qua hoàn toàn trong phép đo; trên Orin, bộ

nhớ hợp nhất cho phép tránh phần lớn các bản sao này, nhưng chỉ khi bạn cấp phát đúng loại bộ nhớ ngay từ đầu — một quyết định phải làm ở lúc thiết kế chứ không sửa được về sau.

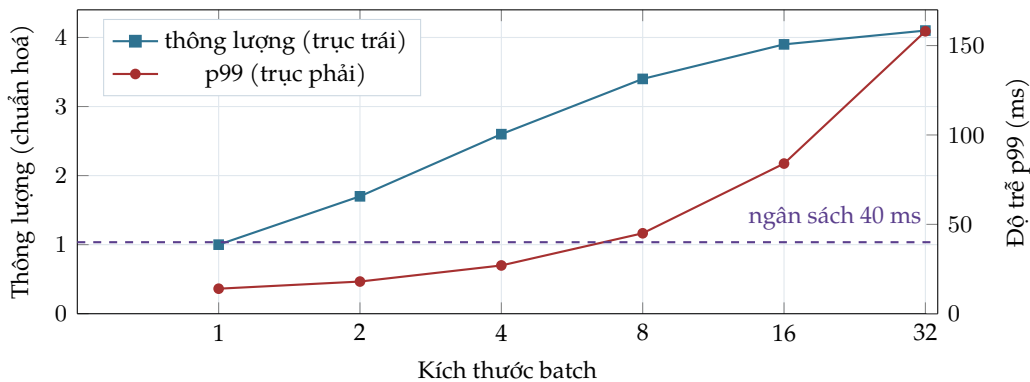
↪ **ràng buộc bởi** B/11 — trên Orin, GPU và CPU chia chung băng thông bộ nhớ, nên một tiến trình khác đang chạy đủ để đổi con số độ trễ mà không đổi một dòng mã nào.

3.5 Batching động và ngân sách độ trễ

Gộp batch làm thông lượng tăng và độ trễ tăng. Cách đúng để nghĩ về nó là coi trần thời gian chờ là một tham số thiết kế, rồi tính xem tốc độ đến của yêu cầu có đủ để lấp batch trong trần đó không.

Ví dụ nhỏ nhất tính tay được. Ngân sách p99 là 40 ms. Forward cho batch 8 tốn 20 ms, nên còn 20 ms cho hàng đợi. Muốn lấp đầy batch 8 trong 20 ms thì tốc độ đến phải ít nhất $8/0,02 = 400$ yêu cầu mỗi giây. Nếu thực tế chỉ có 100 yêu cầu mỗi giây, trong 20 ms bạn chỉ gom được 2 yêu cầu — nên bạn đang trả cái giá về độ trễ của batching mà chỉ nhận được lợi ích của batch 2.

Kết luận thiết kế: batching động chỉ có lãi khi tốc độ đến vượt ngưỡng $B/T_{\text{chờ}}$. Dưới ngưỡng đó, cấu hình đúng là batch nhỏ hoặc không batch, và mọi nỗ lực chỉnh tham số batching là vô ích. Hình 3.2 cho thấy hình dạng của cuộc đánh đổi.



Hình 3.2. Thông lượng bão hoà trong khi p99 tiếp tục tăng gần tuyến tính theo kích thước batch (số liệu minh hoạ). Sau batch 8 bạn gần như không mua thêm được thông lượng nhưng vẫn trả đủ giá về độ trễ — và đúng ở đó ngân sách bị phá.

3.6 Mô hình sinh và VLM — cấu trúc chi phí khác hẳn

Với mô hình sinh tự hồi quy, chi phí không nằm ở một lượt xuôi mà ở n lượt, mỗi lượt sinh một *token*. KV cache biến bài toán từ “tính lại toàn bộ chuỗi mỗi bước” thành “chỉ tính token mới”: đổi bộ nhớ lấy tốc độ. Nhưng lượng bộ nhớ ấy tăng tuyến tính theo độ dài chuỗi và theo số yêu cầu đồng thời. Nó vì thế trở thành ràng buộc chính của việc phục vụ. Với mô hình khuếch tán, đòn bẩy tương ứng là giảm số bước sampler và chứng cất sampler; với LLM là giải mã suy đoán. Cả ba đều là cùng một mẫu hình: chuyển công việc từ lúc chạy sang lúc huấn luyện.

Điểm cần nhớ cho một hệ thống robotics: đưa một VLM vào vòng điều khiển thời gian thực là một quyết định kiến trúc, không phải một tối ưu. Độ trễ của nó phụ thuộc độ dài đầu ra, tức phụ thuộc *nội dung*, nên nó không có chặn trên tự nhiên — và mọi thành phần thời gian thực nổi vào nó phải được thiết kế với một cơ chế hết giờ và một hành vi dự phòng.

3.7 Vận hành và cách đo cho đúng

Ba nguyên tắc, nêu ngắn vì bạn đã biết chúng ở phía C++:

- **Warm-up là bắt buộc, không phải mẹo.** Lăn chạy đầu gánh cấp phát bộ nhớ, autotune kernel của cuDNN, và nạp trọng số vào cache. Bỏ vài chục lần lặp đầu là chuẩn mực; không bỏ thì bạn đang báo cáo một phân phối trộn hai chế độ.
- **Đo phân vị, không đo trung bình.** Trung bình che giấu đúng thứ làm người dùng khó chịu và làm hệ thống điều khiển mất ổn định. p99 là con số ràng buộc thiết kế; p50 chỉ để biết hệ thống có đang lành mạnh không.
- **Đồng bộ hoá trước khi bấm giờ.** Lệnh CUDA là bất đồng bộ; một phép đo bằng đồng hồ tường quanh lời gọi chỉ đo thời gian xếp hàng.

3.8 Thiết bị biên: nơi benchmark ngắn nói dối

Trên Jetson và các NPU di động, hai ràng buộc vật lý chi phối mọi thứ: ngân sách năng lượng và tản nhiệt. Hệ quả trực tiếp là *một trăm lần lặp không phải một phép đo*. Chip chạy ở xung nhịp cao khi mát, rồi tụt xung khi nhiệt độ chạm ngưỡng; nếu vòng lặp benchmark của bạn kết thúc trước lúc đó, bạn đo được một con số mà hệ thống sẽ không bao giờ duy trì được.

Quy trình đo tối thiểu cho một thiết bị biên gồm bốn điều kiện, và cả bốn đều phải ghi vào báo cáo: chạy *liên tục ít nhất mười phút*; ở *đúng chế độ nguồn* sẽ triển khai (trên Orin là `nvpmodel`, và cần biết rằng `jetson_clocks` không tự giữ qua lần khởi động lại); *cùng với các tiến trình khác* sẽ chạy song song, vì chúng chia sẻ băng thông bộ nhớ; và ghi lại nhiệt độ cùng xung nhịp theo thời gian chứ không chỉ ghi độ trễ. Nếu đường độ trễ theo thời gian có một bậc thang, bạn vừa tìm thấy ngưỡng nhiệt — và con số đúng để báo cáo là con số *sau* bậc thang đó.

3.9 Thay mô hình an toàn

Thay mô hình mới là canh bạc vì tập test của bạn, dù tốt tới đâu, cũng không phải phân phối thật (B/08-dịch-chuyen-phan-phoi). Quy trình giảm rủi ro có hai bước và thứ tự không đảo được: **shadow deployment** — chạy song song trên lưu lượng thật, ghi lại kết quả nhưng không dùng chúng — rồi **A/B test** trên một phần lưu lượng. Shadow bắt được các lỗi kỹ thuật (tiền xử lý sai, shape sai, độ trễ vượt ngân sách, phân phối đầu ra lệch) mà không gây thiệt hại; A/B mới đo được tác động thật lên chỉ số nghiệp vụ. Bỏ qua bước shadow để đi thẳng vào A/B là cách phổ biến để phát hiện một lỗi tiền xử lý bằng chính người dùng.

↔ **tiền đề** B/08 — tập test là một mẫu cố định của quá khứ; chỉ lưu lượng thật mới bộc lộ các vùng phân phối đã trôi kể từ lúc thu dữ liệu.

3.10 Thực hành

Repo và tài nguyên. *Runtime:* [microsoft/onnxruntime](#), [NVIDIA/TensorRT](#), [openvinotoolkit/openvino](#), [pytorch/executorch](#). *Phục vụ:* [triton-inference-server/server](#), [Lightning-AI/LitServe](#). *Pipeline video:* [NVIDIA/DALI](#) và [DeepStream](#). *Phục vụ mô hình sinh*

và VLM: [vllm-project/vllm](https://github.com/vllm-project/vllm), [sgl-project/sglang](https://github.com/sgl-project/sglang). Danh sách chốt tại tháng 9/2026; các runtime hạ tầng ở đây bền hơn nhiều so với tên các bộ phục vụ LLM, vốn thay đổi từng quý.

Câu hỏi kiểm tra hiểu. Sau khi export ONNX, đầu ra lệch 10^{-2} so với PyTorch — bốn nguyên nhân theo thứ tự cần kiểm là gì, và triệu chứng phân biệt chúng? Vì sao p99 mới là con số ràng buộc thiết kế chứ không phải trung bình? Batching động cải thiện thông lượng nhưng làm hỏng gì, và ngưỡng tốc độ đến tối thiểu để nó có lãi được tính thế nào?

Mini project (vài giờ đến hai ngày). Export một bộ phát hiện sang ONNX rồi sang TensorRT. Ở mỗi bước, kiểm tra khớp đầu ra *từng tầng* và đo p50 cùng p99. Lập bảng thời gian theo từng khối: giải mã, tiền xử lý, lượt xuôi, NMS, sao chép. Kết quả đáng giá nhất của bài này thường là phát hiện rằng khối chiếm nhiều thời gian nhất không phải khối bạn định tối ưu.

3.11 Giới hạn và trường hợp hỏng

- **Engine gắn chặt với phiên bản.** Nâng JetPack, đổi phiên bản TensorRT, hay đổi biến thể GPU là phải dựng lại và *kiểm chứng lại* — không phải chỉ dựng lại. Đây là chi phí vận hành thường xuyên chứ không phải chi phí một lần.
- **So khớp từng tầng không bắt được lỗi phụ thuộc dữ liệu.** Một mẫu đầu vào cố định không kích hoạt nhánh điều khiển hiếm; NMS với số hộp bất thường, hoặc một shape ngoài optimization profile, vẫn có thể hỏng sau khi mọi phép so đã qua.
- **p99 đo trên lưu lượng tổng hợp không dự báo p99 thật.** Lưu lượng thật có cụm và có tương quan; một bộ sinh yêu cầu đều đặn cho phân phối hàng đợi lạc quan hơn nhiều so với thực tế.
- **Shadow deployment không phát hiện được vấn đề về vòng phản hồi.** Nếu mô hình mới thay đổi hành vi hệ thống và hành vi đó thay đổi phân phối đầu vào, chỉ A/B mới thấy — shadow theo định nghĩa không tác động gì nên không tạo ra vòng lặp đó.

CẠM BẮY

Ba cách đo sai làm bạn tin vào một con số không tồn tại. **Một:** bấm giờ bằng đồng hồ tường quanh một lời gọi CUDA mà không đồng bộ hoá — bạn đo thời gian *xếp hàng lệnh*, thường nhỏ hơn thời gian chạy thật một bậc. **Hai:** không warm-up, rồi lấy trung bình gồm cả lần chạy đầu vốn gánh autotune và cấp phát; con số ra vừa sai vừa có phương sai lớn giả tạo. **Ba:** benchmark một trăm lần lặp trên thiết bị biên đang mát, rồi báo cáo con số đó như hiệu năng bền vững. Hệ thống sẽ chạy chậm hơn ngay khi vào ca làm việc thật. Và không ai tái hiện được vấn đề, vì mọi người đều benchmark theo cách cũ.

CẦN NHỚ

Một phép đo độ trễ chỉ có nghĩa khi kèm trạng thái: chế độ nguồn, nhiệt độ đã bão hoà chưa, có tiến trình nào chạy cùng không, đã warm-up chưa, và đó là phân vị nào. Thiếu bất kỳ mục nào trong danh sách đó thì con số không tái hiện được và không dùng để ra quyết định được. Và nguyên tắc bao trùm cả chương: sau mỗi lần dịch — Python sang ONNX, ONNX sang engine — phải có một phép kiểm chứng riêng, vì mỗi lần dịch là một cơ hội để ngữ nghĩa đổi im lặng. [T2]

■ **Active recall** (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
●●● Export	Chuyển mô hình từ mã Python sang một đồ thị tĩnh mà runtime khác nạp được; ONNX là định dạng trung gian phổ biến nhất. Mỗi lần dịch là một cơ hội đổi ngữ nghĩa im lặng.
●● Opset và plugin	Opset là phiên bản tập toán tử ONNX; plugin là phần bạn phải tự viết khi runtime không có op đó. Không viết thì op bị thay bằng một xấp xỉ.
●●● Engine TensorRT	Kết quả biên dịch cho một tổ hợp cụ thể gồm phiên bản thư viện, kiến trúc GPU và dải shape. Đối bất kỳ thứ nào là phải dựng lại và kiểm chứng lại.
●● Optimization profile	Khai báo trước dải shape động để TensorRT sinh kernel phù hợp. Không khai báo thì shape bị đóng băng lúc trace.
●● Vì sao phải so khớp đầu ra từng tầng sau export?	Để giữ không gian nghi vấn ở một tầng thay vì cả chuỗi ba lần dịch. Cùng lập luận với kiểm tra gradient từng tầng ở chương 19.
●●● Lệnh sau export	Theo thứ tự kiểm: shape động bị cố định lúc trace; op không hỗ trợ bị thay bằng xấp xỉ; khác thứ tự phép toán ở fp16; và tiền xử lý lệch giữa hai phía.
●●● Triệt phi cực đại	Hậu xử lý tuần tự, phụ thuộc dữ liệu, chi phí tăng theo số hộp trước ngưỡng. Ngưỡng tin cậy thấp làm p99 xấu đi nhiều hơn p50.
●●● Vì sao phải warm-up trước khi đo?	Lần chạy đầu gánh cấp phát bộ nhớ, autotune kernel và nạp trọng số vào cache. Không bỏ chúng đi thì bạn báo cáo một phân phối trộn hai chế độ.
●●● Vì sao p99 mới là con số ràng buộc thiết kế?	Vì trung bình che đậy cái đuôi mà người dùng cảm nhận và mà hệ điều khiển phải chịu. p50 chỉ để biết hệ thống có đang lành mạnh không.
●● Batching động	Gom yêu cầu đến gần nhau thành một batch, kèm trần thời gian chờ. Chỉ có lỗi khi tốc độ đến vượt $B/T_{chờ}$.
●● KV cache	Lưu khoá và giá trị attention của các token đã sinh để không tính lại chuỗi mỗi bước. Bộ nhớ tăng tuyến tính theo độ dài chuỗi và số yêu cầu đồng thời.
●● DLA trên Orin	Bộ máy suy luận riêng bên cạnh GPU, dùng để giải phóng GPU cho phần khác của hệ thống. Hỗ trợ ít op hơn, nên phần không chạy được rơi ngược về GPU.
●● Vì sao bấm giờ quanh một lời gọi CUDA cho số sai?	Lệnh CUDA bất đồng bộ, nên đồng hồ tường chỉ đo thời gian xếp hàng. Phải đồng bộ hoá hoặc dùng CUDA event.
● Shadow deployment	Chạy mô hình mới song song trên lưu lượng thật nhưng không dùng kết quả. Bắt lỗi kỹ thuật mà không gây thiệt hại; không bắt được vấn đề vòng phản hồi.
●● Thông lượng tăng gấp ba mà p99 vượt ngân sách?	Batch quá lớn. Thông lượng bão hoà trong khi p99 vẫn tăng gần tuyến tính, nên sau một điểm bạn trả giá mà không mua thêm được gì.
●● Đầu ra đúng với một kích thước ảnh, sai với mọi kích thước khác?	Shape động đã bị cố định lúc trace, hoặc thiếu optimization profile. Đây là nguyên nhân phổ biến nhất và rẻ nhất để loại trừ.
●● Độ trễ ổn định tám phút rồi nhảy một bậc?	Chip chạm ngưỡng nhiệt và tụt xung. Con số đúng để báo cáo là con số sau bậc thang đó.
●● Tối ưu lượt xuôi nhanh hơn 30% mà hệ thống chỉ nhanh hơn 7%?	Lượt xuôi chỉ chiếm khoảng một phần tư thời gian. Phần còn lại là giải mã, tiền xử lý, triệt phi cực đại và sao chép giữa CPU với GPU.

●●● phải thuộc ●● nên nắm ● biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
ONNX Runtime, TensorRT, OpenVINO	Toàn bộ mạch export → engine → kernel, kể cả optimization profile cho shape động	Hạ tầng; ONNX là định dạng trung gian bền nhất trong nhóm
Triton Inference Server	Batching động kèm trần thời gian chờ — đúng cuộc đánh đổi thông lượng với p99 ở mức batching	Hạ tầng phục vụ; cơ chế batching của nó là hiện thực sạch của bất đẳng thức $B/T_{\text{chờ}}$
vllm, sglang	KV cache và lập lịch yêu cầu cho mô hình sinh, tức đổi bộ nhớ lấy tốc độ ở quy mô nhiều người dùng	Nhóm công cụ đổi nhanh nhất trong danh sách; ý tưởng KV cache thì không đổi
DALI, DeepStream	Đưa giải mã và tiền xử lý lên GPU vì chúng, chứ không phải lượt xuôi, mới chiếm phần lớn thời gian	Đúng kết luận của Hình 3.1, đóng gói thành sản phẩm
DLA trên Jetson Orin	Đẩy một phần đồ thị sang bộ máy khác để giải phóng GPU cho phần còn lại của hệ thống	Chỉ có lái khi toàn bộ nhánh chạy được trên DLA; op không hỗ trợ rơi ngược về GPU và tạo chuyển đổi tốn kém

MINI PROJECT SLAM • Phân rã độ trễ của một pipeline SLAM có thành phần học • 1–2 ngày

Mục tiêu	tìm ra khối nào thật sự ăn thời gian trong hệ của bạn. Rồi đo nó theo cách mà con số còn đúng sau mười phút chạy liên tục, không chỉ trong một trăm lần lặp.
Dữ liệu	hệ SLAM của bạn chạy trên Orin với một chuỗi EuRoC phát lại ở tốc độ thật, có cảm tăng mô tả INT8 từ mini project chương 20 vào frontend.
Các bước	bọc năm khối bằng CUDA event — giải mã và hiệu chỉnh méo, phát hiện đặc trưng, mô tả bằng mạng, ghép cặp cùng RANSAC, và tối ưu — chứ không bấm giờ bằng đồng hồ tường. Bỏ 200 khung hình đầu làm warm-up. Chạy liên tục mười phút ở đúng chế độ nvmodel sẽ triển khai, với các tiến trình thật chạy song song. Ghi lại nhiệt độ và xung nhịp theo thời gian cùng với độ trễ.
Xong khi	bạn chỉ ra được khối chiếm nhiều thời gian nhất, kèm phần trăm của nó. Bạn nói được p99 cao hơn trung bình bao nhiêu phần trăm. Và trên đồ thị độ trễ theo thời gian, bạn chỉ được bậc thang do tụt xung vì nhiệt — hoặc chứng minh bằng đường nhiệt độ rằng nó chưa xuất hiện trong mười phút.
Nối với	D/21 cho hình phân rã thời gian và cuộc đánh đổi thông lượng với p99; B/11 cho lý do bằng thông chung trên Orin làm con số đôi khi có tiến trình khác. Bộ đo dựng ở đây chính là thứ mini project chương 22 đem đi kiểm tính tái lập.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì notebook đo một hệ thống ở trạng thái tốt nhất: đã warm-up hoặc chưa, mát, không có tiến trình nào tranh băng thông, và thường bấm giờ bằng đồng hồ tường quanh một lệnh CUDA bất đồng bộ. Cả bốn sai lệch đều đi cùng một hướng — lạc quan.
- A2. Vào những khối bạn không đo: giải mã, resize và chuẩn hoá, triết phi cực đại, và các bản sao giữa CPU với GPU. Nếu lượt xuôi chỉ chiếm một phần tư thời gian thì 30% của một phần tư đúng bằng 7,5% tổng — số học chứ không phải nghịch lý.
- A3. Vì thông lượng bão hoà trong khi p99 tiếp tục tăng gần tuyến tính theo kích thước batch. Sau một điểm nào đó bạn gần như không mua thêm được thông lượng nhưng vẫn trả đủ giá độ trễ, và nếu tốc độ đến nhỏ hơn $B/T_{\text{chờ}}$ thì bạn còn không lấp nổi batch trong trần chờ.
- A4. Vì phút thứ tám là một trạng thái khác: chip đã chạm ngưỡng nhiệt và tụt xung. Đường độ trễ theo thời gian có một bậc thang, và con số đúng để báo cáo là con số *sau* bậc thang đó.
- A5. Bằng triệu chứng. Lệch do số học thì nhỏ, phân tán đều, cỡ epsilon của fp16 và tích lũy dần theo độ sâu. Lệch do bug thì tập trung ở một tầng và không giảm khi bạn tăng độ chính xác — hoặc lớn và có hệ thống ngay từ tầng đầu, dấu hiệu kinh điển của tiền xử lý lệch.

NÓI LẠI THẬT ĐƠN GIẢN

Một nhà hàng bị chê phục vụ chậm. Chủ quán liền thuê đầu bếp giỏi hơn, nấu nhanh hơn ba mươi phần trăm — và khách vẫn chê. Lý do: nấu chỉ chiếm một phần tư thời gian một món. Phần còn lại là nhặt rau, rửa, thái, xếp đĩa và bưng ra. Muốn nhanh thì phải bấm giờ từng khâu trước đã, chứ không phải cải thiện đúng cái khâu ai cũng nói tới.

Chuyện thứ hai là cách báo cáo. Nói “trung bình hai mươi phút một món” nghe rất ổn, nhưng khách nhớ cái lần họ chờ năm mươi phút. Con số ràng buộc thiết kế là cái lần xui xẻo đó, không phải trung bình.

Chuyện thứ ba là trạng thái của bếp. Bếp lúc mới mở khác hẳn bếp đã chạy tám phút liên tục: nóng lên thì mọi thứ chậm lại. Đo mười lượt lúc bếp còn nguội rồi công bố con số ấy là tự lừa mình.

Chuyện thứ tư là gộp đơn. Nấu một lượt cho tám bàn thì tổng số món phục vụ được nhiều hơn hẳn, nhưng bàn đến trước phải ngồi đợi cho đủ tám. Nếu quán chỉ có một trăm khách một giờ thì chờ cho đủ tám bàn là quá lâu, và bạn đang trả cái giá của gộp đơn mà không nhận được lợi ích của nó.

Cái giá chung của cả chương: mọi con số bạn đo chỉ đúng kèm điều kiện — bếp nóng hay nguội, đông hay vắng, đã chạy thử chưa.

Một câu: Đừng hỏi mô hình chạy nhanh bao nhiêu; hãy hỏi cả hệ thống, ở đúng trạng thái thật, phục vụ được người khách xui xẻo thứ chín mươi chín trong một trăm.

3.12 Kết nối

Chương này nhận đầu vào từ [D/20-toi-uu-va-nen-mo-hinh](#) và trả lại cho nó một bài học ngược: mọi phép đo dùng để vẽ đường Pareto ở chương 20 chỉ hợp lệ nếu được đo theo đúng kỷ luật của chương này, nên hai chương thực chất là một vòng lặp chứ không phải một chuỗi. Nó là ứng dụng của [B/11-gioi-han-tinh-toan](#) ở phía **suy luận** (*inference*), và nó dựa vào [B/08-dich-chuyen-phan-phoi](#) cho toàn bộ lập luận về shadow deployment. Về phía sau, nó là tiền đề của [D/22-tai-lap-va-mlops](#): một khi mô hình đã chạy trong production, câu hỏi đổi từ “nó có nhanh không” sang “sáu tháng nữa tôi có dựng lại được đúng nó không, và tôi có biết nó đang xuống cấp không”. Và nó nối thẳng sang [E/25-do-tin-cay-va-van-hanh](#) cho phần hành vi khi mô hình gặp đầu vào ngoài phân phối.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Sau khi export ONNX, đầu ra lệch 10^{-2} — bốn nguyên nhân theo thứ tự cần kiểm là gì, và triệu chứng nào phân biệt “lệch do số học” với “lệch do toán tử bị thay thế”?
2. Vì sao p99 là con số ràng buộc thiết kế còn trung bình thì không, và điều đó đổi cách bạn chọn ngưỡng tin cậy của NMS như thế nào?
3. Với ngân sách p99 là 40 ms và lượt xuôi batch 8 tốn 20 ms, tốc độ đến tối thiểu để batching động có lãi là bao nhiêu, và dưới ngưỡng đó cấu hình đúng là gì?
4. Vì sao một benchmark một trăm lần lặp trên Jetson nói dối, và phép đo tối thiểu để không bị nó lừa gồm những điều kiện nào?
5. Vì sao engine TensorRT phải dựng lại sau khi nâng JetPack, và vì sao “dựng lại” là chưa đủ?
6. Shadow deployment bắt được loại lỗi nào mà tập test không bắt được, và loại lỗi nào nó vẫn không bắt được?

- [1] Y. Wu and K. He, “Group Normalization,” in *ECCV*, 2018.
- [2] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer Normalization,” *arXiv:1607.06450*, 2016.
- [3] N. Srivastava, G. Hinton, A. Krizhevsky, et al., “Dropout: A Simple Way to Prevent Neural Networks from Overfitting,” *JMLR*, vol. 15, pp. 1929–1958, 2014.
- [4] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986.
- [5] S. Rajbhandari, J. Rasley, O. Ruwase, et al., “ZeRO: Memory Optimizations Toward Training Trillion Parameter Models,” *SC*, 2020.
- [6] P. Goyal, P. Dollár, R. Girshick, et al., “Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour,” *arXiv:1706.02677*, 2017.
- [7] P. Micikevicius, S. Narang, et al., “Mixed Precision Training,” in *ICLR*, 2018.
- [8] G. Hinton, O. Vinyals, and J. Dean, “Distilling the Knowledge in a Neural Network,” in *NeurIPS Deep Learning Workshop*, 2015.
- [9] M. Nagel, M. Fournarakis, R. A. Amjad, et al., “A White Paper on Neural Network Quantization,” *arXiv:2106.08295*, 2021.
- [10] B. Jacob, S. Kligys, B. Chen, et al., “Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference,” in *CVPR*, 2018.
- [11] S. Han, H. Mao, and W. J. Dally, “Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding,” in *ICLR*, 2016.
- [12] J. Frankle and M. Carbin, “The Lottery Ticket Hypothesis: Finding Sparse, Trainable Neural Networks,” in *ICLR*, 2019.
- [13] H. Touvron, M. Cord, M. Douze, et al., “Training Data-Efficient Image Transformers and Distillation Through Attention,” in *ICML*, 2021.
- [14] A. G. Howard, M. Zhu, et al., “MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications,” *arXiv:1704.04861*, 2017.
- [15] M. Tan and Q. V. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” in *ICML*, 2019.
- [16] D. Sculley, G. Holt, D. Golovin, et al., “Hidden Technical Debt in Machine Learning Systems,” *NeurIPS*, 2015.
- [17] N. Bodla, B. Singh, R. Chellappa, et al., “Soft-NMS: Improving Object Detection with One Line of Code,” in *ICCV*, 2017.
- [18] P. Henderson, R. Islam, P. Bachman, et al., “Deep Reinforcement Learning That Matters,” *AAAI*, 2018.

Tái lập và MLOps tối thiểu (Reproducibility and Minimal MLOps)

PHỤ THUỘC B/06-chat-luong-du-lieu và B/07-chia-du-lieu-va-danh-gia (dữ liệu là thứ thay đổi nhiều nhất); B/08-dich-chuyen-phan-phoi (vì sao cần giám sát sau triển khai); D/19/02-pytorch-thanh-thao (seed, chế độ tắt định); D/21-inference-va-trien-khai (mô hình đã ở production); E/24-thi-nghiem-va-ablation (một thay đổi = một config diff).

TÓM TẮT

Sáu tháng sau, bạn cần biết chính xác cái gì đã tạo ra con số này — mọi công cụ trong chương tồn tại vì trí nhớ con người không làm được việc đó. Chương đi qua config tập trung, theo dõi thí nghiệm, quản lý phiên bản dữ liệu, đóng gói artifact đầy đủ, các nguồn bất định trên GPU, CI cho mô hình, và giám sát drift sau triển khai. Đọc xong bạn dựng lại được một kết quả cũ chỉ từ một config, và — quan trọng hơn — biết khi nào việc dựng lại *từng chữ số* là mục tiêu sai. Nếu chỉ nhớ một điều: mục tiêu đúng không phải tái lập từng bit mà là tái lập *phân phối* kết quả, nên một con số không kèm phương sai không phải bằng chứng.

KIẾN THỨC CẦN CÓ TRƯỚC

- **seed** — số khởi tạo mọi bộ sinh ngẫu nhiên trong một lần chạy — khởi tạo trọng số, thứ tự trộn dữ liệu, augmentation, dropout. Cùng seed không đủ để cùng kết quả, nhưng khác seed thì chắc chắn khác kết quả.
- **config** — tệp mô tả toàn bộ siêu tham số và lựa chọn của một lần chạy, tách khỏi mã nguồn. Nếu hai lần chạy khác nhau mà không diff được config thì bạn không ablate được.
- **experiment tracking** — hệ thống ghi lại mỗi lần chạy kèm commit mã, config, phiên bản dữ liệu và các chỉ số theo thời gian, để sau này truy ngược được từ một con số về nguồn gốc của nó.
- **data versioning** — quản lý phiên bản cho dữ liệu giống như git quản lý mã: mỗi trạng thái của tập dữ liệu có một mã băm, và một lần chạy trở tới đúng mã băm đó.
- **artifact** — gói đầy đủ cần để tái hiện hoặc triển khai một mô hình: trọng số, config, phiên bản thư viện, phiên bản dữ liệu, và mã tiền xử lý. Chỉ có checkpoint là chưa đủ.
- **tính tắt định (determinism)** — chạy lại cùng đầu vào cho ra cùng đầu ra tới từng bit. Trên GPU nó không mặc định, và ép nó phải trả giá bằng tốc độ.
- **CI cho mô hình** — một tập kiểm tra rẽ chạy tự động mỗi commit — overfit một batch, kiểm shape, kiểm export thành công, kiểm chỉ số không tụt quá ngưỡng trên một tập nhỏ cố định.
- **drift** — sự trôi dạt của phân phối dữ liệu thật so với phân phối lúc huấn luyện. Nó làm mô hình xuống cấp mà không có lỗi nào được ném ra.

CÂU HỎI MỞ ĐẦU

- Q1. Bạn cố định *seed*, chạy lại đúng mã trên đúng máy, và vẫn nhận hai con số khác nhau — ai đã đổi thứ tự phép cộng sau lưng bạn?
- Q2. Vì sao ép mọi thứ tắt định lại là một mục tiêu *sai*, dù nó nghe đúng hết thứ một kỹ sư nên làm?

- Q3. Cấu hình mới hơn cấu hình cũ 0,3 mAP. Vì sao con số đó có thể hoàn toàn vô nghĩa, và cần bao nhiêu lần chạy để nó có nghĩa?
- Q4. Sáu tháng sau bạn có checkpoint, có mã, có config — và vẫn không dựng lại được kết quả cũ. Thứ còn thiếu thường là gì?

4.1 Đặt vấn đề

Câu hỏi dẫn dắt rất cụ thể và rất quen: *sáu tháng sau, bạn cần biết chính xác cái gì đã tạo ra con số này*. Không phải “đại khái cấu hình nào”, mà là chính xác: commit nào, config nào, phiên bản dữ liệu nào, thư viện phiên bản mấy, chạy trên GPU gì. Mọi công cụ trong chương tồn tại vì trí nhớ con người không làm được việc đó, và vì thư mục `runs/exp_final_v2_real_final` không phải một hệ thống.

Nhưng chương này còn một nửa thứ hai, và nửa ấy mới là phần khó. Ngay cả khi bạn đã ghi lại mọi thứ và cố định seed, kết quả *vẫn* không lặp lại chính xác. Người đã debug bắt định trong một hệ thống C++ đa luồng biết ngay tại sao: khi thứ tự các phép toán không được xác định, kết quả số học cũng không được xác định. Trong deep learning, số nguồn gây ra điều đó nhiều hơn hẳn, và phần lớn nằm dưới tầng trừu tượng mà bạn không nhìn thấy.

Học xong chương này thì làm được gì mà trước đó không làm được? Bạn phân biệt được ba tình huống mà người ta hay gộp làm một: *có bug, có bất định nhưng vẫn đúng, và có thay đổi thật nhưng bị nhiễu che*. Ba tình huống ấy đòi ba hành động hoàn toàn khác nhau, và nhằm chúng là cách phổ biến nhất để hoặc đuổi theo một con ma, hoặc công bố một cải tiến không tồn tại.

TRỰC GIÁC

Coi mỗi con số bạn báo cáo là giá trị của một hàm băm với rất nhiều đối số: mã nguồn, config, dữ liệu, phiên bản thư viện, kiến trúc GPU, seed, và cả những thứ bạn không kiểm soát như thứ tự hoàn thành của các luồng. Kỹ thuật tái lập không phải là “làm cho hàm băm trở nên tất định” — điều đó thường bất khả thi và luôn đắt. Nó là hai việc: *ghim càng nhiều đối số càng tốt, và đo phương sai gây ra bởi những đối số còn lại*.

Bạn đã sống qua đúng bài học này. Khi một benchmark SLAM cho kết quả khác nhau giữa các lần chạy, cách chữa không phải là ép mọi thứ tất định rồi đi tiếp. Cách chữa là tách làm hai chế độ. Chế độ ghim chặt — một luồng, thứ tự cố định — *chỉ* dùng để cô lập bug. Chế độ chạy thật lặp nhiều lần dùng để báo cáo, kèm khoảng biến thiên. Chương này là cùng một kiến trúc suy nghĩ, áp cho một hệ thống có nhiều nguồn bất định hơn.

4.2 Ghim các đối số kiểm soát được

A. Bốn lớp phải ghim

Bốn nút thắt của việc truy ngược một con số

1 Vấn đề

siêu tham số rải rác trong code.

Giải pháp

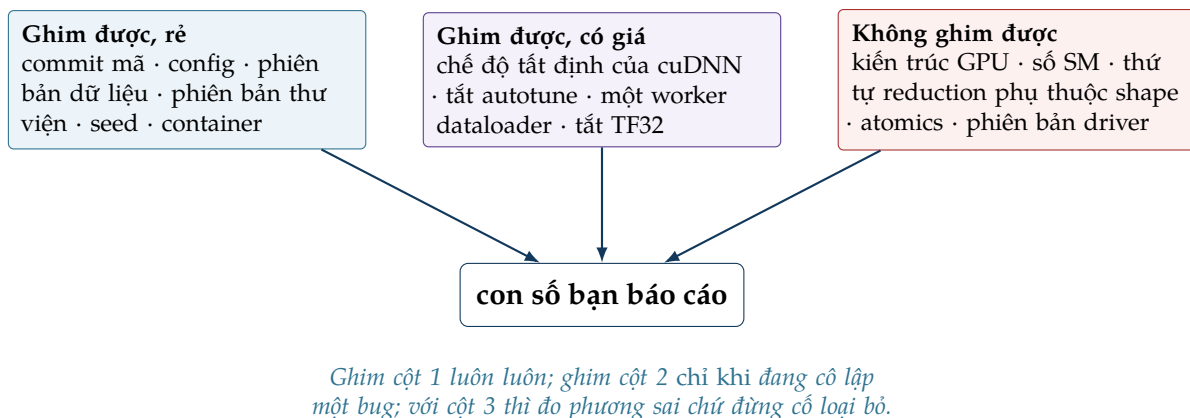
config tập trung (Hydra, hoặc YAML thuần cũng được).

Giả định

bạn tuân thủ nguyên tắc *một thay đổi = một config diff*.

Hệ quả	nếu không diff được thì không ablate được, và toàn bộ E/24-thi-nghiem-va-ablation sụp đổ.
2 Vấn đề	không biết lần chạy nào ứng với mã nào.
Giải pháp	experiment tracking gắn mỗi lần chạy với commit, config và phiên bản dữ liệu — tự động, không dựa vào kỷ luật của con người lúc hai giờ sáng.
3 Vấn đề	vấn đề bị bỏ quên nhiều nhất: dữ liệu thay đổi nhiều hơn mã.
Giải pháp	data versioning.
Giả định	mọi so sánh giữa hai thời điểm chỉ có nghĩa khi bạn biết dữ liệu có đổi hay không.
Vấn đề mới	không có nó thì một cải tiến 2 điểm có thể chỉ là kết quả của việc ai đó dọn lại nhân tuần trước.
4 Vấn đề	checkpoint không đủ để chạy lại.
Giải pháp	lưu artifact đầy đủ: trọng số, config, phiên bản thư viện, phiên bản dữ liệu, và mã tiền xử lý.
Hệ quả	đúng cái tiền xử lý mà chương 21 đã chỉ ra là nguyên nhân tổn giờ người nhất khi export.

Hình 4.1 liệt kê toàn bộ đối số của hàm bấm ấy và chia chúng theo mức độ bạn kiểm soát được. Cách đọc hình: cột trái ghim được bằng công cụ, cột giữa ghim được nhưng phải trả giá, cột phải không ghim được và chỉ có thể đo phương sai.



Hình 4.1. “Chữ ký” của một con số. Sai lầm thường gặp là dồn công sức vào cột 2 — ép tắt định bằng mọi giá — trong khi cột 1 còn chưa ghim đủ và cột 3 thì vốn không thể ghim.

4.3 Các nguồn bất định — và vì sao chế độ tắt định không đủ

Đây là phần cốt lõi của chương và là phần nổi thẳng vào kinh nghiệm bạn đã có. Bảng 4.1 liệt kê các nguồn theo cơ chế, vì hiểu cơ chế thì bạn tự suy ra được nguồn mới khi gặp thư viện lạ.

Bảng 4.1. Các nguồn bất định trong huấn luyện GPU, theo cơ chế.

Nguồn	Cơ chế	Ghim thế nào	Còn lại gì sau khi ghim
Autotune của cuDNN	Chọn kernel bằng cách <i>bấm giờ</i> lúc chạy; máy đang bận thì chọn kernel khác, và kernel khác cộng theo thứ tự khác	<code>cuda.nv.benchmark = False</code>	Chậm hơn ở các shape cố định
Atomics dấu phẩy động	<code>atomicAdd</code> cộng theo thứ tự luồng hoàn thành; cộng số thực không kết hợp nên tổng khác nhau ở bit cuối	<code>use_deterministic_algorithms(True)</code>	Một số op không có bản tắt định → ném lỗi thay vì chạy
Thứ tự reduction	Chia khối phụ thuộc shape và số SM của GPU	Không ghim được	Đổi GPU là đổi kết quả, kể cả khi mọi thứ khác giống hệt
TF32 và fp16	Giảm số bit phần định trị của phép nhân ma trận; kết quả đúng nhưng khác	Tắt TF32, dùng fp32	Chậm hơn nhiều
DataLoader nhiều worker	Mỗi worker có luồng ngẫu nhiên riêng; số worker đổi là chuỗi augmentation đổi	Cố định số worker và seed từng worker	Không còn tự do chỉnh hiệu năng
Thứ tự duyệt tập hợp	Duyệt một set các đối tượng Python, hay <code>glob</code> không <code>sort()</code> , cho thứ tự phụ thuộc địa chỉ bộ nhớ hoặc hệ tệp	Luôn <code>sorted()</code> , luôn khoá theo id ổn định	Không mất gì — đây là lỗi thuần tuý, nên sửa

Dòng cuối đáng dừng lại vì nó là *cùng một lỗi* mà bạn đã gặp ở phía SLAM: một tập hợp được sắp theo địa chỉ con trỏ thì thứ tự duyệt phụ thuộc vào cách bộ cấp phát tình cờ xếp bộ nhớ trong lần chạy đó. Trong C++ nó là `std::set<T*>`; trong Python nó là một set các đối tượng không có `__hash__` tùy biến, hay một `os.listdir` chưa sắp. Điểm chung, và cũng là điểm đáng nhớ nhất: *đây không phải bất định số học mà là một bug*, nên nó không thuộc cột “không ghim được” — nó phải được sửa, và sửa xong thì không mất gì cả.

Từ đó rút ra phát biểu trung thực về giới hạn: tái lập chính xác tới từng bit giữa hai máy khác GPU thường *không khả thi*, vì thứ tự reduction phụ thuộc số SM và cách chia khối. Chế độ tắt định của PyTorch giải quyết được các nguồn trong cùng một máy, không giải quyết được nguồn giữa các máy. Mục tiêu đúng vì thế không phải tái lập từng chữ số mà là tái lập *phân phối* kết quả.

→ **hệ quả của D/19/02** — chế độ tắt định của PyTorch chỉ chặn được các nguồn trong cùng một máy, nên nó mua một đảm bảo hẹp hơn nhiều so với cái tên gợi ra.

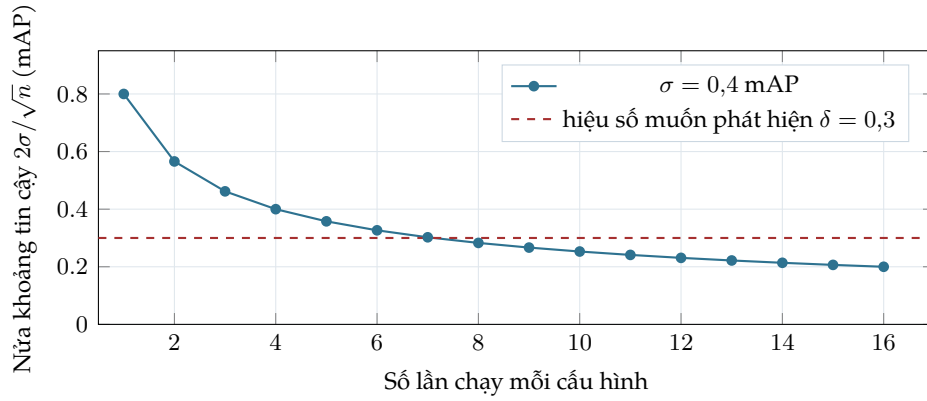
A. Ví dụ nhỏ nhất: bao nhiêu lần chạy thì đủ

Giả sử độ lệch chuẩn giữa các seed của chỉ số bạn quan tâm là $\sigma = 0,4$ điểm mAP — một giá trị không hiếm với dữ liệu vừa. Bạn thấy cấu hình mới hơn cấu hình cũ 0,3 điểm trong một lần chạy mỗi bên. Con số đó nói gì? Gần như không gì cả, vì nó nhỏ hơn cả độ lệch chuẩn của nhiễu.

Chạy n lần mỗi bên thì sai số chuẩn của trung bình là $\sigma/\sqrt{n}$. Muốn khoảng ± 2 sai số chuẩn hẹp hơn hiệu số δ mà bạn muốn phát hiện, cần

$$\frac{2\sigma}{\sqrt{n}} < \delta \iff n > \left(\frac{2\sigma}{\delta}\right)^2.$$

Ý nghĩa: độ nhạy của thí nghiệm tăng theo *căn* số lần chạy, nên muốn phát hiện hiệu số nhỏ đi một nửa thì phải chạy nhiều gấp bốn. Thay số: với $\sigma = 0,4$ và $\delta = 0,3$, ta cần $n > (0,8/0,3)^2 = 7,1$, tức tối thiểu tám lần chạy mỗi cấu hình. Nếu không đủ ngân sách cho tám lần chạy, kết luận đúng không phải “chạy một lần rồi tin”. Kết luận đúng là “hiệu số 0,3 nằm ngoài khả năng phân giải của tôi”. Đó là một kết luận hợp lệ và đáng viết ra. Hình 4.2 cho thấy vì sao.



Hình 4.2. Độ nhảy của một so sánh tăng theo căn bậc hai số lần chạy. Hai đường cắt nhau ở $n \approx 7$: dưới ngưỡng đó, mọi kết luận về hiệu số 0,3 mAP đều là đọc nhiều.

4.4 CI cho mô hình và giám sát sau triển khai

A. Bốn kiểm tra rẻ chạy mỗi commit

Mã học máy có một đặc điểm khó chịu: nó hiếm khi hỏng bằng cách ném lỗi. Vì vậy CI cho mô hình không thể là “chạy unit test” mà phải là một tập kiểm tra nhắm vào các chế độ hỏng im lặng, và cả bốn đều phải rẻ để không ai muốn tắt chúng:

↔ **tiền đề** E/24 — không có config diff được và không có phương sai đo được thì mọi ablation chỉ là kể chuyện; CI là cách ép hai điều kiện đó thành thói quen.

1. **Overfit một batch.** Cho mô hình học thuộc đúng một batch nhỏ; loss phải xuống gần không trong vài chục bước. Nếu không, có lỗi ở gradient, ở nhãn, hoặc ở chỗ nối — và bạn biết ngay trong một phút thay vì sau một đêm huấn luyện.
2. **Kiểm shape và dtype** ở mọi ranh giới khối, đặc biệt sau khi đổi độ phân giải hoặc số lớp.
3. **Kiểm export thành công** sang định dạng triển khai, và so khớp đầu ra với một mẫu cố định. Điều này biến cả chương 21 thành một bài kiểm tra tự động thay vì một sự kiện đau đớn mỗi quý.
4. **Kiểm chỉ số không tụt quá ngưỡng** trên một tập nhỏ cố định. Ngưỡng phải đặt dựa trên phương sai giữa các seed đã đo, nếu không bạn sẽ có một bài kiểm tra hỏng ngẫu nhiên — và một bài kiểm tra hỏng ngẫu nhiên sẽ bị vô hiệu hoá trong hai tuần.

B. Sau triển khai: drift và vòng lặp dữ liệu

Mô hình xuống cấp sau khi triển khai mà không có lỗi nào được ném ra, vì thế giới đổi còn trọng số thì không. Hai việc tối thiểu. Thứ nhất là **giám sát drift**: theo dõi phân phối đầu vào và phân phối đầu ra, chứ không chỉ độ chính xác. Lý do là chân trị thường đến muộn hoặc không bao giờ đến. Thứ hai là **thu thập lỗi làm dữ liệu mới**, tức biến các trường hợp mô hình sai hoặc không chắc thành hàng đợi gắn nhãn.

Bước thứ hai là chỗ Phần D nói ngược về Phần B và đóng vòng của cả tài liệu. Dữ liệu sinh ra mô hình; mô hình chạy trong thực tế; thực tế sinh ra dữ liệu mới. Chất lượng của vòng lặp ấy, chứ không phải kiến trúc, mới quyết định hệ thống tốt lên hay xấu đi theo thời gian.

↔ **hệ quả của** B/08 — drift là dịch chuyển phân phối quan sát từ phía vận hành; thu thập lỗi làm dữ liệu mới là cách đóng vòng lại về khâu dữ liệu.

4.5 Thực hành

Repo và tài nguyên. *Config:* [facebookresearch/hydra](#). *Theo dõi thí nghiệm:* [wandb/wandb](#), [mlflow/mlflow](#), [aimhubio/aim](#). *Phiên bản dữ liệu:* [iterative/dvc](#). *Nổi đánh giá với dữ liệu:* [voxel51/fiftyone](#). *Drift:* [evidentlyai/evidently](#). Danh sách chốt tại tháng 9/2026; nhóm công cụ này thay đổi chậm hơn hầu hết phần còn lại của tài liệu, nhưng các nền tảng theo dõi thí nghiệm dạng dịch vụ thì đến và đi khá nhanh — hãy chọn thứ xuất được dữ liệu ra định dạng mở.

Câu hỏi kiểm tra hiểu. Bạn có một checkpoint từ sáu tháng trước và số đo không tái lập được — liệt kê theo thứ tự sáu thứ có thể đã đổi. Vì sao bật chế độ tắt định không đảm bảo tái lập giữa hai máy khác GPU? Với $\sigma = 0,4$ mAP, hiệu số nhỏ nhất mà bạn có thể phát hiện với năm lần chạy mỗi bên là bao nhiêu?

Mini project (vài giờ đến hai ngày). Chuyển một dự án hiện có sang Hydra cộng một hệ theo dõi thí nghiệm, rồi chạy lại một thí nghiệm cũ *chỉ từ config* và kiểm tra khớp số. Phần đáng giá nhất không phải lúc nó khớp mà lúc nó lệch: mỗi chỗ lệch chỉ ra đúng một đối số của hàm băm mà bạn tưởng đã ghim nhưng chưa. Ghi lại danh sách đó — nó là bản kiểm kê nợ kỹ thuật của chính dự án bạn.

4.6 Giới hạn và trường hợp hỏng

- **Tắt định không phải chính xác.** Một pipeline hoàn toàn tắt định vẫn có thể sai hoàn toàn; tái lập chỉ đảm bảo bạn sai một cách nhất quán. Đây là lỗi phân loại phổ biến nhất trong cả chương.
- **Chế độ tắt định che giấu chính phương sai bạn cần biết.** Nếu bạn chỉ chạy tắt định một seed, bạn không bao giờ đo được độ nhạy của kết quả với khởi tạo — và một phương pháp chỉ hoạt động với một seed là một phương pháp không hoạt động [18].
- **Ghi lại mọi thứ không đồng nghĩa với truy ngược được.** Nếu artifact thiếu mã tiền xử lý hoặc thiếu phiên bản thư viện, bạn có một kho lưu trữ đầy đủ về mặt hình thức mà vô dụng về mặt thực hành.
- **Giám sát drift phát hiện thay đổi, không phát hiện thiệt hại.** Phân phối đầu vào có thể trôi mà hiệu năng không đổi, hoặc ngược lại — đứng yên mà hiệu năng vẫn tụt vì quan hệ giữa đầu vào và nhãn đã đổi. Cảnh báo drift là tín hiệu để đi xem, không phải kết luận.
- **Chi phí vận hành là chi phí thật.** Toàn bộ hạ tầng trong chương cần người bảo trì; một hệ MLOps bị bỏ mặc còn nguy hiểm hơn không có, vì nó tạo cảm giác an toàn sai.

CẠM BẮY

Hai hiểu lầm về `use_deterministic_algorithms(True)`. **Một:** nó không đảm bảo tái lập giữa hai máy khác GPU — thứ tự reduction phụ thuộc số SM và cách chia khối, cả hai đều là thuộc tính phần cứng. Kết quả là bạn trả giá tốc độ để mua một sự đảm bảo hẹp hơn bạn tưởng. **Hai:** một lần chạy tình cờ tắt định không chứng minh được gì. Nếu mô hình của bạn hôm nay không đi qua nhánh nào dùng atomics, hai lần chạy sẽ khớp; thêm một tầng dùng `scatter_add` vào tuần sau là nó hết khớp, và bạn sẽ đi tìm bug ở đúng chỗ không có bug.

CẦN NHỚ

Một con số không kèm phương sai không phải bằng chứng. Quy trình tối thiểu: ghim mọi thứ ghim được rẻ (commit, config, dữ liệu, thư viện, container), *chỉ* bật chế độ tắt định khi đang cô lập một bug, và báo cáo trung bình kèm khoảng biến thiên trên nhiều seed. Và luôn tách ba tình huống: có bug, có bất định mà vẫn đúng, có thay đổi thật bị nhiễu che — ba tình huống ấy đòi ba hành động khác nhau. [T2]

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
●●● Seed	Số khởi tạo mọi bộ sinh ngẫu nhiên: khởi tạo trọng số, thứ tự trộn dữ liệu, augmentation, dropout. Cùng seed chưa đủ để cùng kết quả; khác seed thì chắc chắn khác.
●●● Config	Tập mô tả toàn bộ siêu tham số của một lần chạy, tách khỏi mã. Nguyên tắc: một thay đổi bằng một config diff; không diff được thì không ablate được.
●● Experiment tracking	Ghi tự động mỗi lần chạy kèm commit, config và phiên bản dữ liệu. Phải tự động, vì nó tồn tại chính để bù cho trí nhớ con người lúc hai giờ sáng.
●●● Data versioning	Quản lý phiên bản cho tập dữ liệu như git cho mã. Đây là mục hay bị bỏ quên nhất, và thiếu nó thì mọi so sánh giữa hai thời điểm đều mơ hồ.
●●● Artifact	Gói đủ để tái hiện: trọng số, config, phiên bản thư viện, phiên bản dữ liệu, và mã tiền xử lý. Thiếu một mục là cả gói mất giá trị.
●●● cudnn.benchmark	Chế độ autotune: chọn kernel bằng cách bấm giờ lúc chạy. Máy đang bận thì chọn kernel khác, và kernel khác cộng theo thứ tự khác.
●●● atomicAdd	Cộng dồn song song theo thứ tự luồng hoàn thành. Cộng số thực không có tính kết hợp, nên kết quả lệch ở bit cuối giữa hai lần chạy.
●● TF32	Chế độ nhân ma trộn giảm số bit phần định trị trên GPU Ampere trở lên, bật mặc định. Nhanh hơn nhưng cho số khác fp32 thuần.
●● Vì sao chế độ tắt định không đủ khi đổi máy?	Thứ tự reduction phụ thuộc cách chia khối và số SM, tức phụ thuộc phần cứng. <code>use_deterministic_algorithms</code> chặn được nguồn trong cùng một máy, không chặn được nguồn giữa hai máy.
●● Vì sao cùng seed, cùng máy vẫn ra hai số khác nhau?	Ba nguồn chính: autotune của cuDNN, atomics dấu phẩy động, và thứ tự reduction phụ thuộc cách chia khối. Không nguồn nào nằm trong mã của bạn.
●●● Vì sao thứ tự duyệt tập hợp là bug, không phải nhiễu?	Vì nó phụ thuộc địa chỉ bộ nhớ mà bộ cấp phát tình cờ trả về, chứ không phụ thuộc phần cứng. Sửa bằng <code>sorted()</code> và khoá theo id ổn định; sửa xong không mất gì cả.
●●● Vì sao tái lập phân phối mới là mục tiêu đúng?	Vì ép trùng từng bit xuyên máy thường bất khả thi và luôn đắt. Chạy nhiều lần rồi báo cáo khoảng dao động cho bạn đúng thứ bạn cần: biết hiệu số nào là thật.
●●● $n > (2\sigma/\delta)^2$	Số lần chạy cần để phân giải một hiệu số δ khi độ lệch giữa seed là σ . Với $\sigma = 0,4$ và $\delta = 0,3$ thì cần tám lần mỗi bên.
●● CI cho mô hình	Bốn bài kiểm tra mỗi commit: overfit một batch, kiểm shape, kiểm export và so mẫu cố định, kiểm chỉ số không tụt quá ngưỡng.
●● Drift	Phân phối dữ liệu thật trôi dần khỏi phân phối lúc huấn luyện, làm mô hình xuống cấp mà không ném lỗi nào. Giám sát cả đầu vào lẫn đầu ra, vì chân trị thường đến muộn.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●● Checkpoint sáu tháng tuổi không tái hiện được số cũ?	Thường thiếu mã tiền xử lý và phiên bản thư viện, kể đó là phiên bản dữ liệu. Checkpoint chỉ chứa trọng số.
●● Bài kiểm CI lúc đỏ lúc xanh mà không ai đổi gì?	Ngưỡng đặt không dựa trên phương sai giữa các seed đã đo. Một bài kiểm hòng ngẫu nhiên sẽ bị vô hiệu hoá trong hai tuần.
●● Drift báo động nhưng độ chính xác không đổi?	Phân phối đầu vào đổi mà quan hệ đầu vào–nhân chưa đổi. Cảnh báo drift là tín hiệu để đi xem, không phải kết luận.

●●● phải thuộc ●● nên nắm ● biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
<code>iterative/dvc</code>	Quản lý phiên bản dữ liệu bằng mã băm, để một lần chạy trở tới đúng trạng thái tập dữ liệu	Hạ tầng; nhóm công cụ ổn định nhất trong chương
<code>hydra + mlflow / wandb / aim</code>	Config tập trung cộng truy ngược từ một con số về commit, config và phiên bản dữ liệu	Hạ tầng; nên chọn thứ xuất được dữ liệu ra định dạng mở vì các dịch vụ đến và đi khá nhanh
<code>torch.use_deterministic</code>	Hiện thực chính xác của ranh giới “ghim được có giá” so với “không ghim được” ở Hình 4.1	Tài liệu của nó liệt kê thẳng các op không có bản tắt định — đáng đọc như một bảng kiểm
<code>evidentlyai/evidently</code>	Giám sát drift phân phối đầu vào và đầu ra khi chân trị đến muộn hoặc không đến	Công cụ; ý tưởng bền hơn tên công cụ
<code>voxel51/fiftyone</code>	Nối đánh giá với dữ liệu để biến trường hợp sai thành hàng đợi gắn nhãn — bước khép vòng	Chỗ mà chương 22 gặp lại chương 6 và 7 trong một giao diện

MINI PROJECT SLAM · Ép một lần chạy SLAM có thành phần học tái lập tới từng bit · 1–2 ngày

Mục tiêu	đạt được tái lập bit-for-bit, hoặc chứng minh được vì sao không thể. Hai kết quả có giá trị ngang nhau, miễn là bạn nêu tên từng nguồn bất định chứ không nói “GPU vốn thể”.
Dữ liệu	chính pipeline của mini project chương 21, một chuỗi EuRoC, cấu hình mặc định, không đổi gì khác.
Các bước	chạy mười lần, ghi ATE mỗi lần, tính σ ban đầu. Rồi ghim từng nguồn một và sau mỗi lần ghim chạy lại năm lần để xem σ còn lại bao nhiêu: seed toàn cục; <code>sorted()</code> cho mọi lần duyệt tập hợp, <code>glob</code> và <code>listdir</code> ; một <i>worker</i> dataloader; <code> cudnn.benchmark = False; use_deterministic_algorithms(True); tắt TF32</code> . Ghi cả cái giá về thời gian của từng bước.
Xong khi	mười lần chạy cho ATE giống nhau tới chữ số cuối cùng bạn in ra. Hoặc một bảng ba cột: nguồn bất định, đã cố định được chưa, và giá phải trả hay lý do không cố định được. Lý do phải cụ thể: op nào không có bản tắt định, chỗ nào dùng <code>atomicAdd</code> , chỗ nào phụ thuộc số SM. Kèm một câu dùng được ngay: “chênh lệch ATE nhỏ hơn X cm giữa hai cấu hình là nhiều, không phải cải tiến”.
Nối với	D/22 cho hình chữ ký của một con số và bảng nguồn bất định; E/24 cho cách đọc con số σ đó khi so hai cấu hình; E/26 nếu bạn muốn xếp các mini project này vào một lộ trình học theo mục tiêu.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Chủ yếu là ba thứ: autotune của cuDNN chọn kernel bằng cách bấm giờ nên máy bạn thì chọn khác; atomicAdd cộng theo thứ tự luồng hoàn thành mà cộng số thực không có tính kết hợp; và thứ tự reduction phụ thuộc cách chia khối. Không thứ nào trong ba thứ đó nằm trong mã của bạn.
- A2. Vì tất định không phải chính xác — một pipeline hoàn toàn tất định vẫn có thể sai một cách nhất quán — và vì chạy tất định một seed che mất chính phương sai bạn cần biết. Một phương pháp chỉ hoạt động với một seed là một phương pháp không hoạt động.
- A3. Vì 0,3 nhỏ hơn cả độ lệch chuẩn giữa các seed, vốn thường quanh 0,4 mAP với dữ liệu vừa. Muốn $2\sigma/\sqrt{n} < \delta$ thì cần $n > (0,8/0,3)^2 \approx 7,1$, tức tối thiểu tám lần chạy mỗi bên; không đủ ngân sách thì kết luận đúng là “hiệu số này nằm ngoài khả năng phân giải của tôi”.
- A4. Thường là mã tiền xử lý và phiên bản thư viện, kể đó là phiên bản tập dữ liệu. Checkpoint chỉ chứa trọng số; nếu cách resize hay giá trị chuẩn hoá đã đổi thì mô hình cũ chạy trên dữ liệu mới là một hệ thống khác, dù mọi tệp đều còn nguyên.

NÓI LẠI THẬT ĐƠN GIẢN

Bà bạn có một món ăn ngon. Bà đọc cho bạn công thức, bạn ghi lại, và sáu tháng sau nấu ra một món khác hẳn. Không phải bạn làm sai — chỉ là công thức không ghi loại bột nào, lò của ai, đun mấy phút ở nấc nào, và hôm ấy trời ẩm ra sao.

Đây đúng là tình cảnh của mọi người làm mô hình. Con số đẹp bạn đo hôm nay là kết quả của rất nhiều thứ, phần lớn không nằm trong tệp mã nguồn: phiên bản thư viện, thứ tự dữ liệu, loại máy, và cả những chuyện xảy ra bên trong máy mà bạn không điều khiển được. Máy tính cộng số lẻ theo thứ tự khác nhau thì ra kết quả lệch nhau ở chữ số cuối, và thứ tự đó lại phụ thuộc vào việc lúc ấy máy đang bận thế nào.

Mẹo gồm hai nửa. Nửa thứ nhất là ghi lại mọi thứ ghi được, và ghi tự động chứ đừng trông vào trí nhớ lúc hai giờ sáng: đúng bản mã nào, đúng bộ dữ liệu nào, đúng phiên bản thư viện nào. Nửa thứ hai quan trọng hơn và hay bị bỏ qua: chấp nhận rằng phần còn lại không ghim được, nên hãy nấu năm lần và biết món ăn dao động trong khoảng nào. Khi ai đó bảo cách mới ngon hơn, bạn hỏi được ngay: hơn nhiều hơn khoảng dao động đó không?

Cái giá là công sức: toàn bộ bộ máy ghi chép này cần người duy trì, và một hệ thống ghi chép bị bỏ mặc còn nguy hiểm hơn không có, vì nó tạo cảm giác an toàn giả.

Một câu: Một con số không kèm khoảng dao động không phải bằng chứng — hãy ghim mọi thứ ghim được rẻ, rồi đo phần còn lại thay vì giả vờ nó không tồn tại.

4.7 Kết nối

Chương này là điều kiện cần của E/24-thi-nghiệm-va-ablation: không có config diff được và không có phương sai đo được thì mọi ablation chỉ là kể chuyện. Nó nối ngược về B/06-chat-luong-du-lieu và B/08-dich-chuyen-phan-phoi qua vòng lặp thu thập lỗi, và chính vòng lặp đó là chỗ Phần D khép lại với Phần B. Nó mở rộng mục về tính tất định trong D/19/02-pytorch-thanh-thao từ một đoạn thành một khung suy nghĩ. Và nó là nền cho E/25-do-tin-cay-va-van-hanh: giám sát drift chỉ là một trường hợp riêng của câu hỏi lớn hơn — làm sao biết mô hình đang ở ngoài vùng nó đáng tin.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Bạn có một checkpoint sáu tháng tuổi và số đo không tái lập được — sáu thứ có thể đã đổi, xếp theo xác suất, là những gì?
2. Vì sao bật chế độ tất định không đảm bảo tái lập giữa hai máy khác GPU, và cơ chế phần cứng nào chịu trách nhiệm?

3. Thứ tự duyệt một set các đối tượng thuộc cột “không ghim được” hay cột “phải sửa”, và vì sao phân loại đó quan trọng?
4. Với $\sigma = 0,4$ mAP, cần bao nhiêu lần chạy mỗi cấu hình để phát hiện tin cậy một hiệu số 0,2 mAP, và con số đó nói gì về chi phí của việc theo đuổi các cải tiến nhỏ?
5. Vì sao “overfit một batch” là bài kiểm tra CI đáng giá nhất trong bốn bài, và nó bắt được loại lỗi nào mà kiểm shape không bắt được?
6. Giám sát drift báo động nhưng độ chính xác không đổi — điều đó có nghĩa gì, và ngược lại thì sao?

- [1] Y. Wu and K. He, “Group Normalization,” in *ECCV*, 2018.
- [2] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer Normalization,” *arXiv:1607.06450*, 2016.
- [3] N. Srivastava, G. Hinton, A. Krizhevsky, et al., “Dropout: A Simple Way to Prevent Neural Networks from Overfitting,” *JMLR*, vol. 15, pp. 1929–1958, 2014.
- [4] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986.
- [5] S. Rajbhandari, J. Rasley, O. Ruwase, et al., “ZeRO: Memory Optimizations Toward Training Trillion Parameter Models,” *SC*, 2020.
- [6] P. Goyal, P. Dollár, R. Girshick, et al., “Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour,” *arXiv:1706.02677*, 2017.
- [7] P. Micikevicius, S. Narang, et al., “Mixed Precision Training,” in *ICLR*, 2018.
- [8] G. Hinton, O. Vinyals, and J. Dean, “Distilling the Knowledge in a Neural Network,” in *NeurIPS Deep Learning Workshop*, 2015.
- [9] M. Nagel, M. Fournarakis, R. A. Amjad, et al., “A White Paper on Neural Network Quantization,” *arXiv:2106.08295*, 2021.
- [10] B. Jacob, S. Kligys, B. Chen, et al., “Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference,” in *CVPR*, 2018.
- [11] S. Han, H. Mao, and W. J. Dally, “Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding,” in *ICLR*, 2016.
- [12] J. Frankle and M. Carbin, “The Lottery Ticket Hypothesis: Finding Sparse, Trainable Neural Networks,” in *ICLR*, 2019.
- [13] H. Touvron, M. Cord, M. Douze, et al., “Training Data-Efficient Image Transformers and Distillation Through Attention,” in *ICML*, 2021.
- [14] A. G. Howard, M. Zhu, et al., “MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications,” *arXiv:1704.04861*, 2017.
- [15] M. Tan and Q. V. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” in *ICML*, 2019.
- [16] D. Sculley, G. Holt, D. Golovin, et al., “Hidden Technical Debt in Machine Learning Systems,” *NeurIPS*, 2015.
- [17] N. Bodla, B. Singh, R. Chellappa, et al., “Soft-NMS: Improving Object Detection with One Line of Code,” in *ICCV*, 2017.
- [18] P. Henderson, R. Islam, P. Bachman, et al., “Deep Reinforcement Learning That Matters,” *AAAI*, 2018.

Nguồn

- [1] Y. Wu and K. He, “Group Normalization,” in *ECCV*, 2018.
- [2] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer Normalization,” *arXiv:1607.06450*, 2016.
- [3] N. Srivastava, G. Hinton, A. Krizhevsky, et al., “Dropout: A Simple Way to Prevent Neural Networks from Overfitting,” *JMLR*, vol. 15, pp. 1929–1958, 2014.
- [4] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986.
- [5] S. Rajbhandari, J. Rasley, O. Ruwase, et al., “ZeRO: Memory Optimizations Toward Training Trillion Parameter Models,” *SC*, 2020.
- [6] P. Goyal, P. Dollár, R. Girshick, et al., “Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour,” *arXiv:1706.02677*, 2017.
- [7] P. Micikevicius, S. Narang, et al., “Mixed Precision Training,” in *ICLR*, 2018.
- [8] G. Hinton, O. Vinyals, and J. Dean, “Distilling the Knowledge in a Neural Network,” in *NeurIPS Deep Learning Workshop*, 2015.
- [9] M. Nagel, M. Fournarakis, R. A. Amjad, et al., “A White Paper on Neural Network Quantization,” *arXiv:2106.08295*, 2021.
- [10] B. Jacob, S. Kligys, B. Chen, et al., “Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference,” in *CVPR*, 2018.
- [11] S. Han, H. Mao, and W. J. Dally, “Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding,” in *ICLR*, 2016.
- [12] J. Frankle and M. Carbin, “The Lottery Ticket Hypothesis: Finding Sparse, Trainable Neural Networks,” in *ICLR*, 2019.
- [13] H. Touvron, M. Cord, M. Douze, et al., “Training Data-Efficient Image Transformers and Distillation Through Attention,” in *ICML*, 2021.
- [14] A. G. Howard, M. Zhu, et al., “MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications,” *arXiv:1704.04861*, 2017.
- [15] M. Tan and Q. V. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” in *ICML*, 2019.
- [16] D. Sculley, G. Holt, D. Golovin, et al., “Hidden Technical Debt in Machine Learning Systems,” *NeurIPS*, 2015.
- [17] N. Bodla, B. Singh, R. Chellappa, et al., “Soft-NMS: Improving Object Detection with One Line of Code,” in *ICCV*, 2017.
- [18] P. Henderson, R. Islam, P. Bachman, et al., “Deep Reinforcement Learning That Matters,” *AAAI*, 2018.